

Design System Reference & AI Guide

Every component, template, hook, provider, token, theme and utility — with rendered examples of every variant in light, dark and Arabic RTL, plus ready-to-use prompts for AI assistants.

53
components & templates

3
themes: light · dark · black

LTR/RTL
English & Arabic

20
AI prompts

variant="primary"

Continue

small

Continue

medium

Continue

large

✓ Next >

leading + trailing

○ Saving

loading

Disabled

disabled

variant="secondary"

Continue

small

Continue

medium

Continue

large

✓ Next >

leading + trailing

○ Saving

loading

Disabled

disabled

variant="outline"

Continue

small

Continue

medium

Continue

large

✓ Next >

leading + trailing

○ Saving

loading

Disabled

disabled

variant="ghost"

Continue

small

Continue

medium

Continue

large

✓ Next >

leading + trailing

○ Saving

loading

Disabled

disabled

variant="danger"

Continue

small

Continue

medium

Continue

large

✓ Next >

leading + trailing

○ Saving

loading

Disabled

disabled

variant="link"

[Continue](#)

small

[Continue](#)

medium

[Continue](#)

large

✓ Next >

leading + trailing

○ Saving

loading

Disabled

disabled

Generated October 2026 from the source in [src/design-system](#). Screenshots are real renders of the components (react-native-web, Chromium). Native-only pieces — the platform date grid — are shown with a web stand-in.

Contents

1. [Getting started](#)
2. [Architecture & the RNTheme boundary](#)
3. [Foundations: tokens & themes](#)
4. [Providers, hooks & utilities](#)
5. [Responsive layout & your own tokens](#)
6. [Components](#)
 - **Typography & icons:** [Text](#), [Heading](#), [Icon](#)
 - **Actions:** [Button](#), [IconButton](#), [Link](#), [ActionText](#)
 - **Forms:** [Form](#), [FormField](#)
 - **Text & number inputs:** [Input](#), [PasswordInput](#), [SearchInput](#), [TextArea](#), [AmountInput](#), [AmountWithCurrencyInput](#), [PhoneInput](#), [AmountField](#), [OTPInput](#), [FileInput](#)
 - **Choice & date controls:** [Checkbox](#), [CheckboxGroup](#), [Radio](#), [RadioGroup](#), [Switch](#), [Select](#), [ChipsGroup](#), [RadioImageGroup](#), [BoxGroup](#), [SwatchGroup](#), [Slider](#), [DatePicker](#), [DateRangePicker](#)
 - **Data display:** [Badge](#), [Chip](#), [Avatar](#), [Divider](#), [Card](#), [ListItem](#), [List](#)
 - **Feedback & status:** [Alert](#), [Toast](#), [ProgressBar](#), [Spinner](#), [Skeleton](#), [EmptyState](#), [ErrorState](#)
 - **Disclosure & navigation:** [Accordion](#), [Tabs](#), [Tooltip](#)
 - **Overlays:** [Modal](#), [BottomSheet](#)
 - **Templates:** [NumPad](#), [OTPTemplate](#)
7. [AI prompts](#)
8. [Appendix: type aliases & complete export list](#)

1 • Getting started

Install

```
# inside the design-system repo: build a tarball (npm i && npm pack; prepack rebuilds dist/)
npm run pack

# in your app
npm i ../Bunyan-Design/bunyan-design-system-0.0.1.tgz
npm i react-native-svg @react-native-community/datetimepicker formik # Yup ships inside Bunyan
```

Peer dependencies: `react ≥ 18.2`, `react-native ≥ 0.74`, `react-native-svg ≥ 15`, `@react-native-community/datetimepicker ≥ 8`, `formik ≥ 2.2`. Yup is a regular dependency of Bunyan — import it as `import { Yup } from '@bunyan/design-system'`. Metro resolves the `"react-native": "src/index.ts"` entry (TypeScript source); other bundlers use `dist/`.

Wrap your app

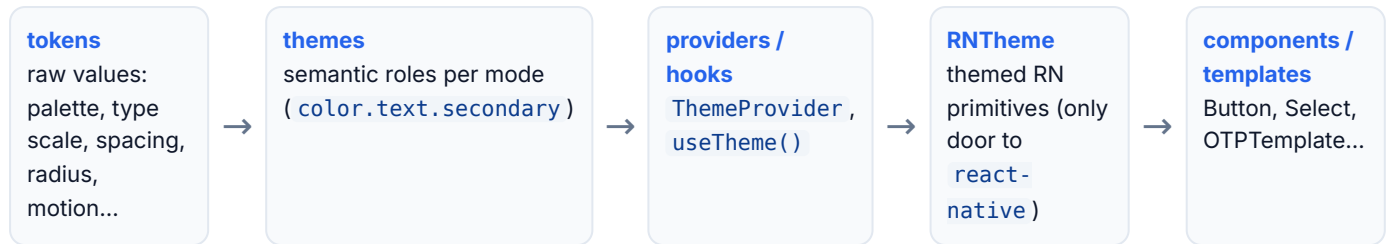
```
import { ThemeProvider, ToastProvider } from '@bunyan/design-system';

export default function App() {
  return (
    <ThemeProvider initialPreference="system" locale="ar-SA" blackForSystemDark>
      <ToastProvider maxVisible={3}>
        <RootNavigator />
      </ToastProvider>
    </ThemeProvider>
  );
}
```

Scripts

<code>npm run pack</code>	<code>npm i && npm pack</code> — installs dependencies, then <code>prepack</code> cleans and rebuilds <code>dist/</code> and writes <code>bunyan-design-system-<version>.tgz</code> .
<code>npm run verify</code>	Typecheck + Jest (includes the test that fails if any file other than <code>RNTheme/native.ts</code> imports <code>react-native</code>).
<code>npm run build / clean</code>	Compile <code>src</code> to <code>dist</code> / delete <code>dist</code> .
<code>npm test, test:watch, test:coverage</code>	Jest with <code>@testing-library/react-native</code> .
<code>npm run storybook[:ios :android :web]</code>	On-device Storybook via the Expo app in <code>showcase/</code> .

2 · Architecture & the RNTheme boundary



Dependencies only flow left to right. Components never import product code, and design-system code never imports `react-native` directly: `components/RNTheme/native.ts` is the single boundary file and a test enforces it.

```
// inside a component
import { Pressable, View, type ViewRef } from '../RNTheme';

// in app code
import { RNTheme, createThemedComponent } from '@bunyan/design-system';
<RNTheme.View themeStyle={({ theme }) => ({ padding: theme.spacing.lg })} />

const Surface = createThemedComponent(RNTheme.View, {
  displayName: 'Surface',
  baseStyle: ({ theme }) => ({ backgroundColor: theme.color.surface.primary, borderRadius:
theme.radius.lg }),
});
```

Themed primitives

PRIMITIVE	THEME DEFAULTS
<code>View</code> , <code>Pressable</code> , <code>Image</code> , <code>RNModal</code>	Layout-neutral; add <code>themeStyle</code> and ref forwarding.
<code>RNText</code>	Text colour, direction-aware font family, <code>writingDirection</code> , font scaling capped at 2×.
<code>TextInput</code>	Text, placeholder, selection and cursor colours; font family; font scaling.
<code>ScrollView</code>	Indicator style per mode; <code>keyboardShouldPersistTaps="handled"</code> .
<code>ActivityIndicator</code>	Primary colour.
<code>RNSwitch</code>	Track colours.
<code>KeyboardAvoidingView</code>	<code>behavior="padding"</code> on iOS.
APIs	<code>Animated</code> , <code>Platform</code> , <code>Linking</code> , <code>StyleSheet</code> , <code>I18nManager</code> , <code>AccessibilityInfo</code> , <code>Appearance</code> ; types <code>ViewStyle</code> , <code>TextStyle</code> , <code>PressableProps</code> ...; refs <code>ViewRef</code> , <code>TextRef</code> , <code>TextInputRef</code> , <code>ScrollViewRef</code> .

createThemedComponent options

PROP	TYPE	DEFAULT
<code>displayName</code> required	<code>string</code> Shown in React DevTools and error messages.	—
<code>defaultProps</code>	<code>(context: ThemeContextValue) => Partial<P></code> Theme-aware props applied first; anything passed explicitly wins.	—

PROP	TYPE	DEFAULT
<code>baseStyle</code>	<code>(context: ThemeContextValue) => ThemedStaticStyle<P></code> Theme-aware base style; merged underneath the caller's <code>style</code> .	—

Style layers merge as `baseStyle` → `themeStyle` → `style` (explicit `style` wins). Pressable-style `(state) => style` functions keep working, and an explicit `undefined` prop never erases a theme default.

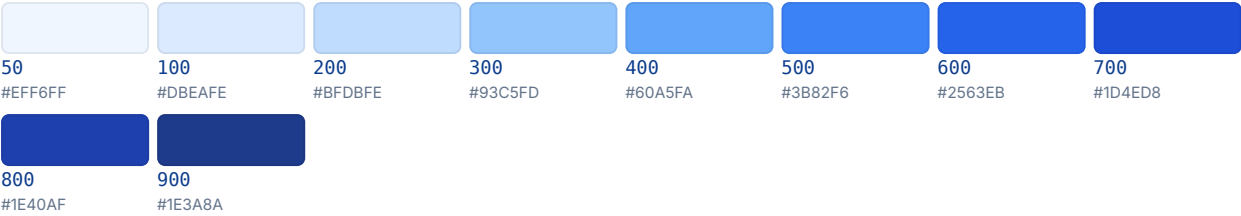
RTL model

Direction is provider-scoped: `locale="ar-SA"` (or `direction="rtl"`) makes a subtree RTL without touching the global `I18nManager`. React Native already mirrors `row` and swaps `left` / `right` when the app runs natively in RTL, so `logicalRow`, `logicalText` and `logicalAlignItems` only mirror when the requested direction differs from the native one. Always use them instead of hard-coding `row-reverse`.

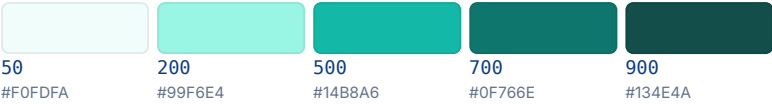
3 · Foundations: tokens & themes

Palette (`tokens.palette`)

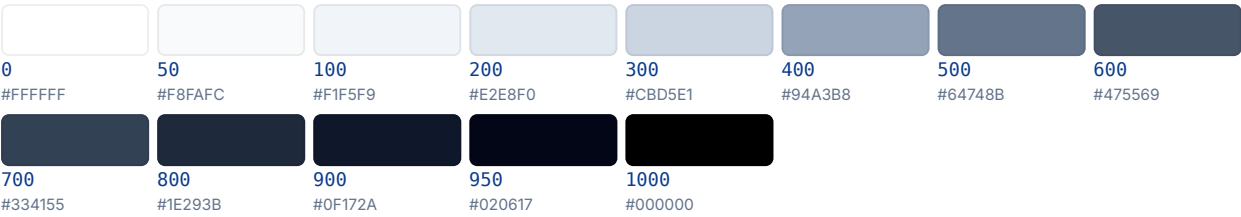
blue



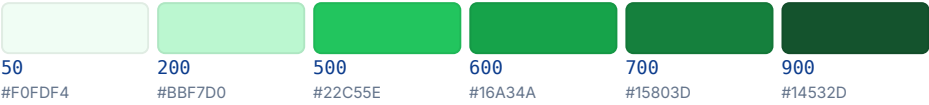
teal



gray



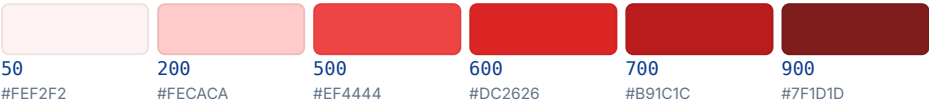
green



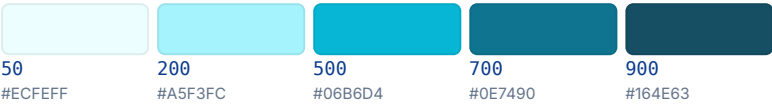
amber



red



cyan



Semantic colours (`theme.color`)

TOKEN	LIGHT	DARK	BLACK
color.primary.default	#2563EB	#60A5FA	#60A5FA
color.primary.hover	#1D4ED8	#93C5FD	#93C5FD
color.primary.pressed	#1E40AF	#BFD8FE	#BFD8FE
color.primary.subtle	#EFF6FF	#1E3A8A	#1E3A8A
color.primary.contrast	#FFFFFF	#020617	#020617
color.secondary.default	#0F766E	#14B8A6	#14B8A6
color.secondary.hover	#134E4A	#99F6E4	#99F6E4
color.secondary.pressed	#134E4A	#99F6E4	#99F6E4

TOKEN	LIGHT	DARK	BLACK
color.secondary.subtle	 #F0FDF4	 #134E4A	 #134E4A
color.secondary.contrast	 #FFFFFF	 #020617	 #020617
color.neutral.default	 #475569	 #CBD5E1	 #CBD5E1
color.neutral.subtle	 #F1F5F9	 #1E293B	 #1E293B
color.neutral.strong	 #0F172A	 #F8FAFC	 #F8FAFC
color.neutral.contrast	 #FFFFFF	 #020617	 #020617
color.success.default	 #16A34A	 #22C55E	 #22C55E
color.success.subtle	 #F0FDF4	 #14532D	 #14532D
color.success.border	 #BBF7D0	 #15803D	 #15803D
color.success.text	 #14532D	 #BBF7D0	 #BBF7D0
color.warning.default	 #D97706	 #F59E0B	 #F59E0B
color.warning.subtle	 #FFFBEF	 #78350F	 #78350F
color.warning.border	 #FDE68A	 #D97706	 #D97706
color.warning.text	 #78350F	 #FDE68A	 #FDE68A
color.error.default	 #DC2626	 #EF4444	 #EF4444
color.error.subtle	 #FEF2F2	 #7F1D1D	 #7F1D1D
color.error.border	 #FECACA	 #B91C1C	 #B91C1C
color.error.text	 #7F1D1D	 #FECACA	 #FECACA
color.information.default	 #0E7490	 #06B6D4	 #06B6D4
color.information.subtle	 #ECFEFF	 #164E63	 #164E63
color.information.border	 #A5F3FC	 #0E7490	 #0E7490
color.information.text	 #164E63	 #A5F3FC	 #A5F3FC
color.background.primary	 #FFFFFF	 #0F172A	 #000000
color.background.secondary	 #F8FAFC	 #020617	 #020617
color.background.inverse	 #020617	 #FFFFFF	 #FFFFFF
color.surface.primary	 #FFFFFF	 #1E293B	 #020617
color.surface.secondary	 #F8FAFC	 #0F172A	 #000000
color.surface.elevated	 #FFFFFF	 #334155	 #0F172A
color.surface.inverse	 #0F172A	 #FFFFFF	 #FFFFFF
color.border.primary	 #CBD5E1	 #475569	 #334155
color.border.secondary	 #E2E8F0	 #334155	 #1E293B
color.border.focus	 #2563EB	 #60A5FA	 #60A5FA
color.border.error	 #DC2626	 #EF4444	 #EF4444
color.border.success	 #16A34A	 #22C55E	 #22C55E
color.text.primary	 #0F172A	 #F8FAFC	 #F8FAFC
color.text.secondary	 #475569	 #CBD5E1	 #CBD5E1
color.text.tertiary	 #64748B	 #94A3B8	 #94A3B8
color.text.inverse	 #FFFFFF	 #020617	 #020617
color.text.link	 #1D4ED8	 #93C5FD	 #93C5FD
color.disabled.background	 #F1F5F9	 #1E293B	 #1E293B
color.disabled.border	 #E2E8F0	 #334155	 #334155
color.disabled.text	 #94A3B8	 #64748B	 #64748B
color.overlay.scrim	 rgba(2, 6, 23, 0.64)	 rgba(0, 0, 0, 0.72)	 rgba(0, 0, 0, 0.72)

TOKEN	LIGHT	DARK	BLACK
color.overlay.subtle	☒ rgba(2, 6, 23, 0.12)	☐ rgba(255, 255, 255, 0.12)	☐ rgba(255, 255, 255, 0.12)
color.overlay.transparent	☐ transparent	☐ transparent	☐ transparent

Typography (`tokens.typography`)

xs	12px / 16px	Bunyan بنيان
sm	14px / 20px	Bunyan بنيان
md	16px / 24px	Bunyan بنيان
lg	18px / 28px	Bunyan بنيان
xl	20px / 30px	Bunyan بنيان
xxl	24px / 34px	Bunyan بنيان
displaySm	30px / 38px	Bunyan بنيان
displayMd	36px / 44px	Bunyan بنيان

Font families: `sans: System`, `arabic: System`, `mono: Courier`.
Weights: `regular: 400`, `medium: 500`, `semibold: 600`, `bold: 700`.
Letter spacing: `tight: -0.4`, `normal: 0`, `wide: 0.4`.

spacing

spacing.none	0px	
spacing.xxs	2px	
spacing.xs	4px	
spacing.sm	8px	■
spacing.md	12px	■
spacing.lg	16px	■
spacing.xl	20px	■
spacing.xxl	24px	■
spacing.xxxl	32px	■
spacing.giant	40px	■
spacing.huge	48px	■

radius

radius.none	0px	
radius.xs	2px	
radius.sm	4px	
radius.md	8px	
radius.lg	12px	
radius.xl	16px	
radius.pill	999px	

iconSize

iconSize.xs	12px
iconSize.sm	16px
iconSize.md	20px
iconSize.lg	24px
iconSize.xl	32px
iconSize.xxl	40px

componentHeight

componentHeight.xs	28px
componentHeight.sm	36px
componentHeight.md	44px
componentHeight.lg	52px
componentHeight.xl	60px

breakpoint

breakpoint.compact	0px
breakpoint.medium	600px
breakpoint.expanded	1024px
breakpoint.wide	1440px

opacity

opacity.invisible	0
opacity.subtle	0.08

borderWidth

<code>borderWidth.none</code>	0px
<code>borderWidth.thin</code>	1px
<code>borderWidth.medium</code>	2px
<code>borderWidth.thick</code>	3px

<code>opacity.muted</code>	0.4
<code>opacity.disabled</code>	0.5
<code>opacity.strong</code>	0.72
<code>opacity.opaque</code>	1

zIndex

<code>zIndex.base</code>	0
<code>zIndex.raised</code>	10
<code>zIndex.dropdown</code>	100
<code>zIndex.sticky</code>	200
<code>zIndex.overlay</code>	300
<code>zIndex.modal</code>	400
<code>zIndex.toast</code>	500
<code>zIndex.tooltip</code>	600

motion

<code>motion.duration.instant</code>	0ms
<code>motion.duration.fast</code>	120ms
<code>motion.duration.normal</code>	200ms
<code>motion.duration.slow</code>	320ms
<code>motion.easing.standard</code>	[0.2, 0, 0, 1]
<code>motion.easing.emphasized</code>	[0.2, 0, 0, 1]
<code>motion.easing.entrance</code>	[0, 0, 0, 1]
<code>motion.easing.exit</code>	[0.3, 0, 1, 1]

shadow

<code>shadow.none</code>	offset 0, radius 0, opacity 0, elevation 0
<code>shadow.sm</code>	offset 1, radius 2, opacity 0.08, elevation 1
<code>shadow.md</code>	offset 4, radius 8, opacity 0.12, elevation 4
<code>shadow.lg</code>	offset 10, radius 20, opacity 0.16, elevation 8

4 · Providers, hooks & utilities

ThemeProvider

PROP	TYPE	DEFAULT
<code>children</code> required	<code>React.ReactNode</code>	—
<code>initialPreference</code>	<code>'light' 'dark' 'black' 'system'</code>	<code>'system'</code>
<code>locale</code>	<code>string</code>	<code>'en'</code>
<code>direction</code>	<code>'ltr' 'rtl'</code>	—
<code>blackForSystemDark</code>	<code>boolean</code>	<code>false</code>
<code>themes</code>	<code>Partial<Record<'light' 'dark' 'black', Theme>></code> Your own themes (e.g. from <code>createThemes</code>). Modes you leave out fall back to Bunyan's defaults. Memoise the object (or define it at module level).	—
<code>translate</code>	<code>TranslateFn</code> Translate function for <code>{ localeKey }</code> texts across Bunyan (Form labels, ChipsGroup items, error messages...). E.g. <code>translate={i18n.t}</code> . Keep it stable.	<code>defaultTranslate</code>

Locale direction: RTL for `ar`, `ckb`, `dv`, `fa`, `he`, `ps`, `sd`, `ug`, `ur`, `yi` (see `isRTLLocale`) or when the native layout is already RTL. With `initialPreference="system"` the provider follows OS appearance changes live.

useTheme()

```
const { theme, mode, preference, setPreference, direction, isRTL, locale } = useTheme();
```

PROP	TYPE	DEFAULT
<code>theme</code> required	<code>Theme</code>	—
<code>mode</code> required	<code>'light' 'dark' 'black'</code>	—
<code>preference</code> required	<code>'light' 'dark' 'black' 'system'</code>	—
<code>setPreference</code> required	<code>(preference: 'light' 'dark' 'black' 'system') => void</code>	—
<code>direction</code> required	<code>'ltr' 'rtl'</code>	—
<code>isRTL</code> required	<code>boolean</code>	—
<code>locale</code> required	<code>string</code>	—
<code>translate</code> required	<code>TranslateFn</code> Translate function used for <code>{ localeKey }</code> texts. Identity when not configured.	—

Works without a provider too (falls back to the light theme, LTR, `en`).

useToast() / ToastProvider

```
const { showToast, dismissToast, dismissAll } = useToast();
const id = showToast({ message: 'Saved', tone: 'success', duration: 4000 });
```

Throws if used outside [ToastProvider](#) . See the Toast component page for options.

Theme objects

`lightTheme`, `darkTheme`, `blackTheme` and `themes[mode]` share one token contract (`Theme`): `mode`, `color`, `typography`, `spacing`, `radius`, `borderWidth`, `shadow`, `iconSize`, `componentHeight`, `breakpoint`, `motion`, `opacity`, `zIndex` .

Style utilities ([utilities/styles](#))

EXPORT	SIGNATURE / NOTES
<code>logicalRow</code>	<code>logicalRow = (direction: Direction): ViewStyle => ({ flexDirection: needsMirroring(direction) ? 'row-reverse' : 'row', })</code>
<code>logicalText</code>	<code>logicalText = (direction: Direction, align: LogicalAlign = 'start'): TextStyle => ({ textAlign: logicalTextAlign(direction, align), writingDirection: direction, })</code>
<code>logicalTextAlign</code>	<code>logicalTextAlign = (direction: Direction, align: LogicalAlign = 'start'): TextStyle['textAlign']</code>
<code>logicalAlignItems</code>	<code>logicalAlignItems = (direction: Direction, align: Exclude<LogicalAlign, 'center'>): ViewStyle['alignItems']</code> <code>alignItems</code> value that places children at the logical start or end.
<code>needsMirroring</code>	<code>needsMirroring = (direction: Direction) => direction !== nativeDirection()</code> React Native already mirrors <code>row</code> and swaps <code>left</code> / <code>right</code> text alignment when the app runs natively in RTL. Bunyan's direction is provider-scoped, so we only mirror when the requested direction differs from the native one — otherwise RTL apps would be flipped twice and render left-to-right.
<code>nativeDirection</code>	<code>nativeDirection = (): Direction => (I18nManager.isRTL ? 'rtl' : 'ltr')</code> Direction the native layout engine is already using (set via <code>I18nManager</code>).
<code>heightForSize</code>	<code>heightForSize = (theme: Theme, size: ComponentSize) => ({ small: theme.componentHeight.sm, medium: theme.componentHeight.md, large: theme.componentHeight.lg, })[size]</code>
<code>horizontalPaddingForSize</code>	<code>horizontalPaddingForSize = (theme: Theme, size: ComponentSize) => ({ small: theme.spacing.md, medium: theme.spacing.lg, large: theme.spacing.xl, })[size]</code>
<code>iconSizeForComponent</code>	<code>iconSizeForComponent = (theme: Theme, size: ComponentSize) => ({ small: theme.iconSize.sm, medium: theme.iconSize.md, large: theme.iconSize.lg, })[size]</code>
<code>iconTokenForSize</code>	<code>iconTokenForSize = (size: ComponentSize) => (size === 'small' ? 'sm' : size === 'large' ? 'lg' : 'md') as 'sm' 'md' 'lg'</code> Icon size token name matching a component size.
<code>statusBorderColor</code>	<code>statusBorderColor = (theme: Theme, status: FeedbackStatus)</code>
<code>fontFamilyFor</code>	<code>fontFamilyFor = (theme: Theme, direction: Direction) => direction === 'rtl' ? theme.typography.fontFamily.arabic : theme.typography.fontFamily.sans</code>
<code>mergeIds</code>	<code>mergeIds = (...ids: Array<string undefined>) => ids.filter(Boolean).join('') undefined</code>

EXPORT

SIGNATURE / NOTES

isRTLLocale	isRTLLocale = (locale: string) => RTL_LOCALE.test(locale)
mergeThemedStyles	mergeThemedStyles = (...layers: AnyStyle[]): AnyStyle Merges style layers, flattening undefined layers away. When any layer is a Pressable-style resolver the result is a resolver as well, so interaction state keeps working through the themed wrapper.
DIRECTIONAL_ICONS	DIRECTIONAL_ICONS: ReadonlySet<IconName> = new Set<IconName>(['backspace', 'chevron-left', 'chevron-right']) Icons whose meaning depends on reading direction and should mirror in RTL.

5 · Responsive layout & your own tokens

Sizes stay fixed in points; **layout** changes at breakpoints. A few values may grow with the screen through clamped scale helpers. Plain JavaScript, no native module, iOS/Android/web, updates live on rotation, split-screen and window resize. Full guide: [docs/RESPONSIVE.md](#).

Cheat sheet

KEYWORD	WHAT IT DOES · EXAMPLE
<code>useResponsive()</code>	All tools below, re-rendering on window changes · <code>const r = useResponsive();</code>
<code>r.breakpoint</code>	Current breakpoint · <code>'compact' 'medium' 'expanded' 'wide'</code>
<code>r.up(bp) / r.down(bp)</code>	At least / narrower than <code>bp</code> · <code>r.up('medium')</code> → tablet and larger
<code>r.between(a, b) / r.is(bp)</code>	Range / exact breakpoint
<code>r.select({...})</code>	Value per breakpoint, mobile-first, optional <code>default</code> · <code>r.select({ compact: 1, medium: 2, expanded: 3 })</code>
<code>r.s(n) / r.vs(n)</code>	Scale with the short / long window side (icons, illustrations, heights)
<code>r.ms(n, f?) / r.mvs(n, f?)</code>	Moderate scale (default half of <code>s</code>) — fonts and radii · <code>r.ms(theme.typography.fontSize.xl)</code>
<code>r.isPortrait / r.isLandscape</code>	Orientation
<code>useBreakpoint() · useResponsiveValue({...})</code>	Just the name · just one value
<code>makeStyles((theme, r) => ({...}))</code>	Memoised theme- and size-aware StyleSheet hook
<code>s, vs, ms, mvs</code>	Static helpers (read the window when called, no re-render) for module-level styles. Long names: <code>scale</code> , <code>verticalScale</code> , <code>moderateScale</code> , <code>moderateVerticalScale</code>
<code>getResponsive({ width, height })</code>	Pure tools for an explicit size (tests, previews, SSR)
<code>List</code> props	<code>columns</code> , <code>spacing</code> , <code>dividerSpacing</code> accept breakpoint maps · <code>columns={{ compact: 1, medium: 2, expanded: 4 }}</code>

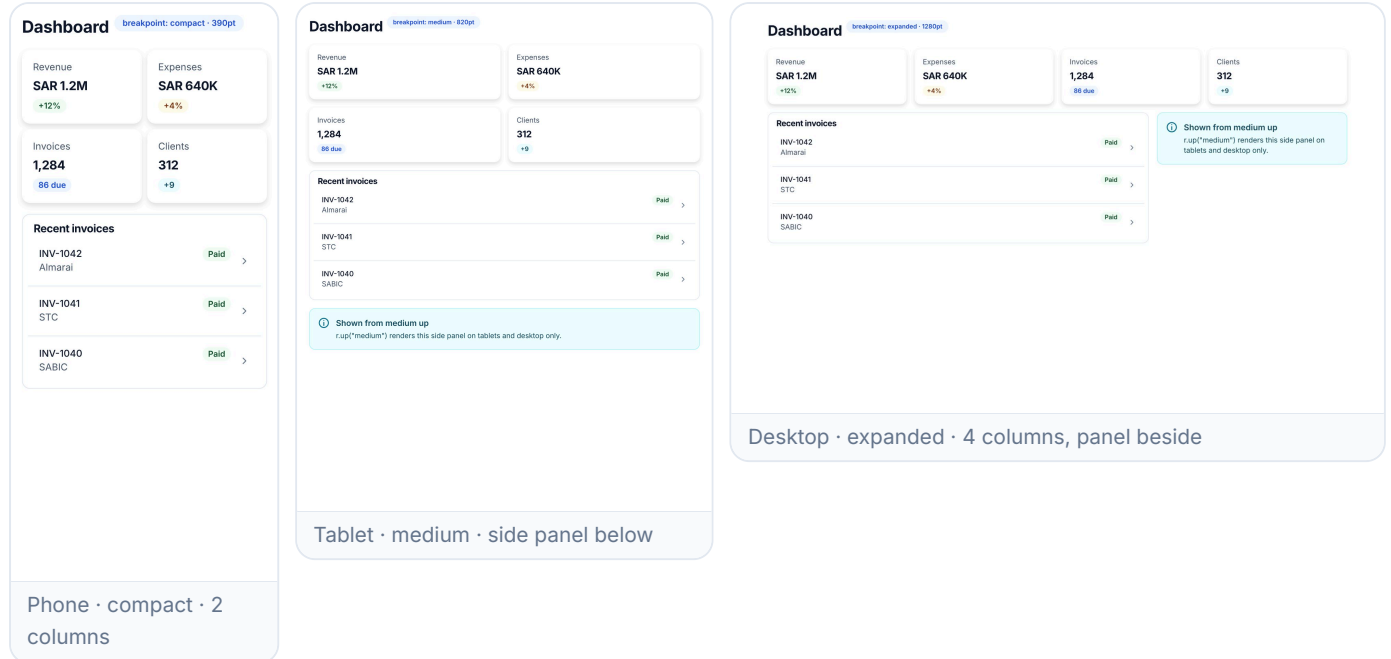
Defaults: theme breakpoints (`compact` 0, `medium` 600, `expanded` 1024, `wide` 1440), 375 × 812 design canvas, scale ratio clamped to 0.85 – 1.35 so small phones keep 44 pt targets and tablets never look like zoomed phones.

```
const useStyles = makeStyles((theme, r) => ({
  page: { padding: r.select({ compact: theme.spacing.lg, medium: theme.spacing.xxl }), maxWidth: 1200, alignSelf: 'center' },
  title: { fontSize: r.ms(theme.typography.fontSize.xxl) },
}));

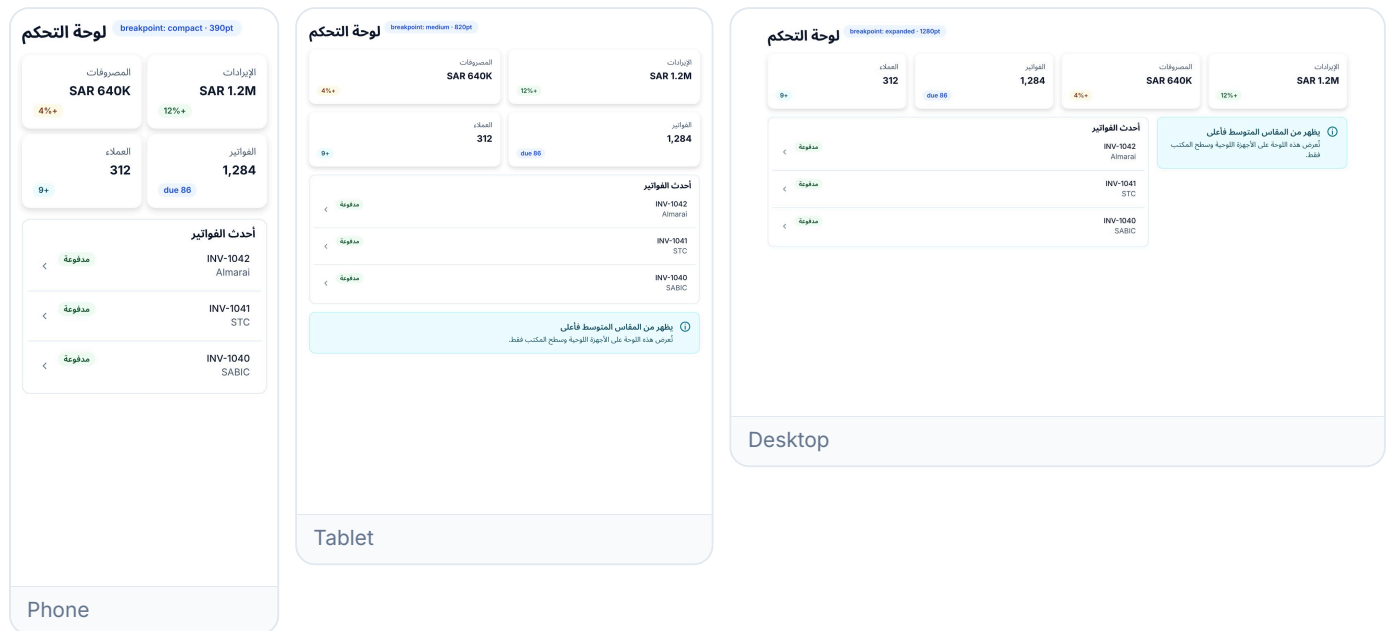
function Dashboard({ stats }: Props) {
  const styles = useStyles();
  const r = useResponsive();
  return (
    <RNTheme.View style={styles.page}>
      <List Component={Card} data={stats} formatItem={toCard} columns={{ compact: 2, expanded: 4 }} />
    </RNTheme.View>
  );
}
```

```
spacing={{ compact: 'sm', medium: 'lg' }} />
    {r.up('medium') ? <SidePanel /> : null}
  </RNTheme.View>
);
}
```

The same screen at 390 pt (compact), 820 pt (medium) and 1280 pt (expanded)



Arabic · RTL



Your own tokens

1 · Theme values — `createTheme` / `createThemes`

Override only what differs; token *names* stay type-checked, *values* are yours. Components, `useResponsive()` and `List` follow them — including your breakpoints.

```
export const brandThemes = createThemes({
  shared: { typography: { fontFamily: { sans: 'Inter', arabic: 'IBMPlexSansArabic' } }, spacing: {
    md: 14 }, breakpoint: { medium: 640 } },
```

```
light: { color: { primary: { default: '#6D28D9', pressed: '#5B21B6', subtle: '#F5F3FF' } } },
dark: { color: { primary: { default: '#A78BFA', pressed: '#C4B5FD', subtle: '#2E1065' } } },
});
```

```
<ThemeProvider themes={brandThemes} initialPreference="system">...</ThemeProvider>
```

2 · Breakpoint names and design canvas — `createResponsive`

```
// src/responsive.ts in your app – import the helpers from here everywhere
export const { useResponsive, useBreakpoint, useResponsiveValue, makeStyles, s, vs, ms, mvs } =
createResponsive({
  breakpoints: { phone: 0, tablet: 768, desktop: 1200 }, // fully typed: r.up('tablet'), r.select({
phone, desktop })
  guidelineWidth: 390, guidelineHeight: 844, // your Figma frame
  minScale: 0.9, maxScale: 1.25, moderateFactor: 0.4,
});
```

OPTION	DEFAULT · MEANING
<code>breakpoints</code>	theme <code>breakpoint</code> tokens · name → minimum width
<code>guidelineWidth</code> / <code>guidelineHeight</code>	375 / 812 · size of the design canvas
<code>minScale</code> / <code>maxScale</code>	0.85 / 1.35 · clamp for the scale ratio
<code>moderateFactor</code>	0.5 · share of the scale used by <code>ms</code> / <code>mvs</code>
<code>roundToPixel</code>	true · round to the nearest physical pixel

Change layout with breakpoints; don't scale everything. Use `ms` for headline sizes and radii, keep body text and spacing tokens fixed — text already follows the user's accessibility size (capped at 2×).

Typography & icons

1 Text

Semantic body, label, caption, and code text.

```
import { Text } from '@bunyan/design-system';
```

Variants & states `body`, `bodySmall`, `caption`, `label`, `code`; tone and weight APIs; inherits native selection and truncation states.

Accessibility Font scaling is enabled with a 2× multiplier; direction follows the provider.

Example `<Text variant="bodySmall" tone="secondary">Updated today</Text>`.

Do/don't Use tone names; do not pass raw color or font styles.

Edge cases Use `numberOfLines` for constrained layouts and test at maximum font size.

TextProps

PROP	TYPE	DEFAULT
<code>variant</code>	'body' 'bodySmall' 'caption' 'label' 'code'	'body'
<code>tone</code>	'primary' 'secondary' 'tertiary' 'inverse' 'link' 'error' 'success'	'primary'
<code>align</code>	'start' 'center' 'end'	'start'
<code>weight</code>	'regular' 'medium' 'semibold' 'bold'	'regular'

Also accepts: `RNTextProps` (native React Native props).

variant

The quick brown fox
body

The quick brown fox
bodySmall

The quick brown fox
caption

The quick brown fox
label

const x = 1;
code

tone

Sample
primary

Sample
secondary

Sample
tertiary

Sample
link

Sample
error

Sample
success

Sample
inverse

weight

Weight
regular

Weight
medium

Weight
semibold

Weight
bold

align

Aligned
start

Aligned
center

Aligned
end

All variants — light theme

variant

The quick brown fox

The quick brown fox

The quick brown fox

The quick brown fox

const x = 1;

body

bodySmall

caption

label

code

tone

Sample

Sample

Sample

Sample

Sample

Sample

primary

secondary

tertiary

link

error

success

Sample

inverse

weight

Weight

Weight

Weight

Weight

regular

medium

semibold

bold

align

Aligned

Aligned

Aligned

start

center

end

Dark theme

variant

نص تجريبي للعرض

نص تجريبي للعرض

نص تجريبي للعرض

نص تجريبي للعرض

نص تجريبي للعرض

نص تجريبي للعرض

body

bodySmall

caption

label

code

code

tone

عينة

عينة

عينة

عينة

عينة

عينة

primary

secondary

tertiary

link

error

success

عينة

inverse

weight

الوزن

الوزن

الوزن

الوزن

regular

medium

semibold

bold

align

محاذة

محاذة

محاذة

start

center

end

Arabic (ar-SA) · RTL

2 Heading

Establishes an accessible content hierarchy.

```
import { Heading } from '@bunyan/design-system';
```

- Variants & states**
- Levels 1–6 map to the type scale.
- Accessibility**
- Uses the header role and web `aria-level`.
- Example**
- `<Heading level={2}>Account summary</Heading>`.
- Do/don't**
- Keep levels sequential; do not choose a level only for visual size.
- Edge cases**
- Long localized headings should wrap; avoid truncating critical titles.

HeadingProps

PROP	TYPE	DEFAULT
<code>level</code>	1 2 3 4 5 6	2
<code>tone</code>	'primary' 'secondary' 'tertiary' 'inverse' 'link' 'error' 'success'	'primary'
<code>align</code>	'start' 'center' 'end'	'start'

Also accepts: `RNTextProps` (native React Native props).

level

Heading level 1

Heading level 2

Heading level 3

Heading level 4

Heading level 5

Heading level 6

tone

Title

Title

Title

Title

Title

primary

secondary

link

error

success

All variants — light theme

level

Heading level 1

Heading level 2

Heading level 3

Heading level 4

Heading level 5

Heading level 6

tone

Title

primary

Title

secondary

Title

link

Title

error

Title

success

Dark theme

level

عنوان المستوى 1

عنوان المستوى 2

عنوان المستوى 3

عنوان المستوى 4

عنوان المستوى 5

عنوان المستوى 6

tone

عنوان

success

عنوان

error

عنوان

link

عنوان

secondary

عنوان

primary

Arabic (ar-SA) · RTL

3 Icon

Renders the controlled Bunyan SVG icon set.

```
import { Icon } from '@bunyan/design-system';
```

- Variants & states**
- Six token sizes and eight semantic tones.
- Accessibility**
- Decorative icons (no `accessibilityLabel`) are hidden from assistive tech; labelled icons get the image role.
- Example**
- `<Icon name="chevron-right" mirroredInRTL />`.
- Do/don't**
- Prefer semantic tone; use `color` only for approved brand exceptions.
- Edge cases**
- Directional icons (chevron-left/right, backspace) mirror automatically in RTL; pass `mirroredInRTL` to override.

IconProps

PROP	TYPE	DEFAULT
<code>name</code> required	<code>IconName</code>	—
<code>size</code>	<code>'xs' 'sm' 'md' 'lg' 'xl' 'xxl'</code>	<code>'md'</code>
<code>tone</code>	<code>'primary' 'secondary' 'tertiary' 'inverse' 'success' 'warning' 'error' 'information'</code>	<code>'primary'</code>
<code>color</code>	<code>string</code>	—
<code>mirroredInRTL</code>	<code>boolean</code> Mirror in RTL. Defaults to <code>true</code> for directional icons only.	<code>DIRECTIONAL_ICONS.has(name)</code>
<code>accessibilityLabel</code>	<code>string</code>	—
<code>testID</code>	<code>string</code>	—

name


alert-circle


backspace


calendar


check


chevron-down


chevron-left


chevron-right


close


eye


eye-off


info


search


success


warning


error


user

size


xs


sm


md


lg


xl


xxl

tone


primary


secondary


tertiary


success


warning


error


information

All variants — light theme

name


alert-circle


backspace


calendar


check


chevron-down


chevron-left


chevron-right


close


eye


eye-off


info


search


success


warning


error


user

size


xs


sm


md


lg


xl


xxl

tone


primary


secondary


tertiary


success


warning


error


information

Dark theme

							name
							
chevron-left	chevron-down	check	calendar	backspace	alert-circle		
							
search	info	eye-off	eye	close	chevron-right		
							
		user	error	warning	success		
							size
							
xxl	xl	lg	md	sm	xs		
							tone
							
error	warning	success	tertiary	secondary	primary		
							information

Arabic (ar-SA) · RTL

Actions

4 Button

Triggers an action, or — with `variant="link"` / `href` — navigates.

```
import { Button } from '@bunyan/design-system';
```

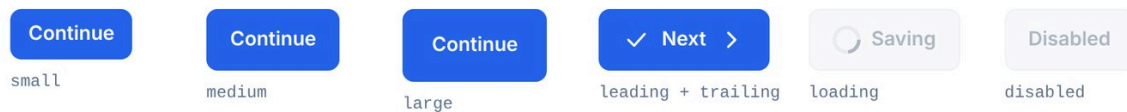
- Variants & states**
- `primary`, `secondary`, `outline`, `ghost`, `danger`, `link`; small/medium/large; pressed, disabled, loading, busy.
- Accessibility**
- Button role (link role for `variant="link"` or when `href` is set); busy/disabled states are automatic; external links announce `externalHint`; compact sizes and links get hitSlop for a 44 pt target.
- Example**
- `<Button variant="primary" size="large" fullWidth>Continue</Button>` · `<Button variant="link" href="https://bunyan.sa" external>Help centre</Button>`.
- Do/don't**
- Use one primary action per region; use `variant="link"` (not `ghost`) for navigation inside text-heavy layouts.
- Edge cases**
- Loading disables duplicate submission; `href` opens after `onPress` unless the event is `preventDefault` -ed; `openURL` failures go to `onOpenError`.

ButtonProps

PROP	TYPE	DEFAULT
<code>children</code> required	<code>React.ReactNode</code>	—
<code>variant</code>	<code>'primary' 'secondary' 'outline' 'ghost' 'danger' 'link'</code> <code>link</code> renders inline link text (no container); every other variant is a filled/outlined button.	<code>'primary'</code>
<code>size</code>	<code>'small' 'medium' 'large'</code>	<code>'medium'</code>
<code>loading</code>	<code>boolean</code>	<code>false</code>
<code>fullWidth</code>	<code>boolean</code>	<code>false</code>
<code>leadingIcon</code>	<code>IconName</code>	—
<code>trailingIcon</code>	<code>IconName</code>	—
<code>href</code>	<code>string</code> URL opened with <code>Linking.openURL</code> after <code>onPress</code> (unless the event was <code>preventDefault</code> -ed). Gives the control the link role.	—
<code>external</code>	<code>boolean</code> Marks the destination as outside the app: shows ↗ and announces <code>externalHint</code> .	<code>false</code>
<code>externalHint</code>	<code>string</code> Screen-reader hint for external destinations.	<code>'Opens in another app'</code>
<code>onOpenError</code>	<code>(error: unknown) => void</code> Called when <code>href</code> cannot be opened.	—

Also accepts: `Omit<PressableProps, 'children' | 'style'>` (native React Native props).

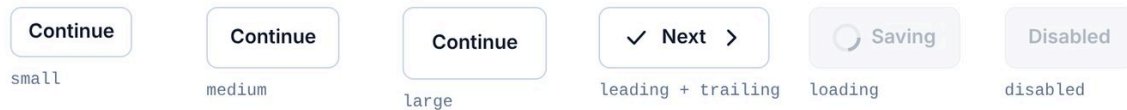
variant="primary"



variant="secondary"



variant="outline"



variant="ghost"



variant="danger"



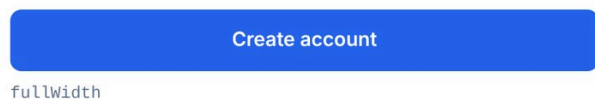
variant="link"



variant="link" with href

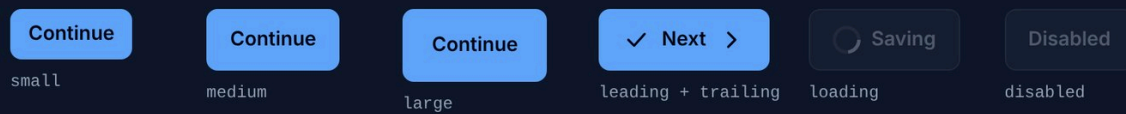


fullWidth

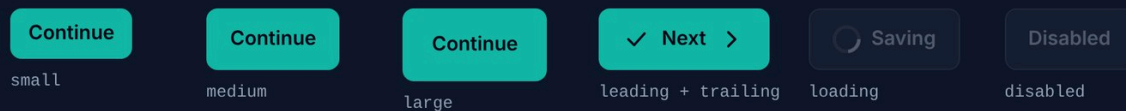


All variants — light theme

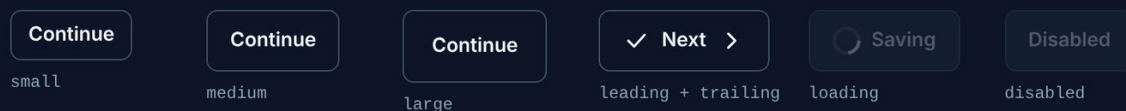
variant="primary"



variant="secondary"



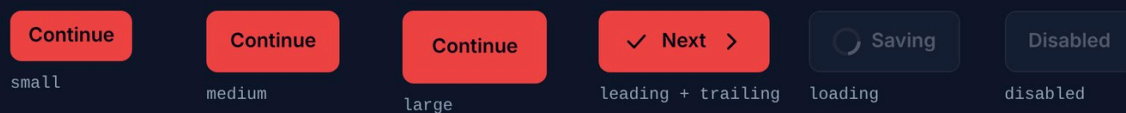
variant="outline"



variant="ghost"



variant="danger"



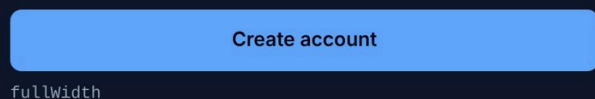
variant="link"



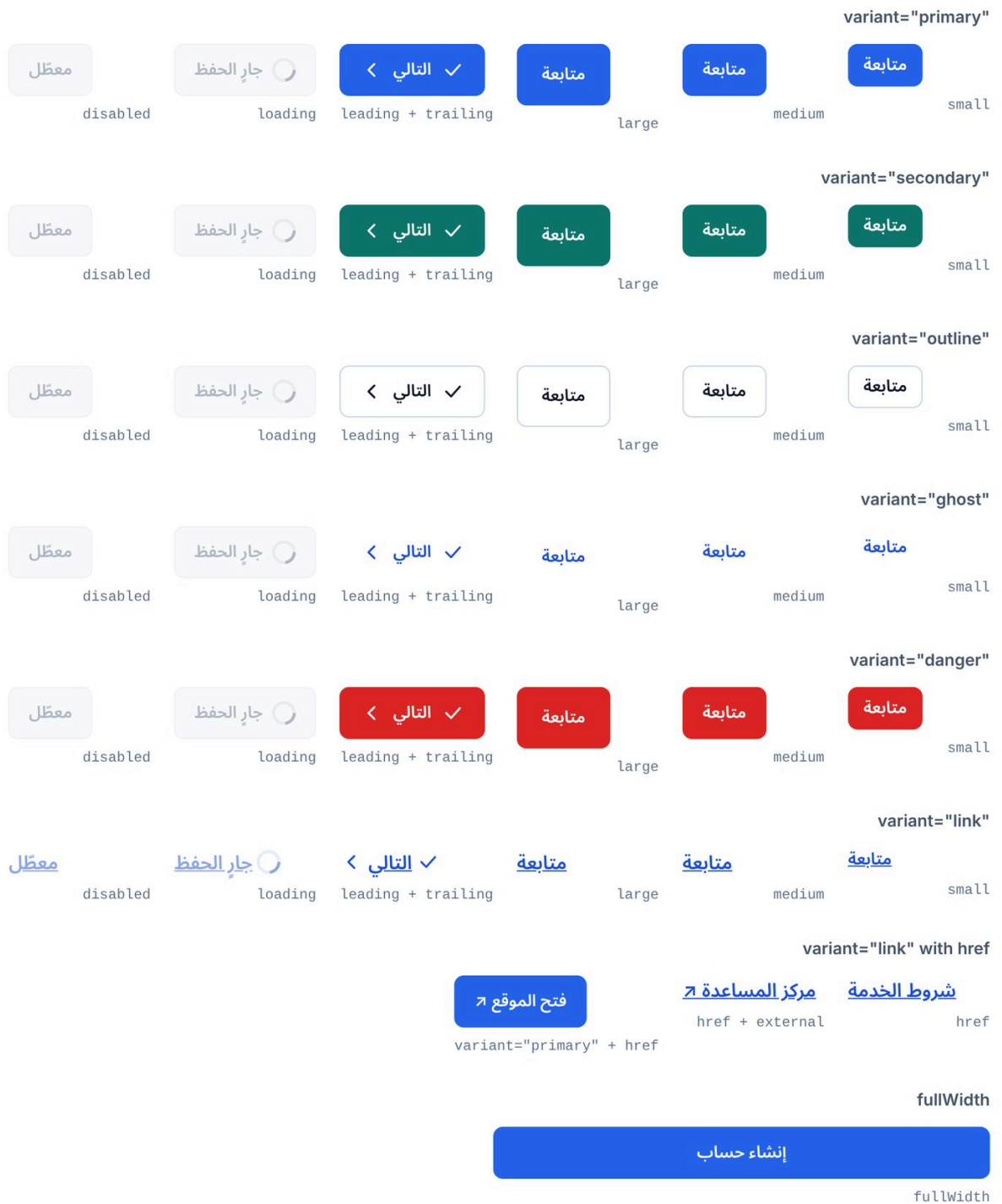
variant="link" with href



fullWidth



Dark theme



Arabic (ar-SA) · RTL

5 IconButton

Compact icon-only action.

```
import { IconButton } from '@bunyan/design-system';
```

- Variants & states**
- Filled, outline, ghost; small/medium/large; pressed, selected, disabled.
- Accessibility**
- A text label is mandatory despite the visual icon-only treatment.
- Example**
- `<IconButton icon="close" accessibilityLabel="Close dialog" />.`
- Do/don't**
- Use familiar icons; do not rely on color alone for selection.
- Edge cases**
- Add surrounding space when several icon buttons form a toolbar.

IconButtonProps

PROP	TYPE	DEFAULT
<code>icon</code> required	<code>IconName</code>	—
<code>accessibilityLabel</code> required	<code>string</code> Overrides the text that's read by the screen reader when the user interacts with the element. By default, the label is constructed by traversing all the children and accumulating all the Text nodes separated by space.	—
<code>variant</code>	<code>'filled' 'outline' 'ghost'</code>	<code>'ghost'</code>
<code>size</code>	<code>'small' 'medium' 'large'</code>	<code>'medium'</code>
<code>tone</code>	<code>'primary' 'secondary' 'tertiary' 'inverse' 'success' 'warning' 'error' 'information'</code>	<code>'primary'</code>
<code>selected</code>	<code>boolean</code>	<code>false</code>
<code>mirroredInRTL</code>	<code>boolean</code> Mirror the glyph in RTL. Defaults to automatic (directional icons only).	—

Also accepts: `Omit<PressableProps, 'children' | 'style'>` (native React Native props).

variant="ghost"

small

medium

large

selected

disabled

tone=error

variant="outline"

small

medium

large

selected

disabled

tone=error

variant="filled"

small

medium

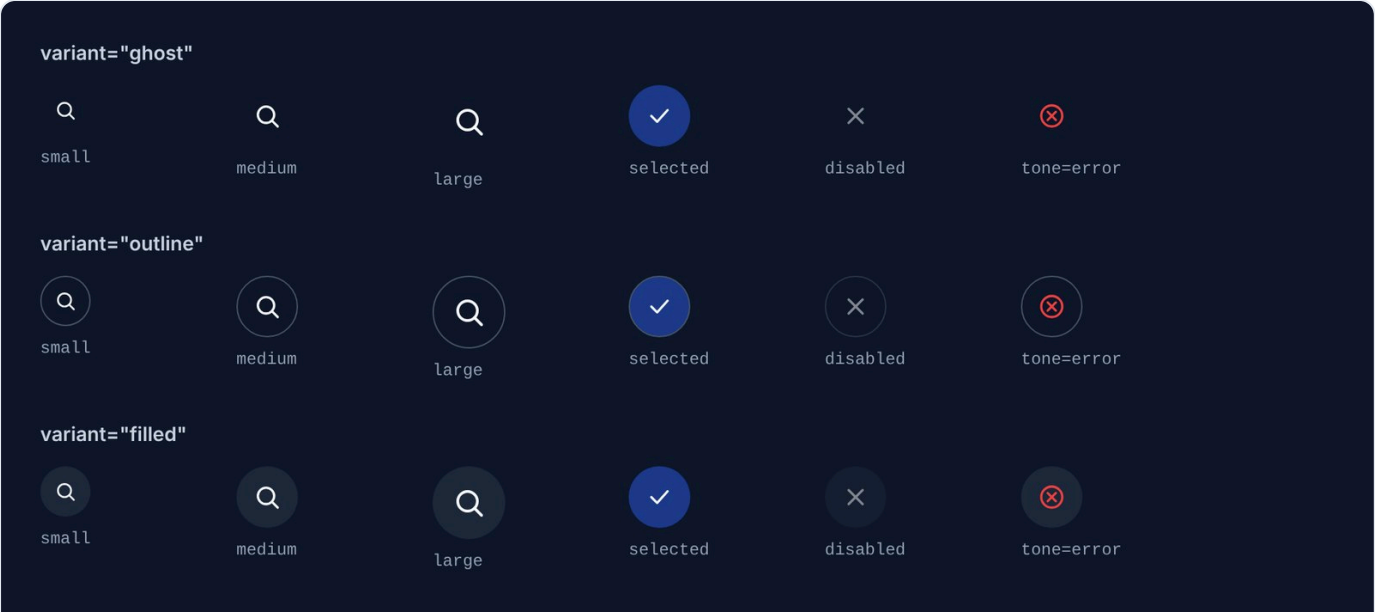
large

selected

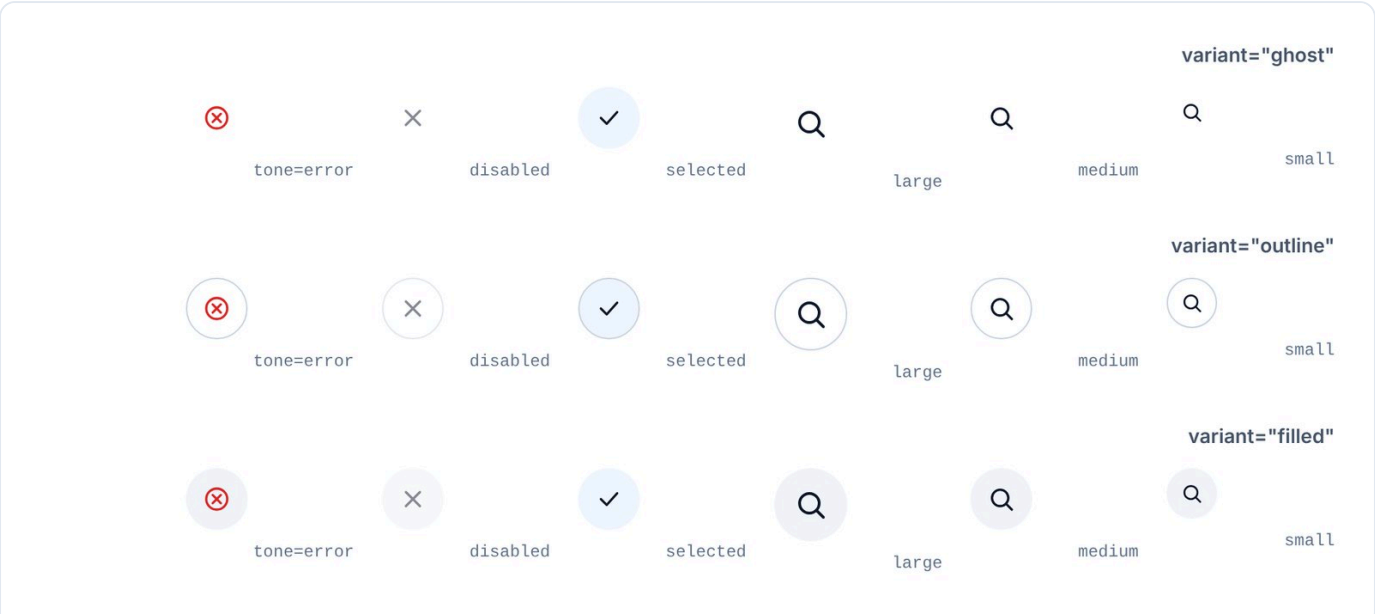
disabled

tone=error

All variants — light theme



Dark theme



Arabic (ar-SA) · RTL

6 Link

Inline navigation text. `Link` is an alias of `<Button variant="link" />` and accepts every `Button` prop except `variant`.

```
import { Link } from '@bunyan/design-system';
```

- Variants & states

small/medium/large text; pressed, disabled, loading.
- Accessibility

Uses the link role; external links show ↗ and announce `externalHint`. `openURL` failures go to `onOpenError`.
- Example

`<Link href="https://example.com" external>Privacy policy</Link>` (same as `<Button variant="link" ...>`).
- Do/don't

Use descriptive text; do not label links "click here."
- Edge cases

Kept for convenience and backwards compatibility; styling lives in `Button`, so both stay in sync.

LinkProps

PROP	TYPE	DEFAULT
<code>children</code> required	<code>React.ReactNode</code>	—
<code>loading</code>	<code>boolean</code>	—
<code>size</code>	<code>'small' 'medium' 'large'</code>	—
<code>leadingIcon</code>	<code>IconName</code>	—
<code>fullWidth</code>	<code>boolean</code>	—
<code>trailingIcon</code>	<code>IconName</code>	—
<code>href</code>	<code>string</code> URL opened with <code>Linking.openURL</code> after <code>onPress</code> (unless the event was <code>preventDefault</code> -ed). Gives the control the link role.	—
<code>external</code>	<code>boolean</code> Marks the destination as outside the app: shows ↗ and announces <code>externalHint</code> .	—
<code>externalHint</code>	<code>string</code> Screen-reader hint for external destinations.	—
<code>onOpenError</code>	<code>(error: unknown) => void</code> Called when <code>href</code> cannot be opened.	—

Also accepts: `Omit<ButtonProps>` (native React Native props).

Terms of service

Help centre ↗

Unavailable

Small link

ⓘ Learn more

⌛ Loading

default

external

disabled

size="small"

leadingIcon

loading

All variants — light theme

Terms of service

Help centre ↗

Unavailable

Small link

ⓘ Learn more

⌛ Loading

default

external

disabled

size="small"

leadingIcon

loading

Dark theme

جارٍ التحميل

loading

اعرف المزيد ⓘ

leadingIcon

رابط صغير

size="small"

غير متاح

disabled

مركز المساعدة 7

external

شروط الخدمة

default

Arabic (ar-SA) · RTL

7 ActionText

A sentence with an inline link ("Don't have an account? Sign up").

```
import { ActionText } from '@bunyan/design-system';
```

Variants & states	Any Text variant; disabled.
Accessibility	The action is a nested text with the link role.
Example	<code><ActionText text="Forgot your password?" actionText="Reset it" onPress={reset} />.</code>
Do/don't	Keep the action short and descriptive.
Edge cases	Wraps naturally across lines and in RTL.

ActionTextProps

PROP	TYPE	DEFAULT
<code>text</code>	<code>TextValue</code> Leading sentence, e.g. "Don't have an account?".	—
<code>actionText</code> required	<code>TextValue</code> Pressable part, e.g. "Sign up".	—
<code>onPress</code> required	<code>() => void</code>	—
<code>trailingText</code>	<code>TextValue</code> Text after the action.	—
<code>variant</code>	<code>'body' 'bodySmall' 'caption' 'label' 'code'</code>	<code>'bodySmall'</code>
<code>align</code>	<code>'start' 'center' 'end'</code>	<code>'start'</code>
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>testID</code>	<code>string</code>	—

Don't have an account? [Sign up](#)

By continuing you accept the [terms](#) of service.

text + actionTextalign="center" + trailingText

All variants — light theme

Don't have an account? [Sign up](#)

By continuing you accept the [terms](#) of service.

text + actionTextalign="center" + trailingText

Dark theme

ليس لديك حساب؟ [سجل الآن](#)

بالتابعة فإنك توافق على [الشروط](#) الخاصة بالخدمة.

text + actionTextalign="center" + trailingText

Arabic (ar-SA) · RTL

Forms

8 Form

Whole form from a `fields` array — Formik state, Yup validation, Bunyan controls with labels, errors, theme, RTL and accessibility. See FORMS.md.

```
import { Form } from '@bunyan/design-system';
```

Variants & states	Field types <code>Input / TextInput</code> , <code>PasswordInput</code> , <code>SearchInput</code> , <code>TextArea</code> , <code>Select / Picker</code> , <code>DatePicker</code> , <code>DateRangePicker</code> , <code>Checkbox</code> , <code>Switch / Toggle</code> , <code>RadioGroup</code> , <code>ChipsGroup</code> , <code>CheckboxGroup</code> , <code>RadioImageGroup</code> , <code>BoxGroup</code> , <code>SwatchGroup</code> , <code>Slider</code> , <code>AmountInput</code> , <code>AmountWithCurrencyInput</code> , <code>PhoneWithCountryInput</code> , <code>AmountField</code> , <code>FileInput</code> , <code>OTP</code> , <code>RepeatedControls</code> , display-only <code>Progress</code> and <code>ActionText</code> , React elements, and custom registered types; pristine, touched, invalid, submitting.
Accessibility	Each control gets its label as accessible name; errors use alert semantics; submit shows busy state while submitting.
Example	<code><Form initialValues={{ email: '' }} validationSchema={schema} onSubmit={save} fields={[{ type: 'Input', name: 'email', label: { localeKey: 'common.email' } }]} /></code> .
Do/don't	Keep rules in <code>validationSchema</code> and texts as locale keys; do not render one Form inside another.
Edge cases	Falsy <code>fields</code> entries are skipped; <code>visibleWhen</code> hides fields based on values; Yup messages can be locale keys; needs <code>formik</code> installed; use the <code>Yup</code> exported by Bunyan for schemas.

FormProps

PROP	TYPE	DEFAULT
<code>initialValues</code> required	Values	—
<code>onSubmit</code> required	(values: Values, helpers: FormikHelpers<Values>) => unknown	—
<code>validationSchema</code>	FormikConfig<Values>['validationSchema'] Yup schema (or any schema Formik accepts).	—
<code>fields</code> required	readonly FormFieldEntry<Values>[] Field list. Falsy entries are skipped.	—
<code>formProps</code>	Omit<FormikConfig<Values>, 'initialValues' 'onSubmit' 'validationSchema' 'children' 'component' 'render'> Extra Formik config, e.g. { <code>enableReinitialize: true</code> , <code>validateOnMount: true</code> , <code>innerRef</code> }.	—
<code>submitButton</code>	FormButtonConfig false Submit button config, or <code>false</code> to hide it (e.g. when submitting from a header button). Default text <code>'Submit'</code> .	{}
<code>resetButton</code>	FormButtonConfig false Optional reset button (<code>text</code> default <code>'Reset'</code>). Resets to <code>initialValues</code> .	false
<code>spacing</code>	Responsive<keyof Theme['breakpoint'], number keyof Theme['spacing']> Gap between fields: points, a spacing token or per breakpoint. Default <code>'lg'</code> .	—

PROP	TYPE	DEFAULT
<code>showOptional</code>	<code>boolean</code> Show <code>(Optional)</code> next to labels of fields that are not <code>required</code> . Default <code>false</code> .	<code>false</code>
<code>optionalLabel</code>	<code>TextValue</code> Text of the optional marker. Default <code>'Optional'</code> .	<code>'Optional'</code>
<code>fieldTypes</code>	<code>Record<string, FormFieldComponent<any> FormFieldRegistration<any>></code> Extra field types for this form only (merged over the global registry).	—
<code>children</code>	<code>React.ReactNode ((form: FormikProps<Values>) => React.ReactNode)</code> Content between the fields and the buttons.	—
<code>renderFooter</code>	<code>(form: FormikProps<Values>) => React.ReactNode</code> Replace the default buttons row.	—
<code>style</code>	<code>StyleProp<ViewStyle></code>	—
<code>testID</code>	<code>string</code>	—

FormFieldBase

PROP	TYPE	DEFAULT
<code>name</code> required	<code>FormFieldName<Values></code>	—
<code>label</code>	<code>TextValue</code>	—
<code>description</code>	<code>TextValue</code>	—
<code>required</code>	<code>boolean</code> Shows <code>*</code> after the label. Validation itself comes from <code>validationSchema</code> .	—
<code>disabled</code>	<code>boolean</code>	—
<code>visibleWhen</code>	<code>(values: Values) => boolean</code> Render the field only when this returns true (based on current form values).	—
<code>onValueChange</code>	<code>(value: unknown, form: FormikProps<Values>) => void</code> Runs after the value changes, e.g. to reset a dependent field.	—
<code>testID</code>	<code>string</code>	—

FormFieldControlProps

PROP	TYPE	DEFAULT
<code>field</code> required	<code>C</code> The field config from <code>fields</code> .	—
<code>name</code> required	<code>string</code>	—
<code>value</code> required	<code>unknown</code>	—
<code>setValue</code> required	<code>(value: unknown) => void</code>	—
<code>setTouched</code> required	<code>() => void</code> Mark as touched (call on blur / after a selection).	—

PROP	TYPE	DEFAULT
<code>error</code>	<code>string undefined</code> Translated error message, shown when the field was touched or the form submitted.	—
<code>invalid</code> required	<code>boolean</code>	—
<code>disabled</code> required	<code>boolean</code>	—
<code>t</code> required	<code>(value: TextValue null undefined) => string undefined</code> Resolves <code>TextValue</code> s with the app translate function.	—
<code>form</code> required	<code>FormikProps<FormikValues></code>	—

Transactions

All

Incoming

Outgoing

Transaction date

Gregorian

Hijri

From

Start date

Select a start date

Reset

To

Oct 3, 2026

filter form (your example) · after pressing Apply with no start date

Apply filter

Full name *

Faisal

City (Optional)

Riyadh

Notifications

Email and SMS

I accept the terms *

Save

Input · Select · Checkbox · Switch fields

All variants — light theme

Transactions

All

Incoming

Outgoing

Transaction date

Gregorian

Hijri

From

Start date

Select a start date

Reset

To

Oct 3, 2026

filter form (your example) · after pressing Apply with no start date

Apply filter

Full name *

Faisal

City (Optional)

Riyadh

Notifications

Email and SMS

I accept the terms *

Save

Input · Select · Checkbox · Switch fields

Dark theme

الاسم الكامل *

Faisal

العمليات

صادر

وارد

الكل

المدينة (Optional)

الرياض

تاريخ العملية

هجري

ميلادي

الإشعارات البريد والرسائل

أوافق على الشروط *

إلى

من

٣ أكتوبر ٢٠٢٦

Start date

تطبيق

إعادة تعيين

حفظ

filter form (your example) · after pressing Apply with no start date

Input · Select · Checkbox · Switch fields

اختار تاريخ البداية

Arabic (ar-SA) · RTL

New transfer

Step 2 of 3

2 / 3

Amount

2,500

SAR

Mobile for OTP

+966

512345678

Beneficiary 1

Name

Sara Alqahtani

IBAN

SA0380000000608010167519

+ Add beneficiary

Continue

Purpose

Family

Bills

Installments

6

Card colour

Verification code

1

2

Didn't get the code? [Resend](#)

Extended field types: Progress, AmountWithCurrencyInput, PhoneWithCountryInput, RepeatedControls, BoxGroup, Slider, SwatchGroup, OTP, ActionText, React element

Bunyan Design System · Reference & AI Guide

35 / 147

New transfer

Step 2 of 3

2 / 3

Amount

2,500

SAR ▾

Mobile for OTP

+966 ▾ 512345678

Beneficiary 1



Name

Sara Alqahtani

IBAN

SA0380000000608010167519

+ Add beneficiary

Continue

Purpose



Family



Bills

Installments

6

Card colour



Verification code

1

2

Didn't get the code? [Resend](#)

Dark theme

3 / 2

الخطوة 2 من 3

المبلغ

▼ SAR

2,500

الجوال لرمز التحقق

512345678

▼ +966

المستفيد 1

الاسم

سارة القحطاني

IBAN

SA0380000000608010167519

إضافة مستفيد +

متابعة

الغرض

فواتير

عائلي

الأقساط

6

لون البطاقة

رمز التحقق

1

2

لم يصلك الرمز؟ [إعادة الإرسال](#)

Arabic (ar-SA) · RTL

9 FormField

Groups label, description, control, error, and success feedback.

```
import { FormField } from '@bunyan/design-system';
```

- Variants & states

Required/optional, error/success.
- Accessibility

Render props provide stable IDs for label and description relationships.
- Example

```
<FormField label="Email">{ids => <Input accessibilityLabelledBy={ids.labelId} />}
</FormField>.
```
- Do/don't

Show one actionable error; do not encode required state only with an asterisk.
- Edge cases

Localize `optionalLabel` and all feedback strings.

FormFieldProps

PROP	TYPE	DEFAULT
<code>label</code> required	<code>string</code>	—
<code>children</code> required	<code>React.ReactNode ((props: FormFieldRenderProps) => React.ReactNode)</code>	—
<code>description</code>	<code>string</code>	—
<code>error</code>	<code>string</code>	—
<code>success</code>	<code>string</code>	—
<code>required</code>	<code>boolean</code>	<code>false</code>
<code>optionalLabel</code>	<code>string</code>	<code>'Optional'</code>
<code>showOptional</code>	<code>boolean</code> Show <code>(Optional)</code> next to non-required labels. Default <code>true</code> .	<code>true</code>

FormFieldRenderProps

PROP	TYPE	DEFAULT
<code>inputId</code> required	<code>string</code>	—
<code>labelId</code> required	<code>string</code>	—
<code>descriptionId</code>	<code>string</code>	—
<code>errorId</code>	<code>string</code>	—

Email *

We never share it.

required + description

IBAN *

Enter a valid IBAN

error

Nickname (Optional)

optional

Username *

Available

success

All variants — light theme

Email *

We never share it.

name@company.com

required + description

Nickname (Optional)

optional

IBAN *

SA00

Enter a valid IBAN

error

Username *

faisal

Available

success

Dark theme

الاسم المستعار (اختياري)

optional

البريد الإلكتروني *

لن نشاركه أبدًا.

name@company.com

required + description

اسم المستخدم *

faisal

متاح

success

الآيبان *

أدخل آيبان صالحًا

SA00

error

Arabic (ar-SA) · RTL

Text & number inputs

10 Input

Single-line text entry primitive.

```
import { Input } from '@bunyan/design-system';
```

- Variants & states** Small/medium/large; default/error/success; focused and disabled.
- Accessibility** Supports native labelling props and announces invalid state with `aria-invalid`.
- Example** `<Input keyboardType="email-address" status="error" />`.
- Do/don't** Wrap in `FormField`; do not use placeholder text as the only label.
- Edge cases** Controlled inputs require `onChangeText`; custom trailing controls must have labels.

InputProps

PROP	TYPE	DEFAULT
size	'small' 'medium' 'large'	'medium'
status	'default' 'error' 'success'	'default'
leadingIcon	IconName	—
trailing	React.ReactNode	—
containerStyle	Pick<ViewStyle, 'flex' 'width' 'maxWidth' 'minWidth' 'alignSelf'>	—
leading	React.ReactNode Content before the text (after <code>leadingIcon</code>), e.g. a country-code picker.	—
textDirection	'ltr' 'rtl' Direction of the typed text. Use <code>'ltr'</code> for phone numbers, IBANs, emails and amounts so they read correctly inside an Arabic UI. Default: the UI direction.	—

Also accepts: `Omit<TextInputProps, 'style'>` (native React Native props).

size

Account name

Account name

Account name

small

medium

large

status

Bunyan Holding

Bunyan Holding

Bunyan Holding

default

error

success

adornments & states

⌕

Username

0.00SAR

Read only

leadingIcon

trailing

editable={false}

All variants — light theme

size

Account name

Account name

Account name

small

medium

large

status

Bunyan Holding

Bunyan Holding

Bunyan Holding

default

error

success

adornments & states

⬢

Username

0.00SAR

Read only

leadingIcon

trailing

editable={false}

Dark theme

size

اسم الحساب

اسم الحساب

اسم الحساب

large

medium

small

status

بنیان القابضة

بنیان القابضة

بنیان القابضة

success

error

default

adornments & states

للقراءة فقط

SAR0.00

اسم المستخدم ⬢

editable={false}

trailing

leadingIcon

Arabic (ar-SA) · RTL

11 PasswordInput

Secure text entry with visibility control.

```
import { PasswordInput } from '@bunyan/design-system';
```

Variants & states Inherits Input states; hidden and visible states.

Accessibility Visibility button announces its current action.

Example `<PasswordInput accessibilityLabel="Password" />`.

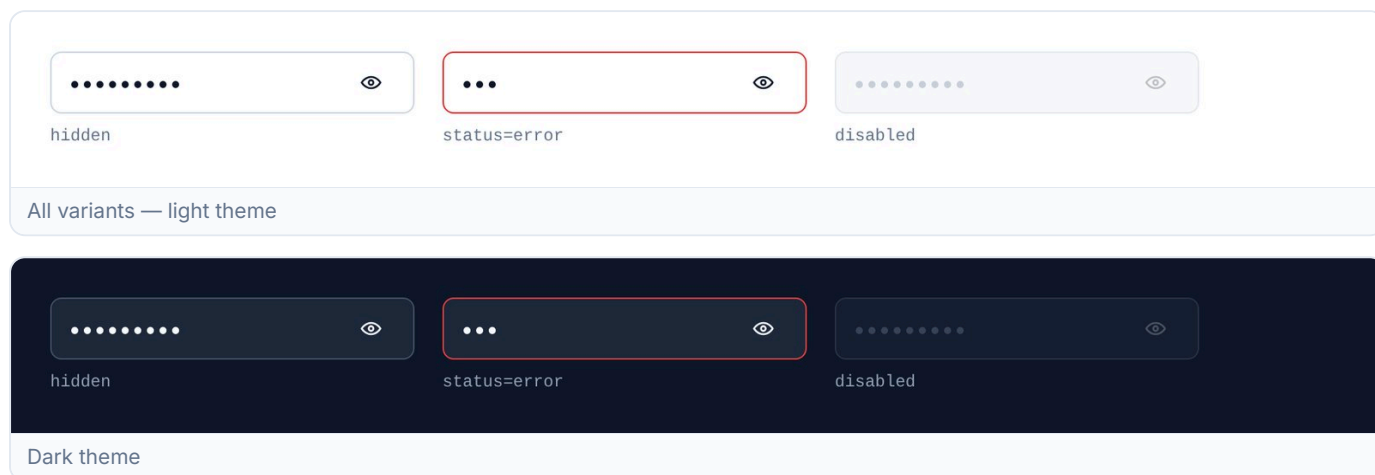
Do/don't Allow password managers; do not block paste.

Edge cases Platform keyboards may briefly retain suggestions after visibility changes.

PasswordInputProps

PROP	TYPE	DEFAULT
<code>status</code>	<code>'default' 'error' 'success'</code>	—
<code>size</code>	<code>'small' 'medium' 'large'</code>	—
<code>leadingIcon</code>	<code>IconName</code>	—
<code>containerStyle</code>	<code>Pick<ViewStyle, 'flex' 'width' 'maxWidth' 'minWidth' 'alignSelf'></code>	—
<code>leading</code>	<code>React.ReactNode</code> Content before the text (after <code>leadingIcon</code>), e.g. a country-code picker.	—
<code>textDirection</code>	<code>'ltr' 'rtl'</code> Direction of the typed text. Use <code>'ltr'</code> for phone numbers, IBANs, emails and amounts so they read correctly inside an Arabic UI. Default: the UI direction.	—
<code>showPasswordLabel</code>	<code>string</code>	<code>'Show password'</code>
<code>hidePasswordLabel</code>	<code>string</code>	<code>'Hide password'</code>

Also accepts: `Omit<InputProps>` (native React Native props).



disabled

status=error

hidden

Arabic (ar-SA) · RTL

Bunyan Design System · Reference & AI Guide

43 / 147

12 SearchInput

Search query entry with search and clear affordances.

```
import { SearchInput } from '@bunyan/design-system';
```

Variants & states	Inherits Input; clear action appears when <code>value</code> is non-empty.
Accessibility	Search return key and labelled clear button.
Example	<code><SearchInput value={query} onChangeText={setQuery} onClear={() => setQuery('')} />.</code>
Do/don't	Debounce network requests outside the component; do not clear without user action.
Edge cases	Works controlled or uncontrolled; Clear calls <code>onChangeText('')</code> and <code>onClear</code> , then refocuses the field.

SearchInputProps

PROP	TYPE	DEFAULT
<code>clearLabel</code>	<code>string</code>	<code>'Clear search'</code>
<code>onClear</code>	<code>() => void</code>	—
<code>status</code>	<code>'default' 'error' 'success'</code>	—
<code>size</code>	<code>'small' 'medium' 'large'</code>	—
<code>containerStyle</code>	<code>Pick<ViewStyle, 'flex' 'width' 'maxWidth' 'minWidth' 'alignSelf'></code>	—
<code>leading</code>	<code>React.ReactNode</code> Content before the text (after <code>leadingIcon</code>), e.g. a country-code picker.	—
<code>textDirection</code>	<code>'ltr' 'rtl'</code> Direction of the typed text. Use <code>'ltr'</code> for phone numbers, IBANs, emails and amounts so they read correctly inside an Arabic UI. Default: the UI direction.	—

Also accepts: `Omit<InputProps, 'leadingIcon' | 'trailing'>` (native React Native props).

Q Search accounts

empty

Q report

with value (clear button)

Q Search

size=small

All variants — light theme

Q Search accounts

empty

Q report

with value (clear button)

Q Search

size=small

Dark theme

Q بحث

size=small

×

تقرير

with value (clear button)

Q ابحث في الحسابات

empty

Arabic (ar-SA) · RTL

13 TextArea

Multi-line text entry.

```
import { TextArea } from '@bunyan/design-system';
```

- Variants & states**
- Default/error/success; focused, disabled, character limit.
- Accessibility**
- Exposes invalid and disabled semantics; counter remains visible.
- Example**
- `<TextArea minRows={6} maxLength={500} />.`
- Do/don't**
- Set realistic limits; do not resize based on every keystroke in dense forms.
- Edge cases**
- Very large font settings increase the rendered minimum height.

TextAreaProps

PROP	TYPE	DEFAULT
status	'default' 'error' 'success'	'default'
minRows	number	4
maxLength	number	—
	Limits the maximum number of characters that can be entered. Use this instead of implementing the logic in JS to avoid flicker.	
showCounter	boolean	Boolean(maxLength)

Also accepts: `Omit<TextInputProps, 'style' | 'multiline'>` (native React Native props).

Describe the issue

default

Delivery delayed

maxLength counter16/120

!

status=error

Locked note

disabled

All variants — light theme

Describe the issue

default

Delivery delayed

maxLength counter16/120

!

status=error

Locked note

disabled

Dark theme

!

status=error

تأخر التوصيل

12/120

maxLength counter

صف المشكلة

default

ملاحظة مقفلة

disabled

Arabic (ar-SA) · RTL

14 AmountInput

Money and phone entry built on Input.

```
import { AmountInput } from '@bunyan/design-system';
```

Variants & states	All Input sizes and statuses; currency / country pickers open a searchable sheet.
Accessibility	Pickers are labelled buttons announcing the current value; amounts, phones and codes are typed left-to-right inside Arabic UIs.
Example	<code><AmountInput value={amount} onChangeValue={setAmount} currency="SAR" /></code> · <code><PhoneInput value={phone} onChange={setPhone} /></code> .
Do/don't	Keep amounts as numbers in state; format only for display (<code>formatAmount</code>). Do not store the grouped text.
Edge cases	Arabic-Indic digits are converted; leading zeros and extra decimals are dropped; <code>toE164()</code> builds <code>+966...</code> ; <code>isValidPhone()</code> checks national-number length.

AmountInputProps

PROP	TYPE	DEFAULT
<code>value</code>	<code>number null</code> Numeric value (<code>null</code> when empty).	—
<code>onChangeValue</code>	<code>(value: number null) => void</code>	—
<code>currency</code>	<code>string</code> Currency code or symbol shown after the amount (e.g. <code>'SAR'</code>).	—
<code>decimals</code>	<code>number</code> Maximum decimals. Default <code>2</code> .	<code>2</code>
<code>allowNegative</code>	<code>boolean</code>	<code>false</code>
<code>groupSeparator</code>	<code>string</code> Thousands separator. Default <code>','</code> .	<code>','</code>
<code>status</code>	<code>'default' 'error' 'success'</code>	—
<code>size</code>	<code>'small' 'medium' 'large'</code>	—
<code>leadingIcon</code>	<code>IconName</code>	—
<code>trailing</code>	<code>React.ReactNode</code>	—
<code>containerStyle</code>	<code>Pick<ViewStyle, 'flex' 'width' 'maxWidth' 'minWidth' 'alignSelf'></code>	—
<code>leading</code>	<code>React.ReactNode</code> Content before the text (after <code>leadingIcon</code>), e.g. a country-code picker.	—
<code>textDirection</code>	<code>'ltr' 'rtl'</code> Direction of the typed text. Use <code>'ltr'</code> for phone numbers, IBANs, emails and amounts so they read correctly inside an Arabic UI. Default: the UI direction.	—

Also accepts: `Omit<InputProps, 'value' | 'defaultValue' | 'onChangeText' | 'keyboardType' | 'inputMode'>` (native React Native props).

AmountInput

12,500.5 SAR	0.00 SAR	999,999 SAR
value={12500.5} currency="SAR"	empty · placeholder	status="error"

AmountWithCurrencyInput (currency picker inside)

 USD ▾
currencies=['SAR', 'USD', 'KWD']

PhoneWithCountryInput / PhoneInput

 +966 ▾ 512345678	 +971 ▾ 50 123 4567
value={{ country: 'SA', number: '512345678' }}	empty · AE

All variants — light theme

AmountInput

12,500.5 SAR	0.00 SAR	999,999 SAR
value={12500.5} currency="SAR"	empty · placeholder	status="error"

AmountWithCurrencyInput (currency picker inside)

 USD ▾
currencies=['SAR', 'USD', 'KWD']

PhoneWithCountryInput / PhoneInput

 +966 ▾ 512345678	 +971 ▾ 50 123 4567
value={{ country: 'SA', number: '512345678' }}	empty · AE

Dark theme

AmountInput

SAR 999,999

status="error"

ر.س

0.00

empty · placeholder

SAR

12,500.5

value={12500.5} currency="SAR"

AmountWithCurrencyInput (currency picker inside)

▼ USD

1,500

currencies={['SAR', 'USD', 'KWD']}

PhoneWithCountryInput / PhoneInput

50 123 4567 ▼ 🇪🇬 +971

empty · AE

512345678 ▼ 🇸🇦 +966

value={{ country: 'SA', number: '512345678' }}

Arabic (ar-SA) · RTL

15 AmountWithCurrencyInput

Money and phone entry built on Input.

```
import { AmountWithCurrencyInput } from '@bunyan/design-system';
```

Variants & states	All Input sizes and statuses; currency / country pickers open a searchable sheet.
Accessibility	Pickers are labelled buttons announcing the current value; amounts, phones and codes are typed left-to-right inside Arabic UIs.
Example	<code><AmountInput value={amount} onChangeValue={setAmount} currency="SAR" /></code> · <code><PhoneInput value={phone} onChange={setPhone} /></code> .
Do/don't	Keep amounts as numbers in state; format only for display (<code>formatAmount</code>). Do not store the grouped text.
Edge cases	Arabic-Indic digits are converted; leading zeros and extra decimals are dropped; <code>toE164()</code> builds <code>+966...</code> ; <code>isValidPhone()</code> checks national-number length.

AmountWithCurrencyInputProps

PROP	TYPE	DEFAULT
<code>value</code>	<code>AmountWithCurrency null</code>	—
<code>onChange</code>	<code>(value: AmountWithCurrency) => void</code>	—
<code>currencies</code> required	<code>readonly (string CurrencyOption)[]</code>	—
<code>currencyTitle</code>	<code>TextValue</code>	<code>'Currency'</code>
<code>currencyAccessibilityLabel</code>	<code>TextValue</code>	<code>'Currency'</code>
<code>status</code>	<code>'default' 'error' 'success'</code>	—
<code>size</code>	<code>'small' 'medium' 'large'</code>	—
<code>leadingIcon</code>	<code>IconName</code>	—
<code>containerStyle</code>	<code>Pick<ViewStyle, 'flex' 'width' 'maxWidth' 'minWidth' 'alignSelf'></code>	—
<code>leading</code>	<code>React.ReactNode</code> Content before the text (after <code>leadingIcon</code>), e.g. a country-code picker.	—
<code>textDirection</code>	<code>'ltr' 'rtl'</code> Direction of the typed text. Use <code>'ltr'</code> for phone numbers, IBANs, emails and amounts so they read correctly inside an Arabic UI. Default: the UI direction.	—
<code>allowNegative</code>	<code>boolean</code>	—
<code>groupSeparator</code>	<code>string</code> Thousands separator. Default <code>','</code> .	—

Also accepts: `Omit<AmountInputProps, 'value' | 'onChangeValue' | 'onChange' | 'currency' | 'trailing' | 'decimals'>` (native React Native props).

CurrencyOption

PROP	TYPE	DEFAULT
<code>code</code> required	<code>string</code>	—

PROP	TYPE	DEFAULT
label	TextValue	—
decimals	number	—

Decimals for this currency (e.g. 3 for KWD).

AmountInput

12,500.5SAR

0.00SAR

999,999SAR

value={12500.5} currency="SAR"

empty · placeholder

status="error"

AmountWithCurrencyInput (currency picker inside)

1,500USD ▾

currencies=['SAR', 'USD', 'KWD']

PhoneWithCountryInput / PhoneInput

+966 ▾

512345678

+971 ▾

50 123 4567

value={{ country: 'SA', number: '512345678' }}

empty · AE

All variants — light theme

AmountInput

12,500.5SAR

0.00SAR

999,999SAR

value={12500.5} currency="SAR"

empty · placeholder

status="error"

AmountWithCurrencyInput (currency picker inside)

1,500USD ▾

currencies=['SAR', 'USD', 'KWD']

PhoneWithCountryInput / PhoneInput

+966 ▾

512345678

+971 ▾

50 123 4567

value={{ country: 'SA', number: '512345678' }}

empty · AE

Dark theme

AmountInput

SAR 999,999

status="error"

ر.س

0.00

empty · placeholder

SAR

12,500.5

value={12500.5} currency="SAR"

AmountWithCurrencyInput (currency picker inside)

▼ USD

1,500

currencies={['SAR', 'USD', 'KWD']}

PhoneWithCountryInput / PhoneInput

50 123 4567 ▼ 🇸🇦 +971

empty · AE

512345678 ▼ 🇸🇦 +966

value={{ country: 'SA', number: '512345678' }}

Arabic (ar-SA) · RTL

16 PhoneInput

Money and phone entry built on Input.

```
import { PhoneInput } from '@bunyan/design-system';
```

Variants & states	All Input sizes and statuses; currency / country pickers open a searchable sheet.
Accessibility	Pickers are labelled buttons announcing the current value; amounts, phones and codes are typed left-to-right inside Arabic UIs.
Example	<code><AmountInput value={amount} onChangeValue={setAmount} currency="SAR" /></code> · <code><PhoneInput value={phone} onChange={setPhone} /></code> .
Do/don't	Keep amounts as numbers in state; format only for display (<code>formatAmount</code>). Do not store the grouped text.
Edge cases	Arabic-Indic digits are converted; leading zeros and extra decimals are dropped; <code>toE164()</code> builds <code>+966...</code> ; <code>isValidPhone()</code> checks national-number length.

PhoneInputProps

PROP	TYPE	DEFAULT
<code>value</code>	<code>PhoneValue null</code>	—
<code>onChange</code>	<code>(value: PhoneValue) => void</code>	—
<code>countries</code>	<code>readonly PhoneCountry[]</code>	<code>DEFAULT_PHONE_COUNTRIES</code>
<code>defaultCountry</code>	<code>string</code> Country when <code>value</code> is empty. Default: first of <code>countries</code> .	—
<code>showFlag</code>	<code>boolean</code> Show flag emoji on the country trigger. Default <code>true</code> .	<code>true</code>
<code>countryTitle</code>	<code>TextValue</code>	<code>'Country code'</code>
<code>countryAccessibilityLabel</code>	<code>TextValue</code>	<code>'Country code'</code>
<code>searchPlaceholder</code>	<code>TextValue</code>	<code>'Search countries'</code>
<code>status</code>	<code>'default' 'error' 'success'</code>	—
<code>size</code>	<code>'small' 'medium' 'large'</code>	—
<code>leadingIcon</code>	<code>IconName</code>	—
<code>trailing</code>	<code>React.ReactNode</code>	—
<code>containerStyle</code>	<code>Pick<ViewStyle, 'flex' 'width' 'maxWidth' 'minWidth' 'alignSelf'></code>	—
<code>textDirection</code>	<code>'ltr' 'rtl'</code> Direction of the typed text. Use <code>'ltr'</code> for phone numbers, IBANs, emails and amounts so they read correctly inside an Arabic UI. Default: the UI direction.	—

Also accepts: `Omit<InputProps, 'value' | 'defaultValue' | 'onChangeText' | 'onChange' | 'keyboardType' | 'leading'>` (native React Native props).

PhoneValue

PROP		TYPE	DEFAULT
<code>country</code>	required	<code>string</code> ISO country code, e.g. <code>'SA'</code> .	—
<code>number</code>	required	<code>string</code> National number, digits only, without the leading 0.	—

PhoneCountry

PROP		TYPE	DEFAULT
<code>code</code>	required	<code>string</code> ISO 3166-1 alpha-2, e.g. <code>'SA'</code> .	—
<code>dialCode</code>	required	<code>string</code> Dialing code with <code>+</code> , e.g. <code>' +966'</code> .	—
<code>name</code>	required	<code>string</code>	—
<code>nameAr</code>		<code>string</code> Arabic name, used when the UI is RTL.	—
<code>lengths</code>		<code>readonly number[]</code> Example national number length(s) for basic validation.	—

AmountInput

12,500.5

SAR

0.00

SAR

999,999

SAR

value={12500.5} currency="SAR"
empty · placeholder
status="error"

AmountWithCurrencyInput (currency picker inside)

1,500

USD ▾

currencies={['SAR', 'USD', 'KWD']}

PhoneWithCountryInput / PhoneInput

 +966 ▾ 512345678

 +971 ▾ 50 123 4567

value={{ country: 'SA', number: '512345678' }}
empty · AE

All variants — light theme

AmountInput

12,500.5SAR

0.00SAR

999,999SAR

value={12500.5} currency="SAR"

empty · placeholder

status="error"

AmountWithCurrencyInput (currency picker inside)

1,500USD ▾

currencies=['SAR', 'USD', 'KWD']

PhoneWithCountryInput / PhoneInput

+966 ▾

512345678

+971 ▾

50 123 4567

value={{ country: 'SA', number: '512345678' }}

empty · AE

Dark theme

AmountInput

SAR999,999

ر.س0.00

SAR12,500.5

status="error"

empty · placeholder

value={12500.5} currency="SAR"

AmountWithCurrencyInput (currency picker inside)

▾ USD1,500

currencies=['SAR', 'USD', 'KWD']

PhoneWithCountryInput / PhoneInput

50 123 4567 ▾

+971

512345678 ▾

+966

empty · AE

value={{ country: 'SA', number: '512345678' }}

Arabic (ar-SA) · RTL

17 AmountField

Large centred amount for transfer and payment screens.

```
import { AmountField } from '@bunyan/design-system';
```

Variants & states Keyboard or NumPad (with decimal key); quick-amount chips; error.

Accessibility The amount input is labelled with label and currency; errors use alert semantics.

Example

```
<AmountField value={amount} onChangeValue={setAmount} currency="SAR" quickAmounts=[{100, 500, 1000}] hint="Available 12,480.00 SAR" />.
```

Do/don't Show the available balance or limits in `hint`.

Edge cases Same sanitising as AmountInput (Arabic digits, decimals limit).

AmountFieldProps

PROP	TYPE	DEFAULT
<code>value</code>	<code>number null</code>	—
<code>onChangeValue</code>	<code>(value: number null) => void</code>	—
<code>currency</code>	<code>string</code>	—
<code>decimals</code>	<code>number</code>	<code>2</code>
<code>label</code>	<code>TextValue</code>	—
<code>hint</code>	<code>TextValue</code> Line under the amount, e.g. "Available balance: 12,480.00 SAR".	—
<code>errorText</code>	<code>string</code>	—
<code>status</code>	<code>'default' 'error' 'success'</code>	<code>'default'</code>
<code>quickAmounts</code>	<code>readonly number[]</code> One-tap amounts shown as chips.	—
<code>useNumPad</code>	<code>boolean</code> Use the on-screen NumPad (with a decimal key) instead of the system keyboard.	<code>false</code>
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>autoFocus</code>	<code>boolean</code>	<code>false</code>
<code>placeholder</code>	<code>string</code>	<code>'0'</code>
<code>testID</code>	<code>string</code>	—

Amount

1,250 SAR

Available balance: 12,480.00 SAR

100

500

1,000

quickAmounts · hint

20,000 SAR

Exceeds your daily limit

1

2

3

4

5

6

7

8

9

.

0

⌫

useNumPad · errorText

All variants — light theme

Amount

1,250 SAR

Available balance: 12,480.00 SAR

100

500

1,000

quickAmounts · hint

20,000 SAR

Exceeds your daily limit

1

2

3

4

5

6

7

8

9

.

0

⌫

useNumPad · errorText

Dark theme

20,000 SAR

يتجاوز الحد اليومي

1	2	3
4	5	6
7	8	9
.	0	✕

useNumPad · errorText

المبلغ

1,250 SAR

الرصيد المتاح: 12,480.00 ر.س

1,000	500	100
-------	-----	-----

quickAmounts · hint

Arabic (ar-SA) · RTL

18 OTPInput

One-time-code slots usable anywhere (also used by OTPTemplate and the `OTP` form field).

```
import { OTPInput } from '@bunyan/design-system';
```

Variants & states	Keyboard or NumPad; secure; error; disabled.
Accessibility	Slot row announces progress; hidden native input supports paste and SMS autofill.
Example	<code><OTPInput value={code} onChange={setCode} length={6} onComplete={verify} />.</code>
Do/don't	Keep verification logic outside.
Edge cases	<code>onComplete</code> fires once per completed code.

OTPInputProps

PROP	TYPE	DEFAULT
<code>value</code> required	<code>string</code>	—
<code>onChange</code> required	<code>(value: string) => void</code>	—
<code>onComplete</code>	<code>(value: string) => void</code> Called once each time the code becomes complete.	—
<code>length</code>	<code>number</code>	<code>4</code>
<code>secure</code>	<code>boolean</code> Show dots instead of digits.	<code>false</code>
<code>error</code>	<code>boolean</code> Red slots. Pass <code>errorText</code> to also show a message under them.	<code>false</code>
<code>errorText</code>	<code>string</code>	—
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>useNumPad</code>	<code>boolean</code> Show Bunyan's NumPad and keep the system keyboard hidden.	<code>false</code>
<code>autoFocus</code>	<code>boolean</code>	<code>false</code>
<code>slotsLabel</code>	<code>(entered: number, length: number) => string</code> Screen-reader label of the slot row; receives entered/total counts.	<code>(entered, total) => `Verification code, \${entered} of \${total} digits entered`</code>
<code>inputAccessibilityLabel</code>	<code>string</code>	<code>'Verification code input'</code>
<code>testID</code>	<code>string</code>	—

1

2

3

4

length={6}

Incorrect code

secure · error

All variants — light theme

1

2

3

4

length={6}

Incorrect code

secure · error

Dark theme

رمز غير صحيح

secure · error

1

2

3

4

length={6}

Arabic (ar-SA) · RTL

19 FileInput

Upload box with the list of picked files.

```
import { FileInput } from '@bunyan/design-system';
```

Variants & states Empty, busy (picking), with files, error, disabled.

Accessibility Upload area is a labelled button with the hint; each file has a labelled remove button.

Example `<FileInput value={files} onChange={setFiles} multiple maxSize={5e6} hint="PDF or JPG, up to 5 MB" />.`

Do/don't Provide the platform picker once with `setDefaultFilePicker`; Bunyan ships no native module.

Edge cases Oversized files and files over `maxFiles` go to `onReject` with a reason.

FileInputProps

PROP	TYPE	DEFAULT
<code>value</code>	<code>readonly PickedFile[] null</code>	—
<code>onChange</code>	<code>(files: PickedFile[]) => void</code>	—
<code>pickFile</code>	<code>FilePicker</code>	—
<code>multiple</code>	<code>boolean</code>	<code>false</code>
<code>maxFiles</code>	<code>number</code>	<code>multiple ? Infinity : 1</code>
<code>maxSize</code>	<code>number</code> Maximum size per file in bytes. Larger files are rejected with <code>onReject</code> .	—
<code>onReject</code>	<code>(rejected: { file: PickedFile; reason: 'maxSize' 'maxFiles' }[]) => void</code> Called with files that were rejected and why.	—
<code>title</code>	<code>TextValue</code>	<code>'Upload file'</code>
<code>hint</code>	<code>TextValue</code> Hint under the title, e.g. "PDF or JPG, up to 5 MB".	—
<code>removeLabel</code>	<code>TextValue</code>	<code>'Remove'</code>
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>status</code>	<code>'default' 'error' 'success'</code>	<code>'default'</code>
<code>accessibilityLabel</code>	<code>TextValue</code>	—
<code>testID</code>	<code>string</code>	—

PickedFile

PROP	TYPE	DEFAULT
<code>uri</code> required	<code>string</code>	—
<code>name</code> required	<code>string</code>	—
<code>size</code>	<code>number null</code> Bytes.	—

PROP	TYPE	DEFAULT
type	string null MIME type, e.g. 'application/pdf'.	—

Upload documents

PDF or JPG, up to 5 MB

empty · hint

Add more

national-id.pdf

471 KB

salary-letter.jpg

1.3 MB

with files (multiple)

Upload file

status="error"

All variants — light theme

Upload documents

PDF or JPG, up to 5 MB

empty · hint

Add more

national-id.pdf

471 KB

salary-letter.jpg

1.3 MB

with files (multiple)

Upload file

status="error"

Dark theme

Bunyan Design System · Reference & AI Guide

62 / 147

Bunyan Design System · Reference & AI Guide

Choice & date controls

20 Checkbox

Selects zero or more independent options.

```
import { Checkbox } from '@bunyan/design-system';
```

- Variants & states**
- Checked, unchecked, mixed, disabled, error.
- Accessibility**
- Checkbox role and true/false/mixed state.
- Example**
- `<Checkbox checked={accepted} onChange={setAccepted} label="Accept terms" />.`
- Do/don't**
- Use for independent choices; do not use for mutually exclusive options.
- Edge cases**
- Pressing a mixed (indeterminate) checkbox always reports `true`.

CheckboxProps

PROP	TYPE	DEFAULT
<code>checked</code> required	<code>boolean</code>	—
<code>onChange</code> required	<code>(checked: boolean) => void</code>	—
<code>label</code> required	<code>string</code>	—
<code>description</code>	<code>string</code>	—
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>indeterminate</code>	<code>boolean</code>	<code>false</code>
<code>error</code>	<code>boolean</code>	<code>false</code>

☐ Accept terms
unchecked

☒ Accept terms
checked

☐ Select all
indeterminate

☒ Notifications
Email and SMS
description

☐ Required
error

☐ Locked
disabled

All variants — light theme

☐ Accept terms
unchecked

☒ Accept terms
checked

☐ Select all
indeterminate

☒ Notifications
Email and SMS
description

☐ Required
error

☐ Locked
disabled

Dark theme

تحديد الكل ☒

indeterminate

أوافق على الشروط ☒

checked

أوافق على الشروط ☐

unchecked

مقفّل ☒

disabled

مطلوب ☐

error

الإشعارات ☒

البريد والرسائل
description

Arabic (ar-SA) · RTL

21 CheckboxGroup

Choice groups — several checkboxes, picture cards, icon boxes and colour swatches.

```
import { CheckboxGroup } from '@bunyan/design-system';
```

Variants & states	Single/multiple; selected; disabled options; responsive columns.
Accessibility	Radio/checkbox roles with checked state; swatches are labelled with their name or value; select-all shows the mixed state.
Example	<pre><BoxGroup value={type} onChange={setType} options={[{ value: 'personal', title: 'Personal', icon: 'user' }]} />.</pre>
Do/don't	Give swatches a <code>label</code> (colour names) for screen readers.
Edge cases	Grids use <code>List</code> (equal-width cells, padded last row); swatch check mark picks black or white for contrast.

CheckboxGroupProps

PROP	TYPE	DEFAULT
<code>options</code> required	<code>readonly CheckboxGroupOption<V>[]</code>	—
<code>value</code>	<code>readonly V[] null</code>	—
<code>onChange</code>	<code>(value: V[]) => void</code>	—
<code>selectAllLabel</code>	<code>TextValue</code> Adds a "select all" checkbox (shows the mixed state when partly selected).	—
<code>max</code>	<code>number</code>	—
<code>layout</code>	<code>'column' 'row'</code>	<code>'column'</code>
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>error</code>	<code>boolean</code>	<code>false</code>
<code>accessibilityLabel</code>	<code>TextValue</code>	—
<code>testID</code>	<code>string</code>	—

CheckboxGroupOption

PROP	TYPE	DEFAULT
<code>label</code> required	<code>TextValue</code>	—
<code>value</code> required	<code>V</code>	—
<code>description</code>	<code>TextValue</code>	—
<code>disabled</code>	<code>boolean</code>	—

All products

Cards

Loans

Personal and auto

Savings

selectAllLabel (mixed)

SMS

Email

Push

layout="row"

All variants — light theme

All products

Cards

Loans

Personal and auto

Savings

selectAllLabel (mixed)

SMS

Email

Push

layout="row"

Dark theme

التنبيهات

البريد

SMS

كل المنتجات

layout="row"

البطاقات

التمويل

شخصي وسيارات

الادخار

selectAllLabel (mixed)

Arabic (ar-SA) · RTL

Bunyan Design System · Reference & AI Guide

67 / 147

22 Radio

Selects exactly one option in a set.

```
import { Radio } from '@bunyan/design-system';
```

- Variants & states

Selected, unselected, disabled.
- Accessibility

Radio role, checked state, and optional value.
- Example

`<Radio selected={plan === 'pro'} onSelect={() => setPlan('pro')} label="Pro" /> .`
- Do/don't

Present radios as a labelled group; do not offer only one radio.
- Edge cases

Re-pressing the selected radio is a no-op; product code owns arrow-key group navigation on web.

RadioProps

PROP	TYPE	DEFAULT
<code>selected</code> required	<code>boolean</code>	—
<code>onSelect</code> required	<code>() => void</code>	—
<code>label</code> required	<code>string</code>	—
<code>description</code>	<code>string</code>	—
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>value</code>	<code>string</code>	—

☒ Standard plan
selected

☐ Premium plan
unselected

☐ Enterprise
For 50+ seats
description

☐ Legacy
disabled

All variants — light theme

☒ Standard plan
selected

☐ Premium plan
unselected

☐ Enterprise
For 50+ seats
description

☐ Legacy
disabled

Dark theme

المؤسسات
لأكثر من 50 مستخدمًا
description

☐

الخطة المميزة

☐ unselected

الخطة القياسية

☒ selected

قديم

☐ disabled

Arabic (ar-SA) · RTL

23 RadioGroup

A labelled set of radios with one selected value.

```
import { RadioGroup } from '@bunyan/design-system';
```

- Variants & states

Column or row; selected, disabled.
- Accessibility

Container uses the radiogroup role; labels accept locale keys.
- Example

```
<RadioGroup value={plan} onChange={setPlan} options={[{ label: 'Basic', value: 'basic' }, { label: 'Pro', value: 'pro' }]} />.
```
- Do/don't

Offer at least two options.
- Edge cases

Values can be strings or numbers.

RadioGroupProps

PROP	TYPE	DEFAULT
options required	readonly RadioGroupOption<V>[]	—
value	V null	—
onChange	(value: V) => void	—
disabled	boolean	false
layout	'column' 'row' Stack (default) or place options side by side.	'column'
accessibilityLabel	TextValue	—
testID	string	—

RadioGroupOption

PROP	TYPE	DEFAULT
label required	TextValue	—
value required	V	—
description	TextValue	—
disabled	boolean	—

☒ Monthly

☐ Yearly
Save 20%

☐ Lifetime
layout="column"

☐ Basic

☒ Pro

☐ Team

layout="row"

All variants — light theme

☒ Monthly

☐ Yearly
Save 20%

☐ Lifetime

☐ Basic

☒ Pro

☐ Team

layout="row"

layout="column"

Dark theme

☐ فريق

☐ احترافي

☒ أساسي

☐

شهرى

☒

سنوي
وقر 20٪

☐

مدى الحياة

☐

layout="row"

layout="column"

Arabic (ar-SA) · RTL

24 Switch

Immediately toggles a setting.

```
import { Switch } from '@bunyan/design-system';
```

- Variants & states** On, off, disabled.
- Accessibility** Native switch role and checked state.
- Example** `<Switch value={enabled} onChange={setEnabled} label="Notifications" />.`
- Do/don't** Use for immediate effects; use Checkbox when submission is required.
- Edge cases** Persist failures should restore the prior value and show feedback.

SwitchProps

PROP	TYPE	DEFAULT
<code>value</code> required	<code>boolean</code>	—
<code>onChange</code> required	<code>(value: boolean) => void</code>	—
<code>label</code> required	<code>string</code>	—
<code>description</code>	<code>string</code>	—
<code>disabled</code>	<code>boolean</code>	<code>false</code>

Notifications

on

Dark mode

off

Biometrics

Use Face ID to sign in

description

Beta features

disabled

All variants — light theme

Notifications

on

Dark mode

off

Biometrics

Use Face ID to sign in

description

Beta features

disabled

Dark theme

الوضع الداكن

off

ميزات تجريبية

disabled

الإشعارات

on

البصمة

استخدم بصمة الوجه للدخول

description

Arabic (ar-SA) · RTL

25 Select

Chooses one value from a list.

```
import { Select } from '@bunyan/design-system';
```

Variants & states	Small/medium/large; expanded, disabled, error, selected, empty search.
Accessibility	Combobox role, current value, expanded and invalid state.
Example	<code><Select options={countries} value={country} onChange={setCountry} searchable />.</code>
Do/don't	Use search for long lists; do not use for fewer than three obvious choices.
Edge cases	Disabled options remain visible; missing values show the placeholder.

SelectProps

PROP	TYPE	DEFAULT
<code>options</code> required	<code>readonly SelectOption[]</code>	—
<code>value</code>	<code>string</code>	—
<code>onChange</code> required	<code>(value: string) => void</code>	—
<code>placeholder</code>	<code>string</code>	<code>'Select an option'</code>
<code>title</code>	<code>string</code>	<code>'Select an option'</code>
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>status</code>	<code>'default' 'error' 'success'</code>	<code>'default'</code>
<code>size</code>	<code>'small' 'medium' 'large'</code>	<code>'medium'</code>
<code>searchable</code>	<code>boolean</code>	<code>false</code>
<code>searchPlaceholder</code>	<code>string</code>	<code>'Search options'</code>
<code>emptyMessage</code>	<code>string</code>	<code>'No options found'</code>
<code>accessibilityLabel</code>	<code>string</code>	—

SelectOption

PROP	TYPE	DEFAULT
<code>value</code> required	<code>string</code>	—
<code>label</code> required	<code>string</code>	—
<code>description</code>	<code>string</code>	—
<code>disabled</code>	<code>boolean</code>	—

state

Select a city

▼

placeholder

Riyadh

▼

selected

Required

▼

status=error

Jeddah

▼

disabled

size

Dammam

▼

small

Dammam

▼

medium

Dammam

▼

large

All variants — light theme

state

Select a city

▼

placeholder

Riyadh

▼

selected

Required

▼

status=error

Jeddah

▼

disabled

size

Dammam

▼

small

Dammam

▼

medium

Dammam

▼

large

Dark theme

state

▼

مطلوب

status=error

▼

الرياض

selected

▼

اختر مدينة

placeholder

▼

جدة

disabled

size

▼

الدمام

large

▼

الدمام

medium

▼

الدمام

small

Arabic (ar-SA) · RTL

Open (searchable) sheet

Jeddah

Select a city

Q Search cities

Riyadh
Central region

Jeddah
Western region

Dammam
Eastern region

Abha

Light

Jeddah

Select a city

Q Search cities

Riyadh
Central region

Jeddah
Western region

Dammam
Eastern region

Abha

Dark

اختر مدينة

ابحث عن مدينة

الرياض
المنطقة الوسطى

Jدة
المنطقة الغربية

الدمام
المنطقة الشرقية

أبها

Arabic · RTL

26 ChipsGroup

Single- or multi-select set of chips (filters, segments, tags).

```
import { ChipsGroup } from '@bunyan/design-system';
```

Variants & states Single, multiple, wrapping or horizontally scrolling; selected and disabled chips.

Accessibility Each chip announces its selected state; texts accept locale keys.

Example

```
<ChipsGroup value={dir} onChange={setDir} data={[{ text: 'All', value: '0' }, { text: 'In', value: '1', leftIcon: 'arrow-down-left' }]} />.
```

Do/don't Use for 2–8 short options; use Select for long lists.

Edge cases Single mode ignores re-pressing the selected chip; multiple mode stops at `max`.

ChipsGroupSingleProps

PROP	TYPE	DEFAULT
<code>data</code> required	<code>readonly ChipsGroupItem<V>[]</code>	—
<code>multiple</code>	<code>false</code>	—
<code>value</code>	<code>V null</code>	—
<code>onChange</code>	<code>(value: V) => void</code>	—
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>scrollable</code>	<code>boolean</code> One horizontally scrolling line instead of wrapping rows.	<code>false</code>
<code>accessibilityLabel</code>	<code>TextValue</code>	—
<code>testID</code>	<code>string</code>	—

Also accepts: `ChipsGroupBaseProps<V>` (native React Native props).

ChipsGroupMultipleProps

PROP	TYPE	DEFAULT
<code>data</code> required	<code>readonly ChipsGroupItem<V>[]</code>	—
<code>multiple</code> required	<code>true</code>	—
<code>value</code>	<code>readonly V[] null</code>	—
<code>onChange</code>	<code>(value: V[]) => void</code>	—
<code>max</code>	<code>number</code> Maximum number of selected chips.	—
<code>disabled</code>	<code>boolean</code>	—
<code>scrollable</code>	<code>boolean</code> One horizontally scrolling line instead of wrapping rows.	—
<code>accessibilityLabel</code>	<code>TextValue</code>	—
<code>testID</code>	<code>string</code>	—

Also accepts: `ChipsGroupBaseProps<V>` (native React Native props).

ChipsGroupItem

PROP		TYPE	DEFAULT
text	required	TextValue	—
value	required	V	—
leftIcon		IconName Icon before the text.	—
disabled		boolean	—
accessibilityLabel		TextValue	—

single · data[].leftIcon

All

↵ Incoming

↗ Outgoing

multiple · max={2}

Card

Transfer

Bills

disabled

All

Incoming

All variants — light theme

single · data[].leftIcon

All

↵ Incoming

↗ Outgoing

multiple · max={2}

Card

Transfer

Bills

disabled

All

Incoming

Dark theme

single · data[].leftIcon

↗ صادر

↵ وارد

الكل

multiple · max={2}

فواتير

تحويل

بطاقة

disabled

وارد

الكل

Arabic (ar-SA) · RTL

27 RadiolImageGroup

Choice groups — several checkboxes, picture cards, icon boxes and colour swatches.

```
import { RadiolImageGroup } from '@bunyan/design-system';
```

Variants & states	Single/multiple; selected; disabled options; responsive columns.
Accessibility	Radio/checkbox roles with checked state; swatches are labelled with their name or value; select-all shows the mixed state.
Example	<code><BoxGroup value={type} onChange={setType} options={[{ value: 'personal', title: 'Personal', icon: 'user' }]} />.</code>
Do/don't	Give swatches a <code>label</code> (colour names) for screen readers.
Edge cases	Grids use <code>List</code> (equal-width cells, padded last row); swatch check mark picks black or white for contrast.

RadiolImageGroupProps

PROP	TYPE	DEFAULT
<code>options</code> required	<code>readonly OptionTile<V>[]</code> <code>{ value, title, description?, image, disabled? }</code>	—
<code>value</code>	<code>V null</code>	—
<code>onChange</code>	<code>(value: V) => void</code>	—
<code>columns</code>	<code>Responsive<keyof Theme['breakpoint'], number></code>	—
<code>imageAspectRatio</code>	<code>number</code> Width / height of the pictures. Default <code>4/3</code> .	—
<code>disabled</code>	<code>boolean</code>	—
<code>accessibilityLabel</code>	<code>TextValue</code>	—
<code>testID</code>	<code>string</code>	—

OptionTile

PROP	TYPE	DEFAULT
<code>value</code> required	<code>V</code>	—
<code>title</code> required	<code>TextValue</code>	—
<code>description</code>	<code>TextValue</code>	—
<code>image</code>	<code>ImageSourcePropType</code> Picture on top of the tile (RadiolImageGroup).	—
<code>icon</code>	<code>IconName</code> Icon on the tile (BoxGroup).	—
<code>disabled</code>	<code>boolean</code>	—



28 BoxGroup

Choice groups — several checkboxes, picture cards, icon boxes and colour swatches.

```
import { BoxGroup } from '@bunyan/design-system';
```

Variants & states	Single/multiple; selected; disabled options; responsive columns.
Accessibility	Radio/checkbox roles with checked state; swatches are labelled with their name or value; select-all shows the mixed state.
Example	<pre><BoxGroup value={type} onChange={setType} options={[{ value: 'personal', title: 'Personal', icon: 'user' }]} />.</pre>
Do/don't	Give swatches a <code>label</code> (colour names) for screen readers.
Edge cases	Grids use <code>List</code> (equal-width cells, padded last row); swatch check mark picks black or white for contrast.

BoxGroupSingleProps

PROP	TYPE	DEFAULT
<code>multiple</code>	<code>false</code>	—
<code>value</code>	<code>V null</code>	—
<code>onChange</code>	<code>(value: V) => void</code>	—
<code>options</code> required	<code>readonly OptionTile<V>[]</code> <code>{ value, title, description?, icon?, disabled? }</code>	—
<code>columns</code>	<code>Responsive<keyof Theme['breakpoint'], number></code>	—
<code>disabled</code>	<code>boolean</code>	—
<code>accessibilityLabel</code>	<code>TextValue</code>	—
<code>testID</code>	<code>string</code>	—

Also accepts: `BoxGroupBase<V>` (native React Native props).

BoxGroupMultipleProps

PROP	TYPE	DEFAULT
<code>multiple</code> required	<code>true</code>	—
<code>value</code>	<code>readonly V[] null</code>	—
<code>onChange</code>	<code>(value: V[]) => void</code>	—
<code>max</code>	<code>number</code>	—
<code>options</code> required	<code>readonly OptionTile<V>[]</code> <code>{ value, title, description?, icon?, disabled? }</code>	—
<code>columns</code>	<code>Responsive<keyof Theme['breakpoint'], number></code>	—
<code>disabled</code>	<code>boolean</code>	—
<code>accessibilityLabel</code>	<code>TextValue</code>	—
<code>testID</code>	<code>string</code>	—

Also accepts: `BoxGroupBase<V>` (native React Native props).

OptionTile

PROP		TYPE	DEFAULT
value	required	V	—
title	required	TextValue	—
description		TextValue	—
image		ImageSourcePropType Picture on top of the tile (RadiolImageGroup).	—
icon		IconName Icon on the tile (BoxGroup).	—
disabled		boolean	—

single · columns={3}



Personal
Salary and savings



Business
For companies



Investment
Funds and stocks

multiple · max={2}



Travel



Shopping

All variants — light theme

single · columns={3}



Personal
Salary and savings



Business
For companies



Investment
Funds and stocks

multiple · max={2}



Travel



Shopping

Dark theme

single · columns={3}

↑

استثماري
صناديق وأسهم

📄

تجاري
للشركات

👤

شخصي
الراتب والادخار

✓

multiple · max={2}

🔍

التسوق

📅

السفر

✓

Arabic (ar-SA) · RTL

Bunyan Design System · Reference & AI Guide

81 / 147

29 SwatchGroup

Choice groups — several checkboxes, picture cards, icon boxes and colour swatches.

```
import { SwatchGroup } from '@bunyan/design-system';
```

Variants & states Single/multiple; selected; disabled options; responsive columns.

Accessibility Radio/checkbox roles with checked state; swatches are labelled with their name or value; select-all shows the mixed state.

Example `<BoxGroup value={type} onChange={setType} options={[{ value: 'personal', title: 'Personal', icon: 'user' }]} />.`

Do/don't Give swatches a `label` (colour names) for screen readers.

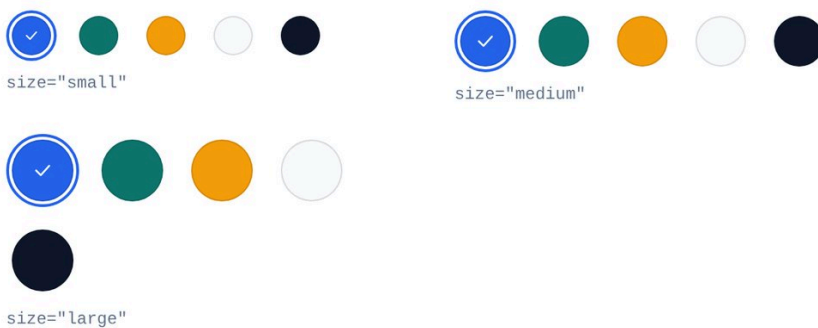
Edge cases Grids use `List` (equal-width cells, padded last row); swatch check mark picks black or white for contrast.

SwatchGroupProps

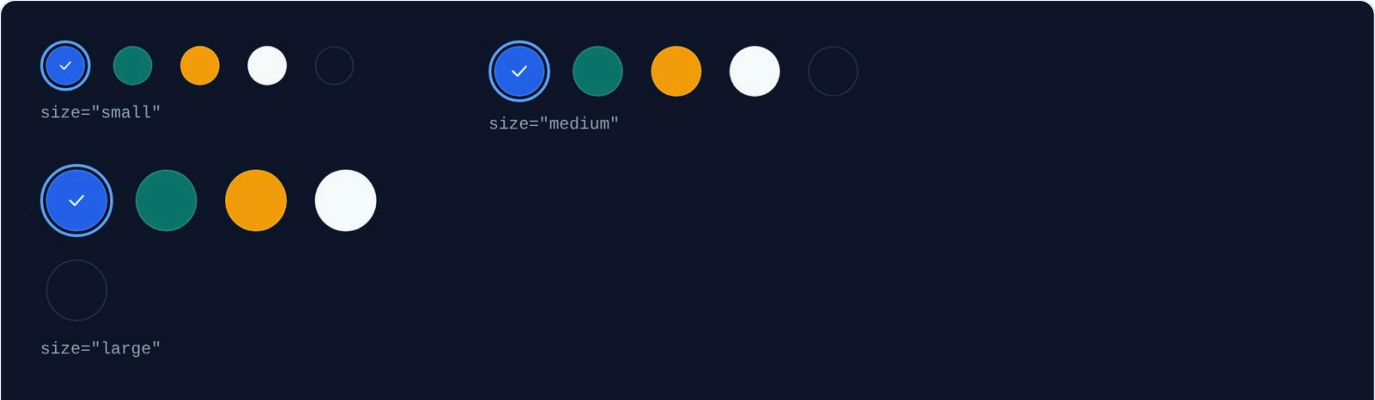
PROP	TYPE	DEFAULT
<code>options</code> required	<code>readonly (string SwatchOption)[]</code>	—
<code>value</code>	<code>string null</code>	—
<code>onChange</code>	<code>(value: string) => void</code>	—
<code>size</code>	<code>'small' 'medium' 'large'</code>	<code>'medium'</code>
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>accessibilityLabel</code>	<code>TextValue</code>	—
<code>testID</code>	<code>string</code>	—

SwatchOption

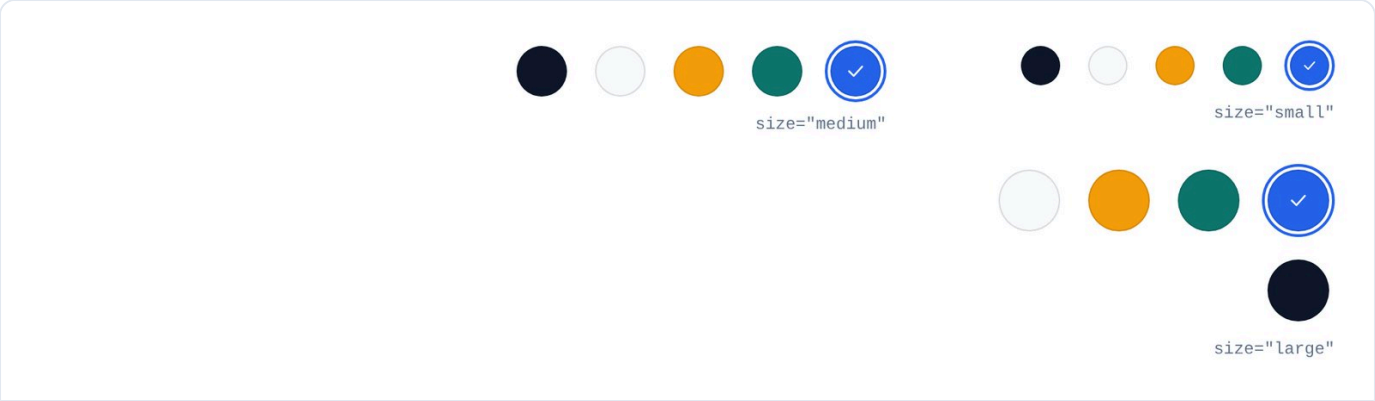
PROP	TYPE	DEFAULT
<code>value</code>	<code>string</code> Stored value (defaults to <code>color</code>).	—
<code>color</code> required	<code>string</code> Any CSS/RN colour.	—
<code>label</code>	<code>TextValue</code>	—
<code>disabled</code>	<code>boolean</code>	—



All variants — light theme



Dark theme



Arabic (ar-SA) · RTL

30 Slider

Choose a number in a range.

```
import { Slider } from '@bunyan/design-system';
```

- Variants & states

With/without value and limits; disabled.
- Accessibility

Adjustable role with increment/decrement actions and a value text.
- Example

```
<Slider label="Installments" min={3} max={24} step={3} value={n} onChange={setN} formatValue={v => `${v} months`} />.
```
- Do/don't

Prefer an input for exact values over large ranges.
- Edge cases

Dependency-free (PanResponder); mirrors in RTL.

SliderProps

PROP	TYPE	DEFAULT
value	number	—
onChange	(value: number) => void	—
onChangeEnd	(value: number) => void Called when the user lifts the finger.	—
min	number	0
max	number	100
step	number	1
disabled	boolean	false
label	TextValue	—
showValue	boolean Show the current value next to the label. Default <code>true</code> .	true
formatValue	(value: number) => string	v => String(v)
showLimits	boolean Show min / max under the track.	false
testID	string	—

Installments

12 months

Limit

40

3 months

24 months

disabled

value={12} step={3} showLimits

Limit

40

disabled

All variants — light theme

Installments

12 months

Limit

40

3 months

24 months

disabled

value={12} step={3} showLimits

Limit

40

disabled

Dark theme

40

الحد

12 شهرًا

الاقساط

disabled

شهرًا 24

3 شهرًا

value={12} step={3} showLimits

Arabic (ar-SA) · RTL

31 DatePicker

Selects a calendar date consistently across native platforms.

```
import { DatePicker } from '@bunyan/design-system';
```

Variants & states Empty, selected, open, disabled, error.

Accessibility Pressable trigger with button role, current value and expanded state; clear is a separate labelled button.

Example `<DatePicker value={date} onChange={setDate} maximumDate={new Date()} />.`

Do/don't Pass a locale and valid bounds; do not parse display text manually.

Edge cases Label uses the Gregorian calendar by default (ar-SA would otherwise show Hijri); override with `formatOptions`. Value is clamped to min/max on confirm.

DatePickerProps

PROP	TYPE	DEFAULT
<code>value</code>	<code>Date</code>	—
<code>onChange</code> required	<code>(date: Date undefined) => void</code>	—
<code>minimumDate</code>	<code>Date</code>	—
<code>maximumDate</code>	<code>Date</code>	—
<code>locale</code>	<code>string</code>	—
<code>placeholder</code>	<code>string</code>	<code>'Select date'</code>
<code>title</code>	<code>string</code>	—
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>status</code>	<code>'default' 'error' 'success'</code>	<code>'default'</code>
<code>size</code>	<code>'small' 'medium' 'large'</code>	<code>'medium'</code>
<code>accessibilityLabel</code>	<code>string</code>	<code>'Date'</code>
<code>clearable</code>	<code>boolean</code>	<code>true</code>
<code>clearLabel</code>	<code>string</code>	<code>'Clear date'</code>
<code>confirmLabel</code>	<code>string</code>	<code>'Confirm'</code>
<code>formatOptions</code>	<code>Intl.DateTimeFormatOptions</code> Display format. Defaults to the Gregorian calendar so the label matches the (Gregorian) native picker — <code>ar-SA</code> would otherwise format as Hijri. Pass <code>{ calendar: 'islamic-umalqura' }</code> to opt into Hijri display.	—
<code>testID</code>	<code>string</code>	—

Select date

empty

Oct 3, 2026

×

with value (clearable)

Date of birth

status=error

Jan 15, 2026

disabled

All variants — light theme

Select date

empty

Oct 3, 2026

×

with value (clearable)

Date of birth

status=error

Jan 15, 2026

disabled

Dark theme

تاريخ الميلاد

status=error

٣ أكتوبر ٢٠٢٦

×

with value (clearable)

اختر التاريخ

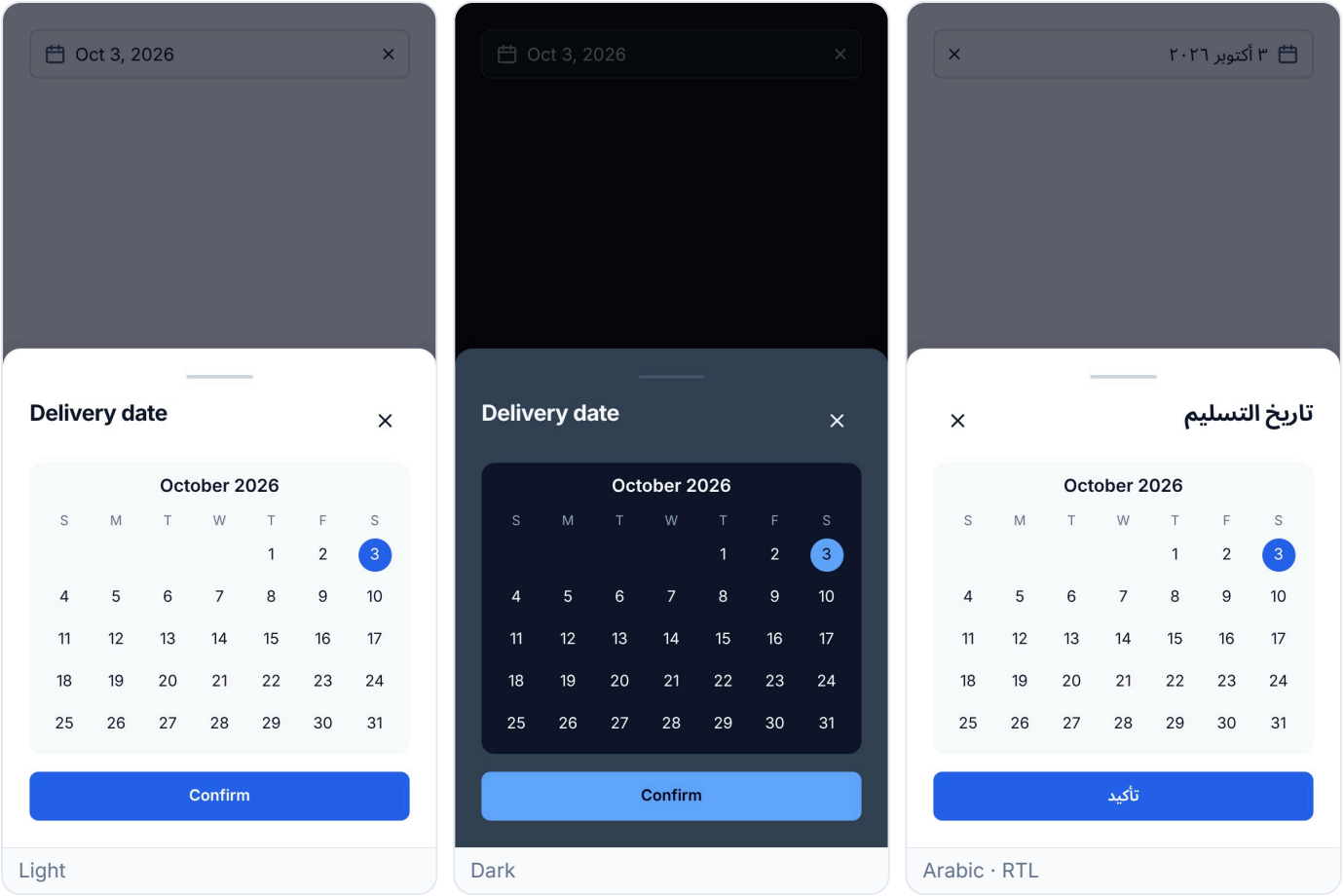
empty

١٥ يناير ٢٠٢٦

disabled

Arabic (ar-SA) · RTL

Open picker sheet (iOS / web presentation)



32 DateRangePicker

From/to date selection with an optional Gregorian/Hijri display toggle.

```
import { DateRangePicker } from '@bunyan/design-system';
```

Variants & states	Row or column; empty, partial, complete; Gregorian or Hijri display; error; disabled.
Accessibility	Each side is a labelled DatePicker; the calendar toggle is a ChipsGroup.
Example	<code><DateRangePicker value={range} onChange={setRange} maximumDate={new Date()} showHijriToggle />.</code>
Do/don't	Validate with Bunyan's <code>Yup.mixed().dateRange()</code> , <code>dateRangeSchema()</code> or <code>validateDateRange()</code> .
Edge cases	Each side is bounded by the other, so <code>from</code> $\leq$ <code>to</code> ; Hijri changes display only — values stay <code>Date</code> s and the native picker grid stays Gregorian.

DateRangePickerProps

PROP	TYPE	DEFAULT
<code>value</code>	<code>DateRange null</code>	—
<code>onChange</code> required	<code>(range: DateRange) => void</code>	—
<code>fromLabel</code>	<code>TextValue</code>	<code>'From'</code>
<code>toLabel</code>	<code>TextValue</code>	<code>'To'</code>
<code>fromPlaceholder</code>	<code>TextValue</code>	<code>'Start date'</code>
<code>toPlaceholder</code>	<code>TextValue</code>	<code>'End date'</code>
<code>minimumDate</code>	<code>Date</code>	—
<code>maximumDate</code>	<code>Date</code>	—
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>status</code>	<code>'default' 'error' 'success'</code>	<code>'default'</code>
<code>layout</code>	<code>'row' 'column'</code> Side by side (default) or stacked.	<code>'row'</code>
<code>locale</code>	<code>string</code>	—
<code>showHijriToggle</code>	<code>boolean</code> Show a Gregorian / Hijri switch for how dates are displayed.	<code>false</code>
<code>calendar</code>	<code>'gregory' 'islamic-umalqura'</code> Controlled display calendar.	—
<code>defaultCalendar</code>	<code>'gregory' 'islamic-umalqura'</code>	<code>'gregory'</code>
<code>onCalendarChange</code>	<code>(calendar: 'gregory' 'islamic-umalqura') => void</code>	—
<code>gregorianLabel</code>	<code>TextValue</code>	<code>'Gregorian'</code>
<code>hijriLabel</code>	<code>TextValue</code>	<code>'Hijri'</code>
<code>clearable</code>	<code>boolean</code>	<code>true</code>
<code>testID</code>	<code>string</code>	—

DateRange

PROP	TYPE	DEFAULT
from	Date undefined	—
to	Date undefined	—

showHijriToggle · maximumDate

Gregorian

Hijri

From

To

📅 Sep 1, 2026

×

📅 Oct 3, 2026

×

Gregorian

Hijri

From

To

📅 Rab. I 19, 1...

×

📅 Rab. II 22, ...

×

defaultCalendar="islamic-umalqura"

empty · status="error" · layout="column"

From

To

📅 Start date

×

📅 End date

×

From

To

📅 Start date

×

📅 End date

×

empty

status="error"

All variants — light theme

showHijriToggle · maximumDate

Gregorian

Hijri

From

To

📅 Sep 1, 2026

×

📅 Oct 3, 2026

×

Gregorian

Hijri

From

To

📅 Rab. I 19, 1...

×

📅 Rab. II 22, ...

×

defaultCalendar="islamic-umalqura"

empty · status="error" · layout="column"

From

To

📅 Start date

×

📅 End date

×

From

To

📅 Start date

×

📅 End date

×

empty

status="error"

Dark theme

showHijriToggle · maximumDate

هجري

ميلادي

إلى

من

×

٢٢ ربيع الآخر...

×

×

١٩ ربيع الأول...

×

إلى

من

×

٣ أكتوبر ٢٠٢٦

×

×

١ سبتمبر ٢٠٢٦

×

defaultCalendar="islamic-umalqura"

empty · status="error" · layout="column"

إلى

من

End date

📅

Start date

📅

إلى

من

End date

📅

Start date

📅

status="error"

empty

Arabic (ar-SA) · RTL

Bunyan Design System · Reference & AI Guide

90 / 147

Data display

33 Badge

Displays compact read-only status or metadata.

```
import { Badge } from '@bunyan/design-system';
```

- Variants & states**
- Neutral, primary, success, warning, error, information; small/medium.
- Accessibility**
- Text remains the source of meaning; color is supplemental.
- Example**
- `<Badge tone="success">Approved</Badge>` .
- Do/don't**
- Keep copy short; do not make badges interactive.
- Edge cases**
- Long localized status text may wrap; use a Chip if interaction is needed.

BadgeProps

PROP	TYPE	DEFAULT
<code>children</code> required	<code>React.ReactNode</code>	—
<code>tone</code>	<code>'neutral' 'primary' 'success' 'warning' 'error' 'information'</code>	<code>'neutral'</code>
<code>size</code>	<code>'small' 'medium'</code>	<code>'medium'</code>
<code>accessibilityLabel</code>	<code>string</code>	—

size="medium"

Neutral

Primary

Success

Warning

Error

Information

neutral

primary

success

warning

error

information

size="small"

Neutral

Primary

Success

Warning

Error

Information

neutral

primary

success

warning

error

information

All variants — light theme

size="medium"

Neutral

Primary

Success

Warning

Error

Information

neutral

primary

success

warning

error

information

size="small"

Neutral

Primary

Success

Warning

Error

Information

neutral

primary

success

warning

error

information

Dark theme

حالة

information

حالة

error

حالة

warning

حالة

success

حالة

primary

حالة

neutral

size="medium"

حالة

information

حالة

error

حالة

warning

حالة

success

حالة

primary

حالة

neutral

size="small"

Arabic (ar-SA) · RTL

34 Chip

Represents a filter, selection, or removable value.

```
import { Chip } from '@bunyan/design-system';
```

- Variants & states

Default, selected, pressed, disabled, removable.
- Accessibility

Label and remove are sibling buttons (never nested), each with its own label and state.
- Example

`<Chip label="Finance" selected onRemove={removeFinance} />.`
- Do/don't

Use concise nouns; do not nest arbitrary controls inside a chip.
- Edge cases

Removing a selected filter should update focus predictably.

ChipProps

PROP	TYPE	DEFAULT
<code>label</code> required	<code>string</code>	—
<code>selected</code>	<code>boolean</code>	<code>false</code>
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>onPress</code>	<code>() => void</code>	—
<code>onRemove</code>	<code>() => void</code>	—
<code>accessibilityLabel</code>	<code>string</code>	—
<code>removeLabel</code>	<code>(label: string) => string</code>	<code>value => `Remove \${value}`</code>
<code>leadingIcon</code>	<code>IconName</code> Icon before the label.	—
<code>testID</code>	<code>string</code>	—

Enterprise

Enterprise

Riyadh ×

Riyadh ×

Archived

default

selected

removable

selected + removable

disabled

All variants — light theme

Enterprise

Enterprise

Riyadh ×

Riyadh ×

Archived

default

selected

removable

selected + removable

disabled

Dark theme

مؤرشف

× الرياض

× الرياض

المؤسسات

المؤسسات

disabled

selected + removable

removable

selected

default

Arabic (ar-SA) · RTL

35 Avatar

Identifies a person or entity with image, initials, or fallback icon.

```
import { Avatar } from '@bunyan/design-system';
```

- Variants & states**
- Four sizes; image, initials, fallback, image-error fallback.
- Accessibility**
- Exposes an image role and meaningful label.
- Example**
- `<Avatar name="Fatimah Ali" source={{ uri }} />.`
- Do/don't**
- Provide `name`; do not use an avatar as the only identity label in critical flows.
- Edge cases**
- Initials use the first two whitespace-separated words.

AvatarProps

PROP	TYPE	DEFAULT
<code>source</code>	<code>ImageSourcePropType</code>	—
<code>name</code>	<code>string</code>	—
<code>size</code>	<code>'small' 'medium' 'large' 'xlarge'</code>	<code>'medium'</code>
<code>accessibilityLabel</code>	<code>string</code>	—

size="small"


image


initials


fallback icon

size="medium"


image


initials


fallback icon

size="large"


image


initials


fallback icon

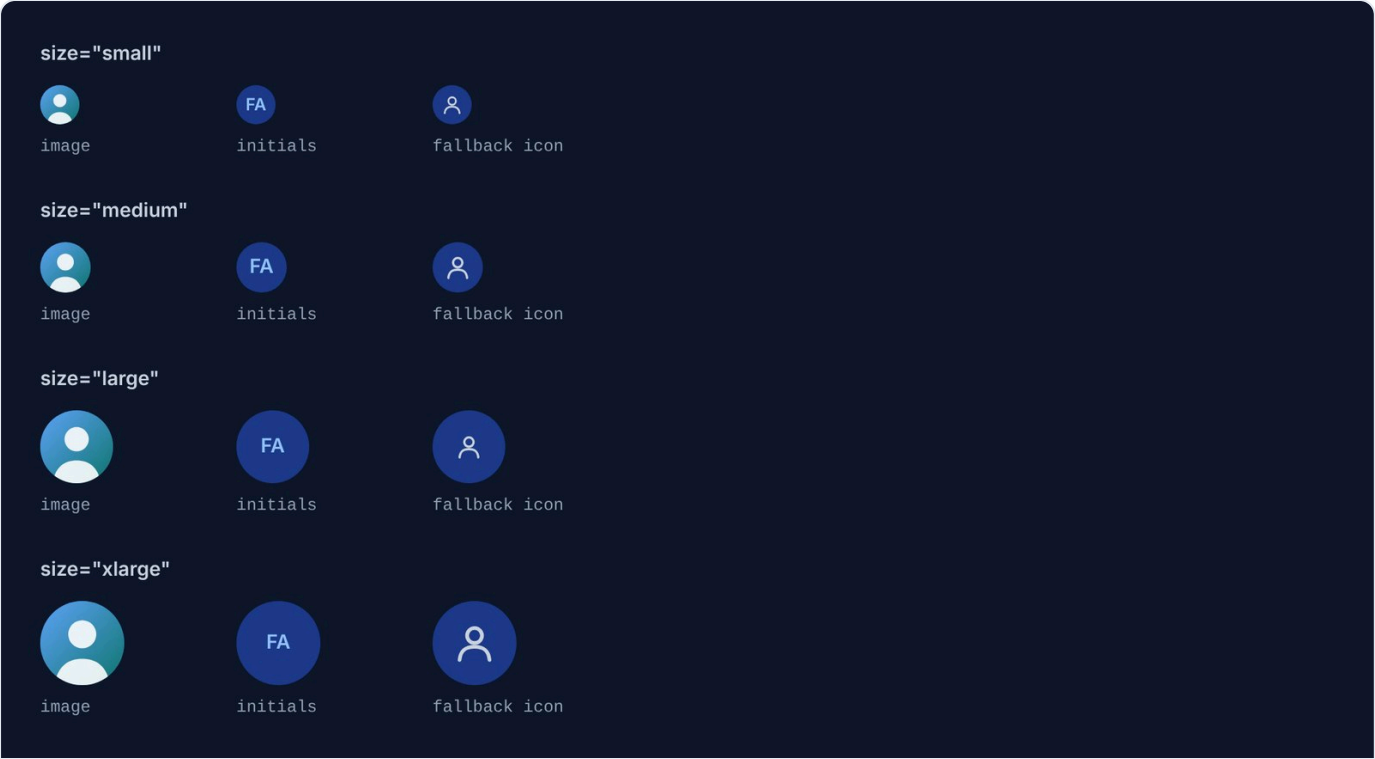
size="xlarge"


image

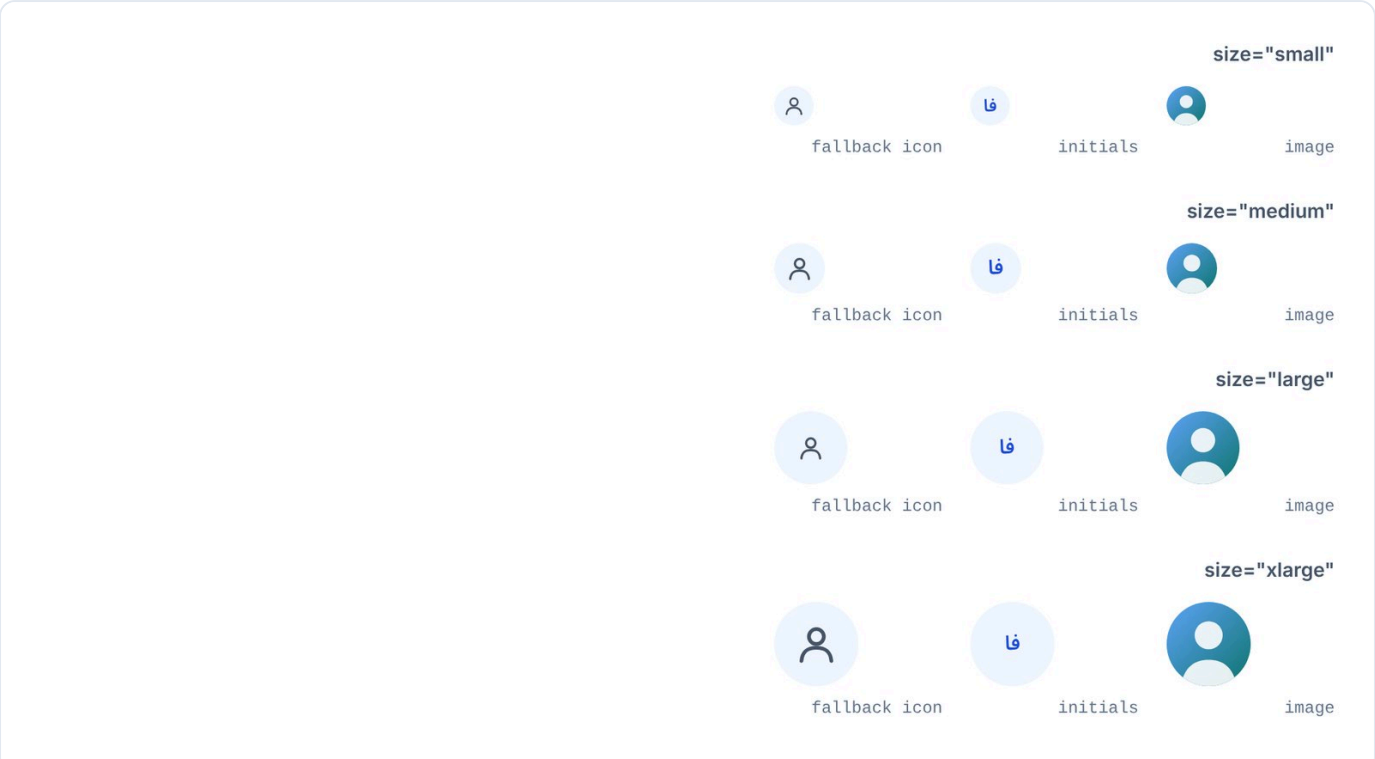

initials


fallback icon

All variants — light theme



Dark theme



Arabic (ar-SA) · RTL

36 Divider

Separates adjacent content groups.

```
import { Divider } from '@bunyan/design-system';
```

- Variants & states**
- Horizontal/vertical and four inset levels.
- Accessibility**
- Decorative by default (hidden from screen readers); `decorative={false}` exposes the separator role.
- Example**
- `<Divider inset="medium" />.`
- Do/don't**
- Use spacing before adding dividers; do not over-segment simple layouts.
- Edge cases**
- Vertical dividers need a parent with defined height.

DividerProps

PROP	TYPE	DEFAULT
<code>orientation</code>	<code>'horizontal' 'vertical'</code>	<code>'horizontal'</code>
<code>inset</code>	<code>'none' 'small' 'medium' 'large'</code>	<code>'none'</code>
<code>decorative</code>	<code>boolean</code>	<code>true</code>

orientation="horizontal" inset="none"

orientation="horizontal" inset="small"

orientation="horizontal" inset="medium"

orientation="horizontal" inset="large"

orientation="vertical"

Left | Right

All variants — light theme

orientation="horizontal" inset="none"

orientation="horizontal" inset="small"

orientation="horizontal" inset="medium"

orientation="horizontal" inset="large"

orientation="vertical"

Left | Right

Dark theme

orientation="horizontal" inset="none"

orientation="horizontal" inset="small"

orientation="horizontal" inset="medium"

orientation="horizontal" inset="large"

orientation="vertical"

يسار | يمين

Arabic (ar-SA) · RTL

37 Card

Groups related content or exposes a single card-level action.

```
import { Card } from '@bunyan/design-system';
```

Variants & states	Elevated, outlined, filled; four padding levels; pressed.
Accessibility	Interactive cards require an accessibility label and use button semantics.
Example	<code><Card variant="elevated"><Text>Summary</Text></Card></code> .
Do/don't	Keep one information hierarchy; do not nest multiple large interactive regions.
Edge cases	Use <code>padding="none"</code> for edge-to-edge media.

CardProps

PROP	TYPE	DEFAULT
<code>children</code> required	<code>React.ReactNode</code>	—
<code>variant</code>	<code>'elevated' 'outlined' 'filled'</code>	<code>'outlined'</code>
<code>padding</code>	<code>'none' 'small' 'medium' 'large'</code>	<code>'medium'</code>
<code>onPress</code>	<code>Nullable<PressableProps['onPress']></code>	—
<code>accessibilityLabel</code>	<code>string</code> Overrides the text that's read by the screen reader when the user interacts with the element. By default, the label is constructed by traversing all the children and accumulating all the Text nodes separated by space.	—

Also accepts: `Omit<PressableProps>`, `Omit<ViewProps>` (native React Native props).

variant

Balance
SAR 12,480.00

Balance
SAR 12,480.00

Balance
SAR 12,480.00

outlined elevated filled

Open account
Tap to view

pressable (onPress)

padding

Content

Content

Content

Content

none small medium large

All variants — light theme

variant

Balance
SAR 12,480.00

Balance
SAR 12,480.00

Balance
SAR 12,480.00

outlined elevated filled

Open account
Tap to view

pressable (onPress)

padding

Content

Content

Content

Content

none small medium large

Dark theme

variant

الرصيد
SAR 12,480.00

الرصيد
SAR 12,480.00

الرصيد
SAR 12,480.00

filled elevated outlined

فتح الحساب
اضغط للعرض

pressable (onPress)

padding

محتوى

محتوى

محتوى

محتوى

large medium small none

Arabic (ar-SA) · RTL

38 ListItem

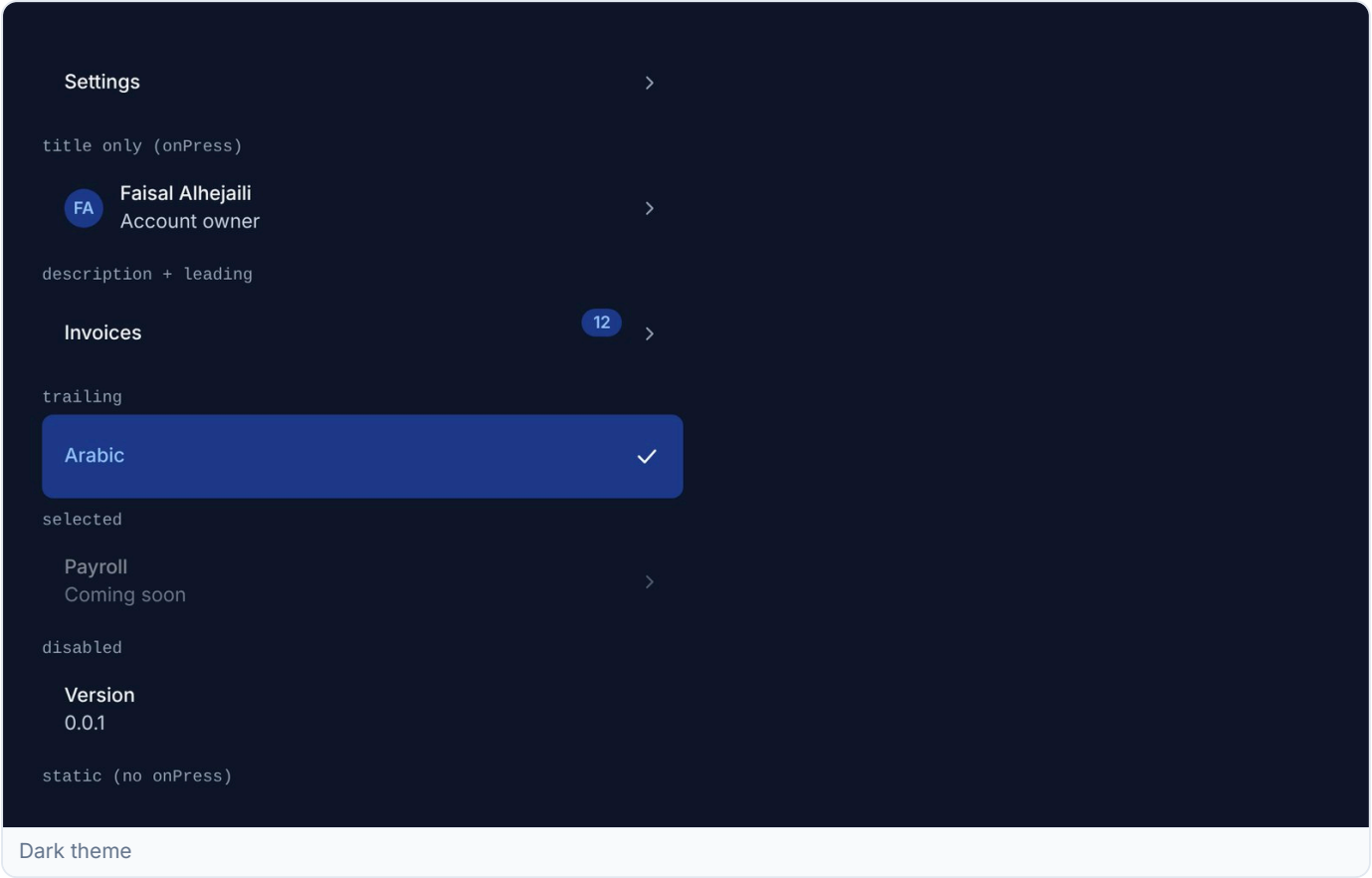
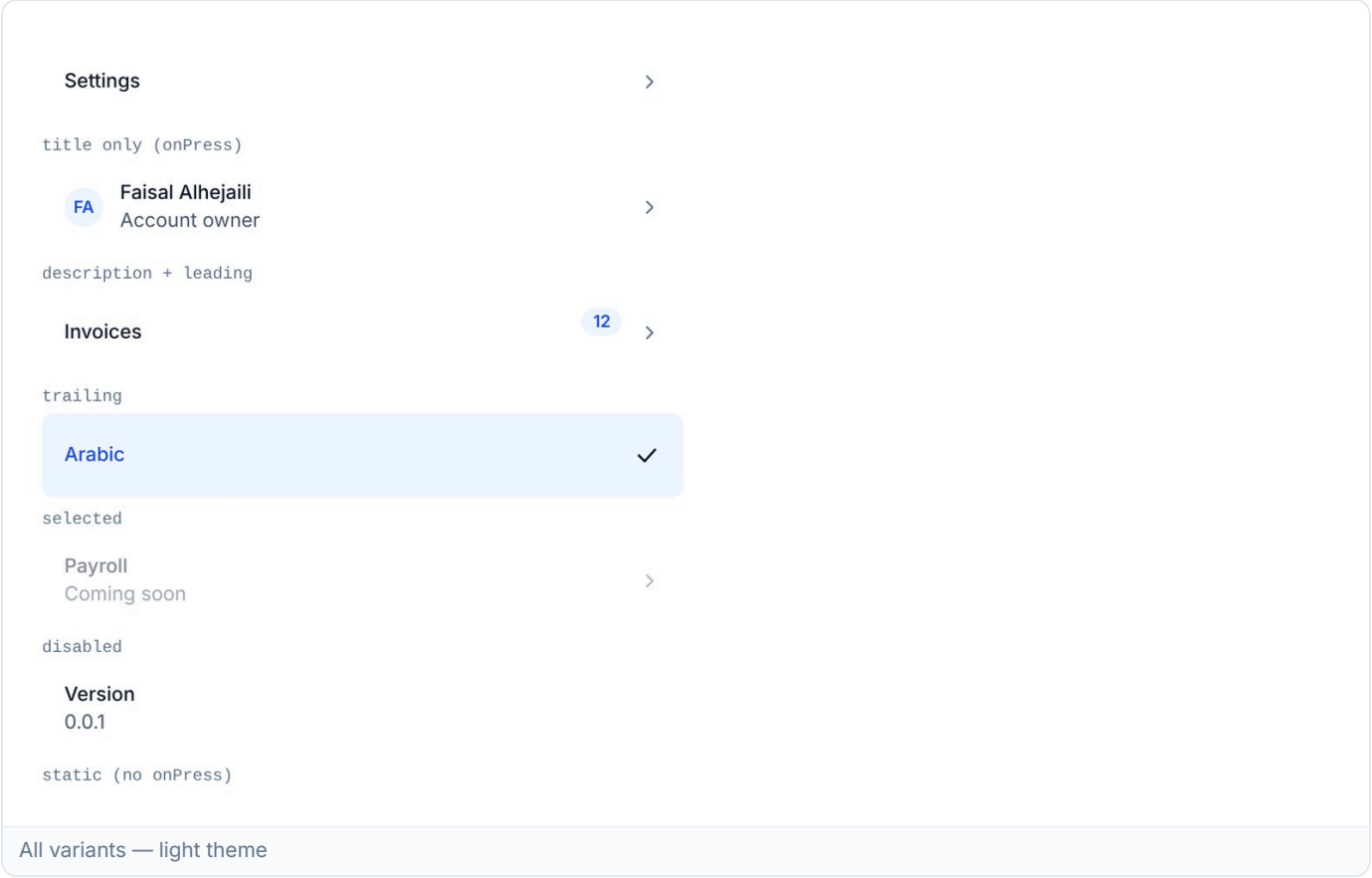
Standard row for settings, navigation, selection, or metadata.

```
import { ListItem } from '@bunyan/design-system';
```

- Variants & states**
- Static, interactive, pressed, selected, disabled.
- Accessibility**
- Interactive rows expose button role and selected state.
- Example**
- `<ListItem title="Security" description="Password and devices" onPress={openSecurity} />.`
- Do/don't**
- Put the primary label in `title`; do not hide critical actions in an unlabeled trailing slot.
- Edge cases**
- Long descriptions wrap while trailing content remains aligned.

ListItemProps

PROP	TYPE	DEFAULT
<code>title</code> required	<code>string</code>	—
<code>description</code>	<code>string</code>	—
<code>leading</code>	<code>React.ReactNode</code>	—
<code>trailing</code>	<code>React.ReactNode</code>	—
<code>onPress</code>	<code>() => void</code>	—
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>selected</code>	<code>boolean</code>	<code>false</code>
<code>showChevron</code>	<code>boolean</code>	<code>Boolean(onPress)</code>
<code>chevronIcon</code>	<code>IconName</code>	<code>'chevron-right'</code>



<

الإعدادات

title only (onPress)

<

فيصل الحجيلي
مالك الحساب

FA

description + leading

<

12

الفواتير

trailing

✓

العربية

selected

<

الرواتب
قريبًا

disabled

الإصدار
0.0.1

static (no onPress)

Arabic (ar-SA) · RTL

39 List

Generic list/grid that renders any component once per data item — rows of `ListItem`, a wrap of `Chip`s, a grid of `Card`s, a row of `Button`s, or your own component.

```
import { List } from '@bunyan/design-system';
```

Variants & states	Column (default), row, wrapping row, horizontal scroll, n-column grid; with/without dividers; inside a <code>Card</code> ; loading skeletons; empty state.
Accessibility	Items container uses the list role and each cell the listitem role; loading is one progressbar announcement for the whole region; item accessibility comes from the rendered component.
Example	<pre><List Component={ListItem} data={users} formatItem={u => ({ title: u.name, description: u.email })} shareProps={{ onPress: open }} withDivider cardVariant="outlined" />.</pre>
Do/don't	Map API objects in <code>formatItem</code> and put common props in <code>shareProps</code> ; do not use it for thousands of rows (it renders every item — use a virtualized list for very long data).
Edge cases	Item props override <code>shareProps</code> ; <code>formatItem</code> is required by TypeScript when <code>data</code> does not match the component's props; returning <code>null</code> from <code>formatItem</code> skips the item (with <code>filterNull</code> , default on); keys default to <code>id</code> , then <code>key</code> , then index; the last grid row is padded so cells keep equal widths; spacing accepts points or token names (<code>'md'</code>); requires TypeScript ≥ 5.4 (<code>NoInfer</code>).

ListProps

PROP	TYPE	DEFAULT
<code>Component</code> required	<code>ListItemComponent<T></code> Component rendered once per item, e.g. <code>Button</code> , <code>Card</code> , <code>ListItem</code> , <code>Chip</code> , or your own.	—
<code>data</code> required	<code>readonly (D null undefined)[] null undefined</code> Items to render; one <code>Component</code> per item.	—
<code>formatItem</code>	<code>(item: D, index: number) => NoInfer<T> Partial<NoInfer<T>> null undefined</code> Map a raw item to <code>Component</code> props. Return <code>null</code> to skip it (with <code>filterNull</code>). Map a raw item to <code>Component</code> props. Required because <code>data</code> does not match <code>Component</code> 's props.	—
<code>shareProps</code>	<code>Partial<NoInfer<T>></code> Props shared by every item. Item props win on conflicts.	—
<code>keyExtractor</code>	<code>(item: D, index: number) => React.Key</code> Stable React key per raw item. Defaults to <code>id</code> , then <code>key</code> (of the raw item, then of the formatted props), then the index.	—
<code>style</code>	<code>StyleProp<ViewStyle></code> Root container style (outside the card / scroll view).	—
<code>paddingStyle</code>	<code>StyleProp<ViewStyle></code> Padding applied inside the root (inside the card when <code>cardVariant</code> is set).	—
<code>rowStyle</code>	<code>StyleProp<ViewStyle></code> Style of every row (one row per item, or per <code>columns</code> items in a grid).	—

PROP	TYPE	DEFAULT
<code>cellStyle</code>	<code>StyleProp<ViewStyle></code> Style of every cell wrapping a rendered item.	—
<code>scrollViewContentContainerStyle</code>	<code>StyleProp<ViewStyle></code> Content container style when <code>isScrolling</code> .	—
<code>flexDirection</code>	<code>'row' 'column' 'row-reverse' 'column-reverse'</code> Main axis of the list. <code>row</code> + <code>isScrolling</code> gives a horizontal carousel. Default <code>'column'</code> . RTL-aware.	<code>'column'</code>
<code>flexWrap</code>	<code>'wrap' 'nowrap' 'wrap-reverse'</code> Wrapping of items along the main axis (e.g. a cloud of Chips). Default <code>'nowrap'</code> .	<code>'nowrap'</code>
<code>columns</code>	<code>Responsive<keyof Theme['breakpoint'], number></code> Grid columns. Items are placed in equal-width cells, <code>columns</code> per row. Default <code>1</code> . Responsive: <code>columns={{ compact: 1, medium: 2, expanded: 3 }}</code> .	<code>1</code>
<code>spacing</code>	<code>Responsive<keyof Theme['breakpoint'], number keyof Theme['spacing']></code> Gap between items/rows: points, a spacing token (<code>'md'</code>), or per breakpoint. Default <code>'md'</code> .	—
<code>withDivider</code>	<code>boolean</code> Draw a Divider between rows.	<code>false</code>
<code>dividerSpacing</code>	<code>Responsive<keyof Theme['breakpoint'], number keyof Theme['spacing']></code> Gap on each side of a divider. Default: same as <code>spacing</code> .	—
<code>dividerStyle</code>	<code>StyleProp<ViewStyle></code>	—
<code>cardVariant</code>	<code>'elevated' 'outlined' 'filled'</code> Wrap the whole list in a Card of this variant.	—
<code>isScrolling</code>	<code>boolean</code> Render inside a ScrollView (horizontal when <code>flexDirection</code> is a row).	<code>false</code>
<code>loading</code>	<code>boolean</code> Show Skeleton placeholders instead of data.	<code>false</code>
<code>loadingConfig</code>	<code>ListLoadingConfig</code>	—
<code>emptyForm</code>	<code>EmptyStateProps</code> EmptyState shown when there is nothing to render (after filtering) and not loading.	—
<code>filterNull</code>	<code>boolean</code> Drop <code>null</code> / <code>undefined</code> entries in <code>data</code> and <code>null</code> / <code>undefined</code> results of <code>formatItem</code> . Default <code>true</code> .	<code>true</code>
<code>ListHeaderComponent</code>	<code>ListSlot</code>	—
<code>ListFooterComponent</code>	<code>ListSlot</code>	—
<code>accessibilityLabel</code>	<code>string</code>	—
<code>testID</code>	<code>string</code>	—

ListLoadingConfig

PROP	TYPE	DEFAULT
count	number Number of placeholder items. Default 3.	—
renderItem	(index: number) => React.ReactNode Custom placeholder for one item (replaces the Skeleton).	—
accessibilityLabel	string Announced once for the whole loading region. Default 'Loading content'.	—
width	ViewStyle['width']	—
height	number	—
radius	'small' 'medium' 'large' 'pill'	—
lines	number	—

Also accepts: `Omit<SkeletonProps, 'accessibilityLabel'>` (native React Native props).

Component={ListItem} + formatItem + shareProps + withDivider + cardVariant="outlined" + header

Team members

- FA

Faisal Alhejaili

Owner

>
- SA

Sara Alqahtani

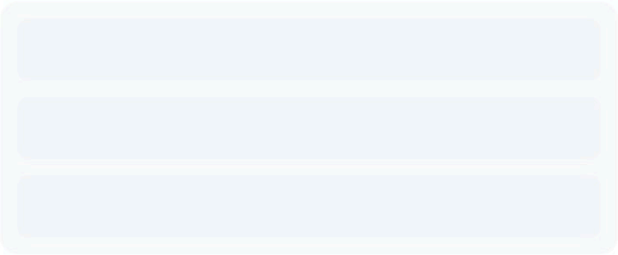
Admin

>
- OA

Omar Alharbi

Viewer

>



loading + loadingConfig={ count: 3, height: 44 }

Component={Card} columns={3} (last row padded)

Revenue
SAR 12,480

Expenses
SAR 12,480

Profit
SAR 12,480

Invoices
SAR 12,480

Component={Chip} flexDirection="row" flexWrap="wrap" spacing="sm"

- Finance
- HR
- Legal
- Sales
- Operations
- IT
- Approve
- Reject
- Details

Component={Button} flexDirection="row"

flexDirection="row" withDivider (vertical dividers) · emptyForm

Overview | Activity | Settings

flexDirection="row" withDivider



No members yet

Invite your team to get started.

data={[]} emptyForm

All variants — light theme

Component={ListItem} + formatItem + shareProps + withDivider + cardVariant="outlined" + header

Team members

- FA

Faisal Alhejaili
Owner

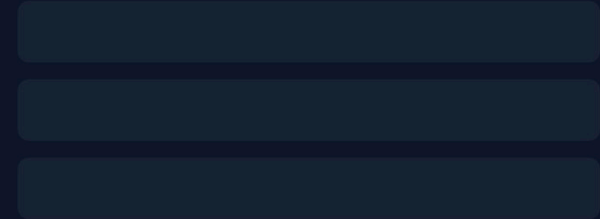
>
- SA

Sara Alqahtani
Admin

>
- OA

Omar Alharbi
Viewer

>



loading + loadingConfig={ count: 3, height: 44 }

Component={Card} columns={3} (last row padded)

Revenue

SAR 12,480

Expenses

SAR 12,480

Profit

SAR 12,480

Invoices

SAR 12,480

Component={Chip} flexDirection="row" flexWrap="wrap" spacing="sm"

- Finance
- HR
- Legal
- Sales
- Operations
- IT
- Approve
- Reject
- Details

Component={Button} flexDirection="row"

flexDirection="row" withDivider (vertical dividers) · emptyForm

Overview | Activity | Settings

flexDirection="row" withDivider

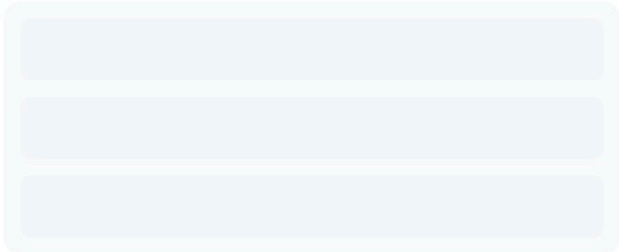
No members yet

Invite your team to get started.

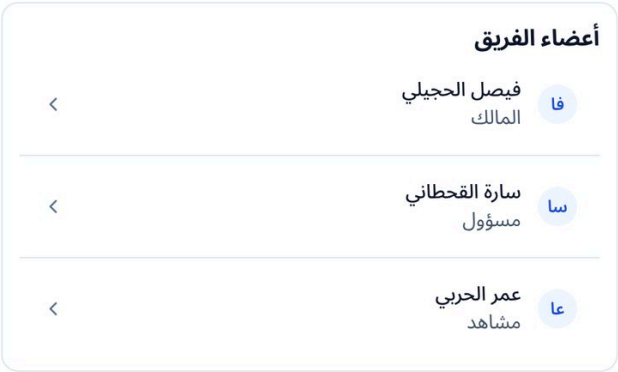
data={[]} emptyForm

Dark theme

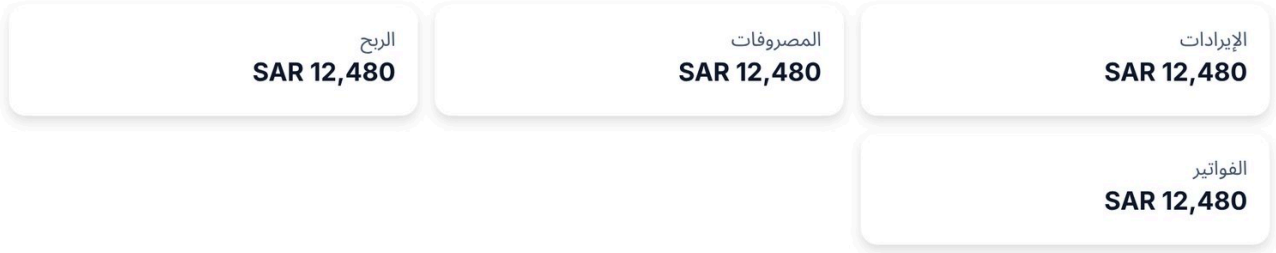
Component={ListItem} + formatItem + shareProps + withDivider + cardVariant="outlined" + header



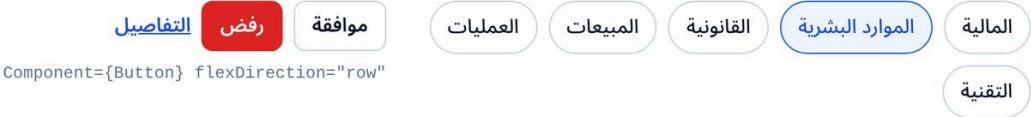
loading + loadingConfig={ count: 3, height: 44 }



Component={Card} columns={3} (last row padded)



Component={Chip} flexDirection="row" flexWrap="wrap" spacing="sm"



Component={Button} flexDirection="row"

flexDirection="row" withDivider (vertical dividers) · emptyForm



data={[]} emptyForm

نظرة عامة | النشاط | الإعدادات
flexDirection="row" withDivider

Feedback & status

40 Alert

Persistent contextual feedback inside a screen.

```
import { Alert } from '@bunyan/design-system';
```

- Variants & states** Information, success, warning, error; actionable, dismissible.
- Accessibility** Warning/error use alert semantics; icon and color are redundant cues.
- Example** `<Alert tone="warning" title="Session expires soon" />.`
- Do/don't** Place near affected content; do not show multiple competing alerts.
- Edge cases** Long actions should move below the description naturally.

AlertProps

PROP	TYPE	DEFAULT
<code>title</code> required	<code>string</code>	—
<code>description</code>	<code>string</code>	—
<code>tone</code>	<code>'information' 'success' 'warning' 'error'</code>	<code>'information'</code>
<code>actionLabel</code>	<code>string</code>	—
<code>onAction</code>	<code>() => void</code>	—
<code>onDismiss</code>	<code>() => void</code>	—



Information message

Your changes were processed.

tone="information"



Success message

Your changes were processed.

tone="success"



Warning message

Your changes were processed.

tone="warning"



Error message

Your changes were processed.

tone="error"



Verify your email

Some features are limited.

[Resend link](#)



action + dismiss

All variants — light theme



Information message

Your changes were processed.

tone="information"



Success message

Your changes were processed.

tone="success"



Warning message

Your changes were processed.

tone="warning"



Error message

Your changes were processed.

tone="error"



Verify your email

Some features are limited.

[Resend link](#)



action + dismiss

Dark theme

عنوان التنبيه ⓘ
تمت معالجة التغييرات.

tone="information"

عنوان التنبيه ✓
تمت معالجة التغييرات.

tone="success"

عنوان التنبيه ⚠
تمت معالجة التغييرات.

tone="warning"

عنوان التنبيه ✖
تمت معالجة التغييرات.

tone="error"

×
تحقق من بريدك ⚠
بعض الميزات محدودة.
إعادة الإرسال

action + dismiss

Arabic (ar-SA) · RTL

41 Toast

Announces transient, non-blocking feedback.

```
import { ToastProvider, useToast } from '@bunyan/design-system';
```

Variants & states	Neutral, success, warning, error, information; timed or persistent.
Accessibility	Polite live region; errors use alert semantics.
Example	<code>useToast().showToast({ message: 'Saved', tone: 'success' })</code> .
Do/don't	Use concise status messages; do not use a toast for required decisions.
Edge cases	<code>duration: 0</code> persists until dismissed; the queue is capped at <code>maxVisible</code> (oldest evicted); pressing an action also dismisses; <code>dismissAll()</code> clears everything.

ToastProviderProps

PROP	TYPE	DEFAULT
<code>children</code> required	<code>React.ReactNode</code>	—
<code>maxVisible</code>	<code>number</code>	<code>3</code>
<code>defaultDuration</code>	<code>number</code>	<code>5000</code>
<code>dismissLabel</code>	<code>string</code>	<code>'Dismiss notification'</code>

ToastOptions

PROP	TYPE	DEFAULT
<code>message</code> required	<code>string</code>	—
<code>tone</code>	<code>'neutral' 'success' 'warning' 'error' 'information'</code>	—
<code>duration</code>	<code>number</code> Milliseconds before auto-dismiss. <code>0</code> keeps the toast until dismissed.	—
<code>actionLabel</code>	<code>string</code>	—
<code>onAction</code>	<code>() => void</code>	—

ToastContextValue

PROP	TYPE	DEFAULT
<code>showToast</code> required	<code>(options: ToastOptions) => number</code>	—
<code>dismissToast</code> required	<code>(id: number) => void</code>	—
<code>dismissAll</code> required	<code>() => void</code>	—

success · error with action · information

✔ Profile saved ×

⊗ Connection lost **Retry** ×

ℹ New version available ×

Light

✔ Profile saved ×

⊗ Connection lost **Retry** ×

ℹ New version available ×

Dark

× تم حفظ الملف ✔

× إعادة انقطع الاتصال ⊗

× يتوفر إصدار جديد ⓘ

Arabic · RTL

42 ProgressBar

Determinate progress (uploads, completion, steps).

```
import { ProgressBar } from '@bunyan/design-system';
```

Variants & states Primary, success, warning, error; three sizes.

Accessibility Progressbar role with min/max/now values.

Example `<ProgressBar label="Profile" value={3} total={5} />.`

Do/don't Use Spinner/Skeleton for unknown durations.

Edge cases Value is clamped to 0–100%.

ProgressBarProps

PROP	TYPE	DEFAULT
<code>value</code> required	<code>number</code> 0 – 1, or <code>current</code> / <code>total</code> when <code>total</code> is given.	—
<code>total</code>	<code>number</code>	—
<code>label</code>	<code>TextValue</code>	—
<code>showValue</code>	<code>boolean</code> Shows "40%" (or "2 / 5" with <code>total</code>) after the label.	<code>true</code>
<code>formatValue</code>	<code>(ratio: number, value: number, total?: number) => string</code>	—
<code>tone</code>	<code>'primary' 'success' 'warning' 'error'</code>	<code>'primary'</code>
<code>size</code>	<code>'small' 'medium' 'large'</code>	<code>'medium'</code>
<code>testID</code>	<code>string</code>	—

Upload 40% Profile 4 / 5 Limit used 85%

`value={0.4}` `total={5} tone="success"` `size="small" tone="warning"`

All variants — light theme

Upload 40% Profile 4 / 5 Limit used 85%

`value={0.4}` `total={5} tone="success"` `size="small" tone="warning"`

Dark theme

85% الحد المستخدم 5 / 4 الملف 40% الرفع

`size="small" tone="warning"` `total={5} tone="success"` `value={0.4}`

Arabic (ar-SA) · RTL

43 Spinner

Indicates indeterminate activity.

```
import { Spinner } from '@bunyan/design-system';
```

- Variants & states**
- Small/large; primary/inverse.
- Accessibility**
- Progress role and text label.
- Example**
- `<Spinner label="Loading accounts" />.`
- Do/don't**
- Use a task-specific label; do not cover an entire screen for background work.
- Edge cases**
- Use Skeleton for content-shaped loading.

SpinnerProps

PROP	TYPE	DEFAULT
size	'small' 'large'	'small'
label	string	'Loading'
tone	'primary' 'inverse'	'primary'



small

Loading



large

Loading



label=""



tone=inverse

Loading

All variants — light theme



small

Loading



large

Loading



label=""



tone=inverse

Loading

Dark theme



tone=inverse

Loading



label=""



large

جارٍ التحميل



small

Loading

Arabic (ar-SA) · RTL

44 Skeleton

Preserves layout while content is loading.

```
import { Skeleton } from '@bunyan/design-system';
```

- Variants & states**
- Single/multiple lines and four radii.
- Accessibility**
- Exposes progress semantics and respects reduced motion.
- Example**
- `<Skeleton lines={3} />.`
- Do/don't**
- Match the final layout; do not use for actions waiting on submission.
- Edge cases**
- The final line is shortened to resemble natural text.

SkeletonProps

PROP	TYPE	DEFAULT
<code>width</code>	<code>ViewStyle['width']</code>	<code>'100%'</code>
<code>height</code>	<code>number</code>	<code>—</code>
<code>radius</code>	<code>'small' 'medium' 'large' 'pill'</code>	<code>'medium'</code>
<code>lines</code>	<code>number</code>	<code>1</code>
<code>accessibilityLabel</code>	<code>string</code>	<code>'Loading content'</code>

lines=1

lines=3

radius=pill, height=40

radius=large, 64×64

All variants — light theme

lines=1

lines=3

radius=pill, height=40

radius=large, 64×64

Dark theme

radius=large, 64×64

radius=pill, height=40

lines=3

lines=1

Arabic (ar-SA) · RTL

45 EmptyState

Explains an empty collection and offers a next step.

```
import { EmptyState } from '@bunyan/design-system';
```

- Variants & states**
- Informational and actionable.
- Accessibility**
- Clear heading hierarchy and real button actions.
- Example**
- `<EmptyState title="No invoices" actionLabel="Create invoice" onAction={create} />.`
- Do/don't**
- Explain why it is empty; do not imply an error when emptiness is valid.
- Edge cases**
- Omit actions when the user cannot change the state.

EmptyStateProps

PROP	TYPE	DEFAULT
<code>title</code> required	<code>string</code>	—
<code>description</code>	<code>string</code>	—
<code>icon</code>	<code>IconName</code>	<code>'info'</code>
<code>actionLabel</code>	<code>string</code>	—
<code>onAction</code>	<code>() => void</code>	—
<code>secondaryActionLabel</code>	<code>string</code>	—
<code>onSecondaryAction</code>	<code>() => void</code>	—



No results

Try a different search term.

Clear filters

Learn more



Something went wrong

Check your connection.

Try again

ErrorState

EmptyState

All variants — light theme



No results

Try a different search term.

Clear filters

Learn more

EmptyState



Something went wrong

Check your connection.

Try again

ErrorState

Dark theme



حدث خطأ ما

تحقق من اتصالك.

حاول مجددًا

ErrorState



لا توجد نتائج

جرب كلمة بحث أخرى.

مسح الفلاتر

اعرف المزيد

EmptyState

Arabic (ar-SA) · RTL

46 ErrorState

Full-region failure with an optional retry.

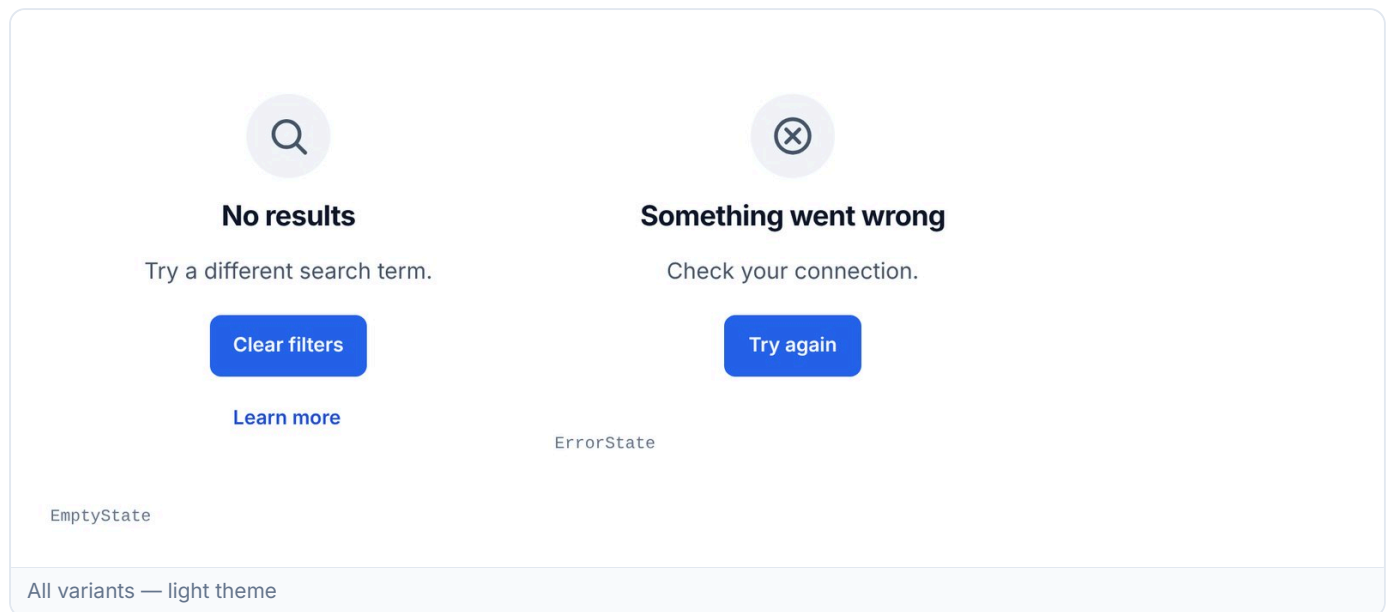
```
import { ErrorState } from '@bunyan/design-system';
```

Variants & states	Static error or retryable error.
Accessibility	Uses visible text and error icon rather than color alone.
Example	<code><ErrorState title="Unable to load" onRetry={reload} /></code> .
Do/don't	Explain recovery; do not expose raw server messages.
Edge cases	Disable or debounce retry in product code while a retry is running.

ErrorStateProps

PROP	TYPE	DEFAULT
<code>title</code> required	<code>string</code>	—
<code>description</code>	<code>string</code>	—
<code>actionLabel</code>	<code>string</code>	—
<code>onAction</code>	<code>() => void</code>	—
<code>secondaryActionLabel</code>	<code>string</code>	—
<code>onSecondaryAction</code>	<code>() => void</code>	—
<code>retryLabel</code>	<code>string</code>	<code>'Try again'</code>
<code>onRetry</code>	<code>() => void</code>	—

Also accepts: `Omit<EmptyStateProps>` (native React Native props).





No results

Try a different search term.

Clear filters

Learn more

EmptyState



Something went wrong

Check your connection.

Try again

ErrorState

Dark theme



حدث خطأ ما

تحقق من اتصالك.

حاول مجددًا

ErrorState



لا توجد نتائج

جرب كلمة بحث أخرى.

مسح الفلاتر

اعرف المزيد

EmptyState

Arabic (ar-SA) · RTL

Disclosure & navigation

47 Accordion

Progressively discloses secondary content.

```
import { Accordion } from '@bunyan/design-system';
```

- Variants & states** Collapsed, expanded, disabled.
- Accessibility** Announces expanded state and connects trigger to content.
- Example** `<Accordion title="Advanced"><AdvancedSettings /></Accordion>`.
- Do/don't** Use for optional detail; do not hide required form fields unexpectedly.
- Edge cases** Controlled mode requires the parent to update `expanded`.

AccordionProps

PROP	TYPE	DEFAULT
<code>title</code> required	<code>string</code>	—
<code>children</code> required	<code>React.ReactNode</code>	—
<code>expanded</code>	<code>boolean</code>	—
<code>defaultExpanded</code>	<code>boolean</code>	<code>false</code>
<code>onExpandedChange</code>	<code>(expanded: boolean) => void</code>	—
<code>disabled</code>	<code>boolean</code>	<code>false</code>

Account details

collapsed

Billing address

King Fahd Rd, Riyadh 12271

Security

disabled

All variants — light theme

Account details

collapsed

Billing address

King Fahd Rd, Riyadh 12271

defaultExpanded

Security

disabled

Dark theme

تفاصيل الحساب

collapsed

عنوان الفوترة

طريق الملك فهد، الرياض 12271

defaultExpanded

الأمان

disabled

Arabic (ar-SA) · RTL

48 Tabs

Switches between peer content views.

```
import { Tabs } from '@bunyan/design-system';
```

- Variants & states**
- Line/pill; selected, pressed, disabled; optional badge.
- Accessibility**
- Tablist and tab roles with selected state.
- Example**
- `<Tabs items={items} value={tab} onValueChange={setTab} /> .`
- Do/don't**
- Keep labels short; do not use tabs as a stepper.
- Edge cases**
- Horizontal scrolling handles long translations; re-pressing the active tab does not fire `onValueChange` .

TabsProps

PROP	TYPE	DEFAULT
<code>items</code> required	<code>readonly TabItem[]</code>	—
<code>value</code> required	<code>string</code>	—
<code>onValueChange</code> required	<code>(value: string) => void</code>	—
<code>variant</code>	<code>'line' 'pill'</code>	<code>'line'</code>
<code>accessibilityLabel</code>	<code>string</code>	<code>'Tabs'</code>
<code>testID</code>	<code>string</code>	—

TabItem

PROP	TYPE	DEFAULT
<code>value</code> required	<code>string</code>	—
<code>label</code> required	<code>string</code>	—
<code>disabled</code>	<code>boolean</code>	—
<code>badge</code>	<code>string</code>	—
<code>accessibilityLabel</code>	<code>string</code>	—

variant="line"

Overview

Activity 12

Settings

Archive

variant="pill"

Overview

Activity 12

Settings

Archive

All variants — light theme

variant="line"

Overview

Activity 12

Settings

Archive

variant="pill"

Overview

Activity 12

Settings

Archive

Dark theme

variant="line"

الأرشيف

الإعدادات

النشاط 12

نظرة عامة

variant="pill"

الأرشيف

الإعدادات

النشاط 12

نظرة عامة

Arabic (ar-SA) · RTL

49 Tooltip

Supplies supplemental context for a focused or hovered control.

```
import { Tooltip } from '@bunyan/design-system';
```

Variants & states Top/bottom; hover, focus, long-press visibility.

Accessibility Content is also provided as an accessibility hint.

Example `<Tooltip content="Archive record"><IconButton ... /></Tooltip>`.

Do/don't Keep it brief; do not put essential instructions only in a tooltip.

Edge cases Touch users reveal tooltips with long press; the bubble stays for `touchDuration` ms after release.

TooltipProps

PROP	TYPE	DEFAULT
<code>content</code> required	<code>string</code>	—
<code>children</code> required	<code>React.ReactElement</code>	—
<code>placement</code>	<code>'top' 'bottom'</code>	<code>'top'</code>
<code>touchDuration</code>	<code>number</code>	<code>1500</code>

How long a long-press tooltip stays visible on touch devices (ms).

Copy account number


placement="top" (keyboard focus)

Hover me

placement="bottom"

Copy account number


placement="top" (keyboard focus)

Hover me

placement="bottom"

مرّر هنا

placement="bottom"

نسخ رقم الحساب


placement="top" (keyboard focus)

Overlays

50 Modal

Interruptive, focused task or confirmation.

```
import { Modal } from '@bunyan/design-system';
```

- Variants & states**
- Small/medium/large; open/closed; dismissible; action loading/disabled.
- Accessibility**
- Isolates modal content and supports platform back dismissal.
- Example**
- `<Modal visible={open} onClose={close} title="Confirm">...</Modal>`.
- Do/don't**
- Keep tasks short; do not stack modals.
- Edge cases**
- Non-dismissible modals must provide a clear completion action.

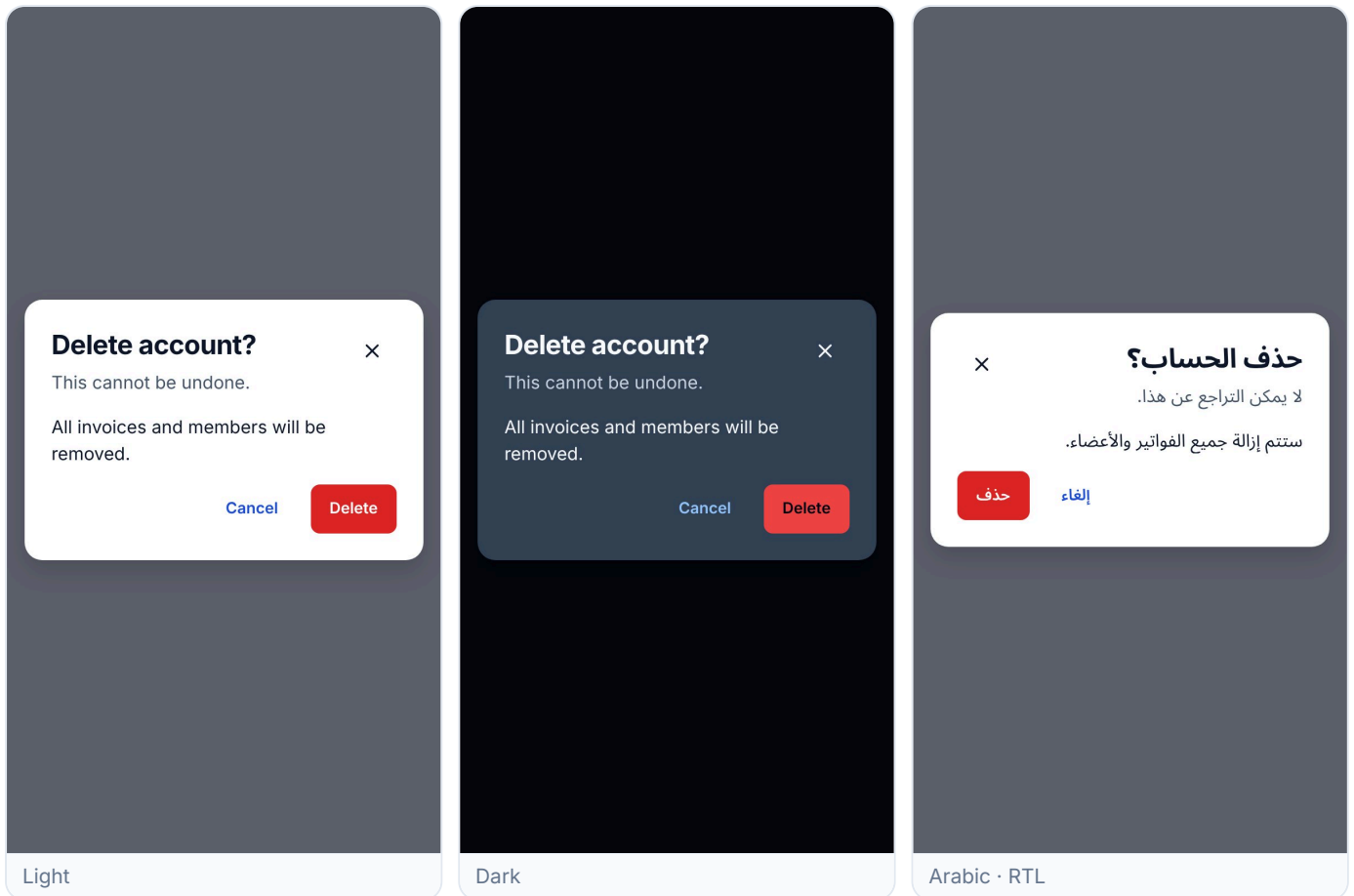
ModalProps

PROP	TYPE	DEFAULT
<code>visible</code> required	boolean	—
<code>onClose</code> required	() => void	—
<code>title</code> required	string	—
<code>description</code>	string	—
<code>children</code> required	React.ReactNode	—
<code>primaryAction</code>	ModalAction	—
<code>secondaryAction</code>	ModalAction	—
<code>dismissible</code>	boolean	true
<code>size</code>	'small' 'medium' 'large'	'medium'
<code>closeLabel</code>	string	'Close modal'
<code>testID</code>	string	—

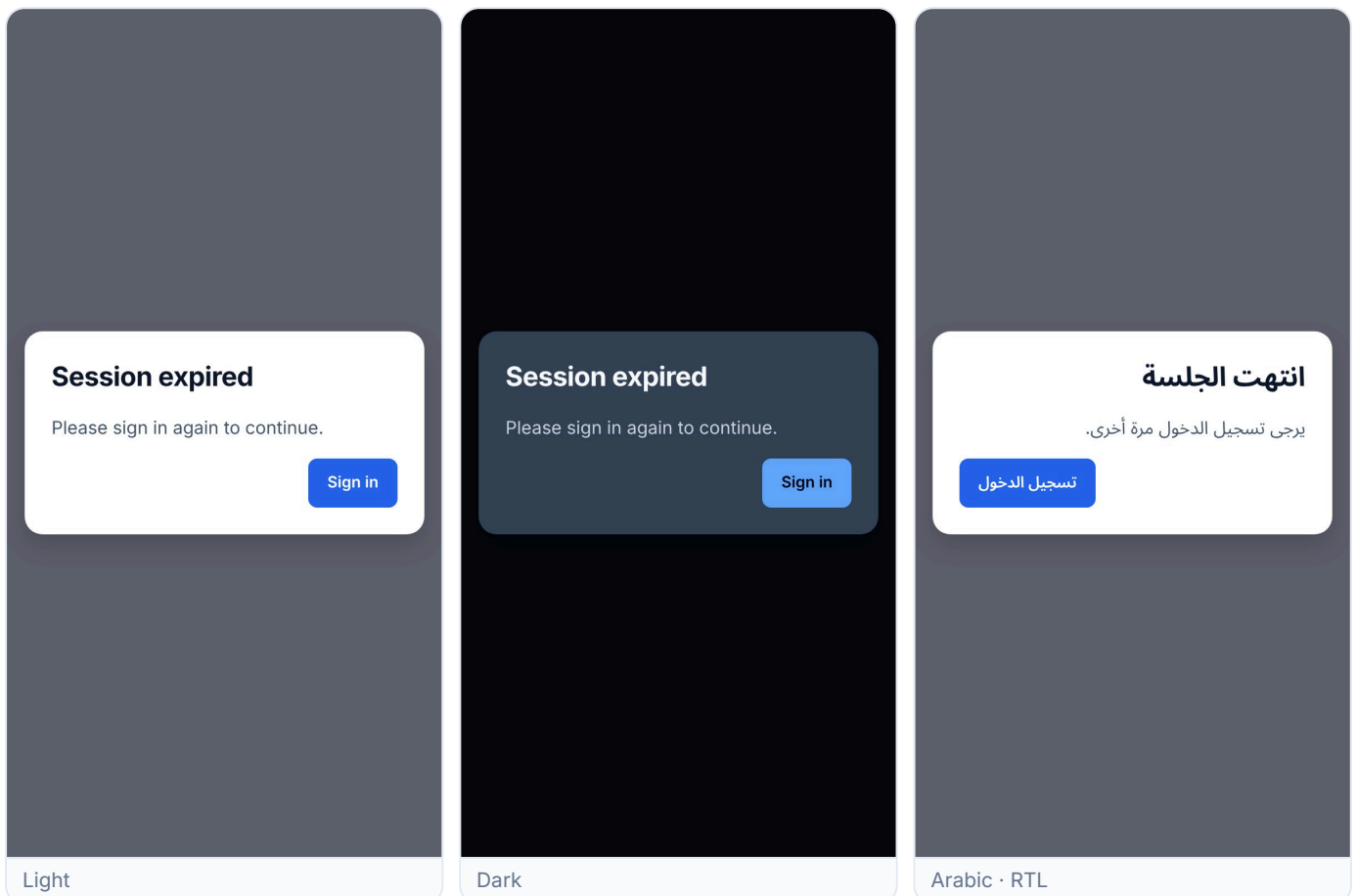
ModalAction

PROP	TYPE	DEFAULT
<code>label</code> required	string	—
<code>onPress</code> required	() => void	—
<code>variant</code>	'primary' 'secondary' 'outline' 'ghost' 'danger' 'link'	—
<code>disabled</code>	boolean	—
<code>loading</code>	boolean	—

Dismissible, with actions



dismissible={false}, size="small"



51 BottomSheet

Mobile-friendly contextual actions or compact tasks.

```
import { BottomSheet } from '@bunyan/design-system';
```

- Variants & states**
- Open/closed and dismissible/non-dismissible.
- Accessibility**
- Modal isolation and labelled close control.
- Example**
- `<BottomSheet visible={open} onClose={close} title="Actions">...</BottomSheet>`.
- Do/don't**
- Use for short choices; use Modal for complex keyboard-heavy tasks.
- Edge cases**
- Content scrolls when it exceeds 90% of viewport height.

BottomSheetProps

PROP	TYPE	DEFAULT
<code>visible</code> required	<code>boolean</code>	—
<code>onClose</code> required	<code>() => void</code>	—
<code>title</code> required	<code>string</code>	—
<code>description</code>	<code>string</code>	—
<code>children</code> required	<code>React.ReactNode</code>	—
<code>dismissible</code>	<code>boolean</code>	<code>true</code>
<code>closeLabel</code>	<code>string</code>	<code>'Close bottom sheet'</code>
<code>testID</code>	<code>string</code>	—

With list content

Share report

Choose a destination

Email>

Copy link>

Download PDF>

Light

Share report

Choose a destination

Email>

Copy link>

Download PDF>

Dark

مشاركة التقرير

اختر الوجهة

<البريد الإلكتروني

<نسخ الرابط

<تنزيل PDF

Arabic · RTL

Templates

52 NumPad

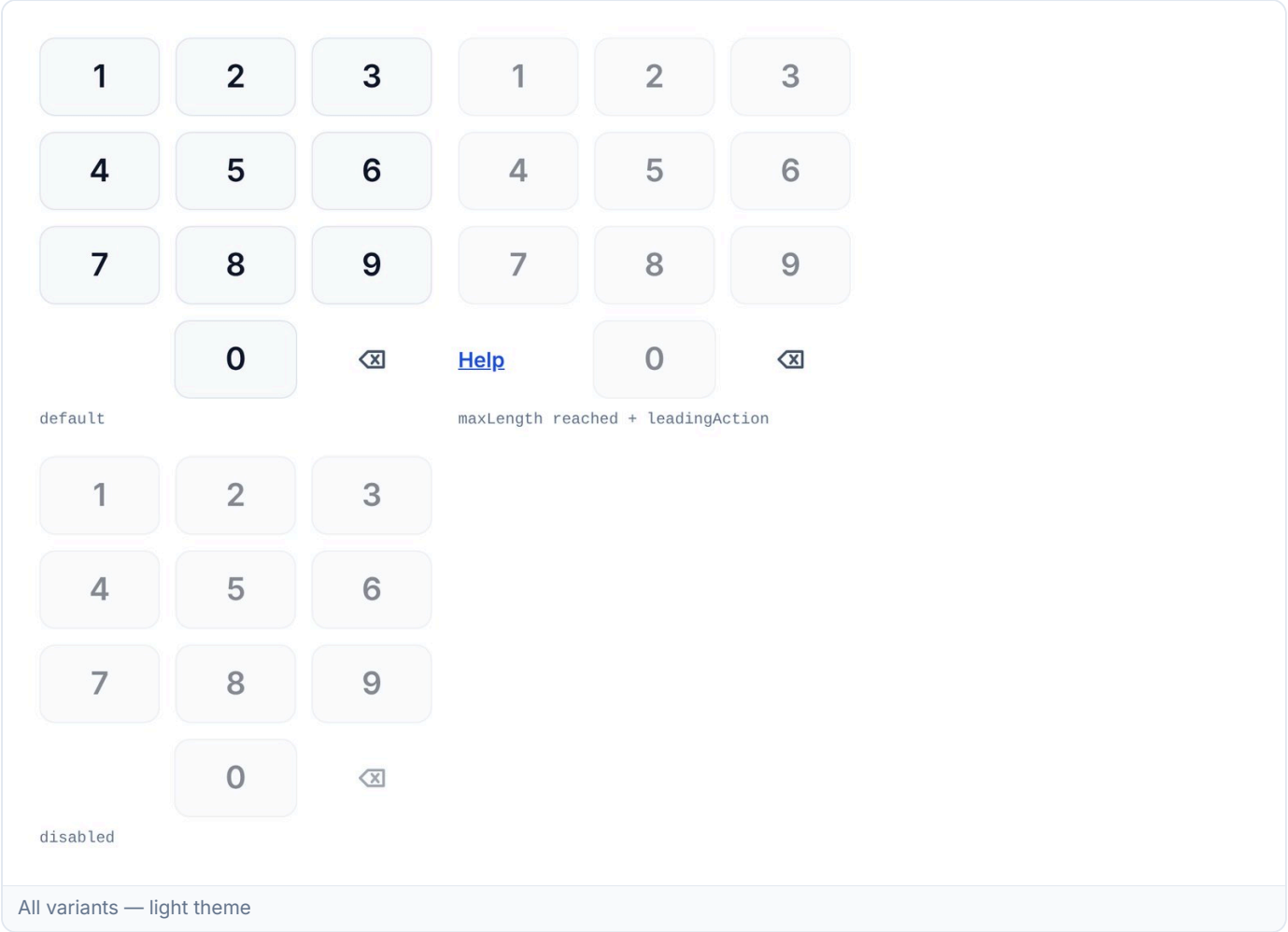
Reusable numeric entry surface for OTP, PIN, amount, and authentication flows.

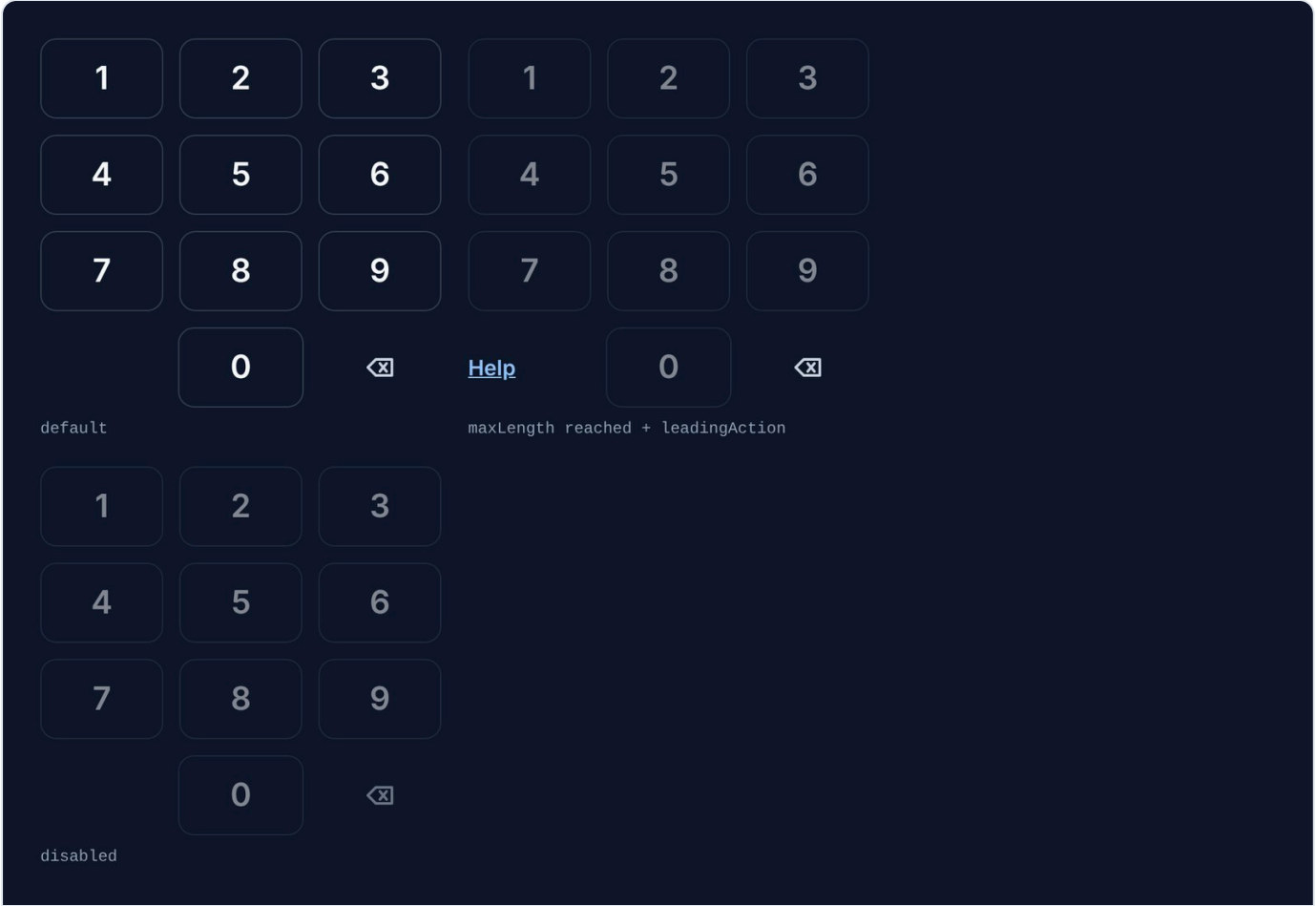
```
import { NumPad } from '@bunyan/design-system';
```

- Variants & states**
- Standard 3×4 keypad; empty, partially entered, length-limited, and disabled states.
- Accessibility**
- Every number and delete action is an individually labelled button. Numeric order stays conventional in both LTR and RTL locales.
- Example**
- `<NumPad value={pin} onChange={setPin} maxLength={4} />.`
- Do/don't**
- Keep the entered value visible elsewhere in the flow; do not use the keypad as the only accessible representation of the current value.
- Edge cases**
- `onDigitPress` and `onDelete` support reducers or secure state stores; `onChange` offers the convenient controlled-value API.

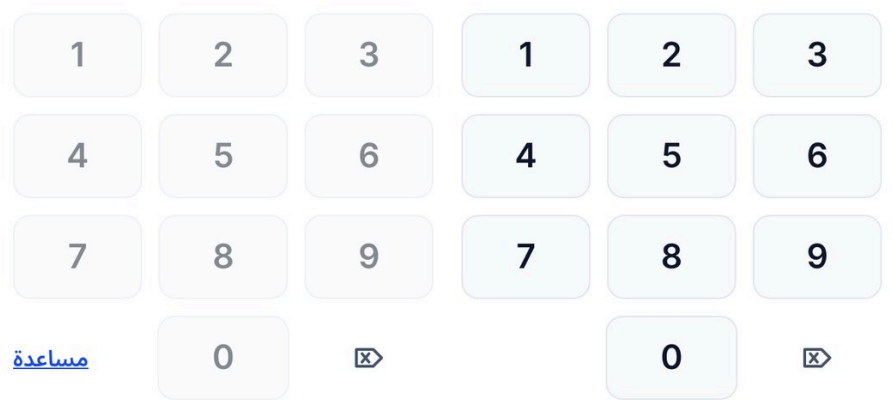
NumPadProps

PROP	TYPE	DEFAULT
<code>value</code>	<code>string</code>	<code>''</code>
<code>onChange</code>	<code>(value: string) => void</code>	—
<code>onDigitPress</code>	<code>(digit: string) => void</code>	—
<code>onDelete</code>	<code>() => void</code>	—
<code>maxLength</code>	<code>number</code>	—
<code>disabled</code>	<code>boolean</code>	<code>false</code>
<code>deleteLabel</code>	<code>string</code>	<code>'Delete last digit'</code>
<code>accessibilityLabel</code>	<code>string</code>	<code>'Numeric keypad'</code>
<code>leadingAction</code>	<code>React.ReactNode</code>	—
<code>testID</code>	<code>string</code>	—



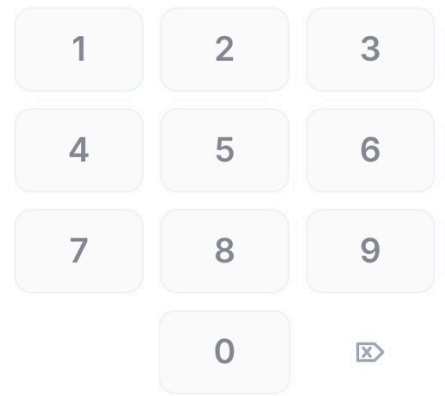


Dark theme



maxLength reached + leadingAction

default



disabled

Arabic (ar-SA) · RTL

53 OTPTemplate

Complete one-time-code verification flow using the same behavior across bottom-sheet, centered-overlay, and full-screen presentations.

```
import { OTPTemplate } from '@bunyan/design-system';
```

Variants & states	<code>bottomSheet</code> , <code>overlay</code> , <code>fullScreen</code> ; four or custom digit length; incomplete, complete, error, loading, secure, and resend-disabled states.
Accessibility	Includes a native one-time-code input for keyboard, paste, and password-manager support; slots announce entered/empty state; NumPad keys and actions are labelled.
Example	<pre><OTPTemplate variant="fullScreen" value={code} onChange={setCode} onSubmit={verify} />.</pre>
Do/don't	Keep verification logic outside the template; do not identify the recipient more than necessary.
Edge cases	Input is digit-filtered and length-limited; <code>autoSubmit</code> fires once per completed value; set <code>useNumPad={false}</code> when the native keyboard is preferred.

OTPTemplateProps

PROP	TYPE	DEFAULT
<code>variant</code> required	<code>'bottomSheet' 'overlay' 'fullScreen'</code>	—
<code>visible</code>	<code>boolean</code>	<code>true</code>
<code>value</code> required	<code>string</code>	—
<code>onChange</code> required	<code>(value: string) => void</code>	—
<code>onSubmit</code> required	<code>(value: string) => void</code>	—
<code>onClose</code>	<code>() => void</code>	—
<code>onResend</code>	<code>() => void</code>	—
<code>length</code>	<code>number</code>	—
<code>title</code>	<code>string</code>	<code>'Enter verification code'</code>
<code>description</code>	<code>string</code>	<code>'Enter the one-time code sent to your registered contact.'</code>
<code>destination</code>	<code>string</code>	—
<code>error</code>	<code>string</code>	—
<code>loading</code>	<code>boolean</code>	—
<code>disabled</code>	<code>boolean</code>	—
<code>secure</code>	<code>boolean</code>	—
<code>autoSubmit</code>	<code>boolean</code>	—
<code>dismissible</code>	<code>boolean</code>	<code>true</code>

PROP	TYPE	DEFAULT
<code>submitLabel</code>	string	—
<code>resendLabel</code>	string	—
<code>resendDisabled</code>	boolean	—
<code>useNumPad</code>	boolean	—
<code>testID</code>	string	—

variant= "fullScreen" + NumPad

×

Enter verification code

We sent a code to +966 5•• ••• 123

1

2

1

2

3

4

5

6

7

8

9

0

⌫

Verify

Resend code

Light

×

Enter verification code

We sent a code to +966 5•• ••• 123

1

2

1

2

3

4

5

6

7

8

9

0

⌫

Verify

Resend code

Dark

×

أدخل رمز التحقق

أرسلنا رمزًا إلى +966 5•• ••• 123

1

2

1

2

3

4

5

6

7

8

9

0

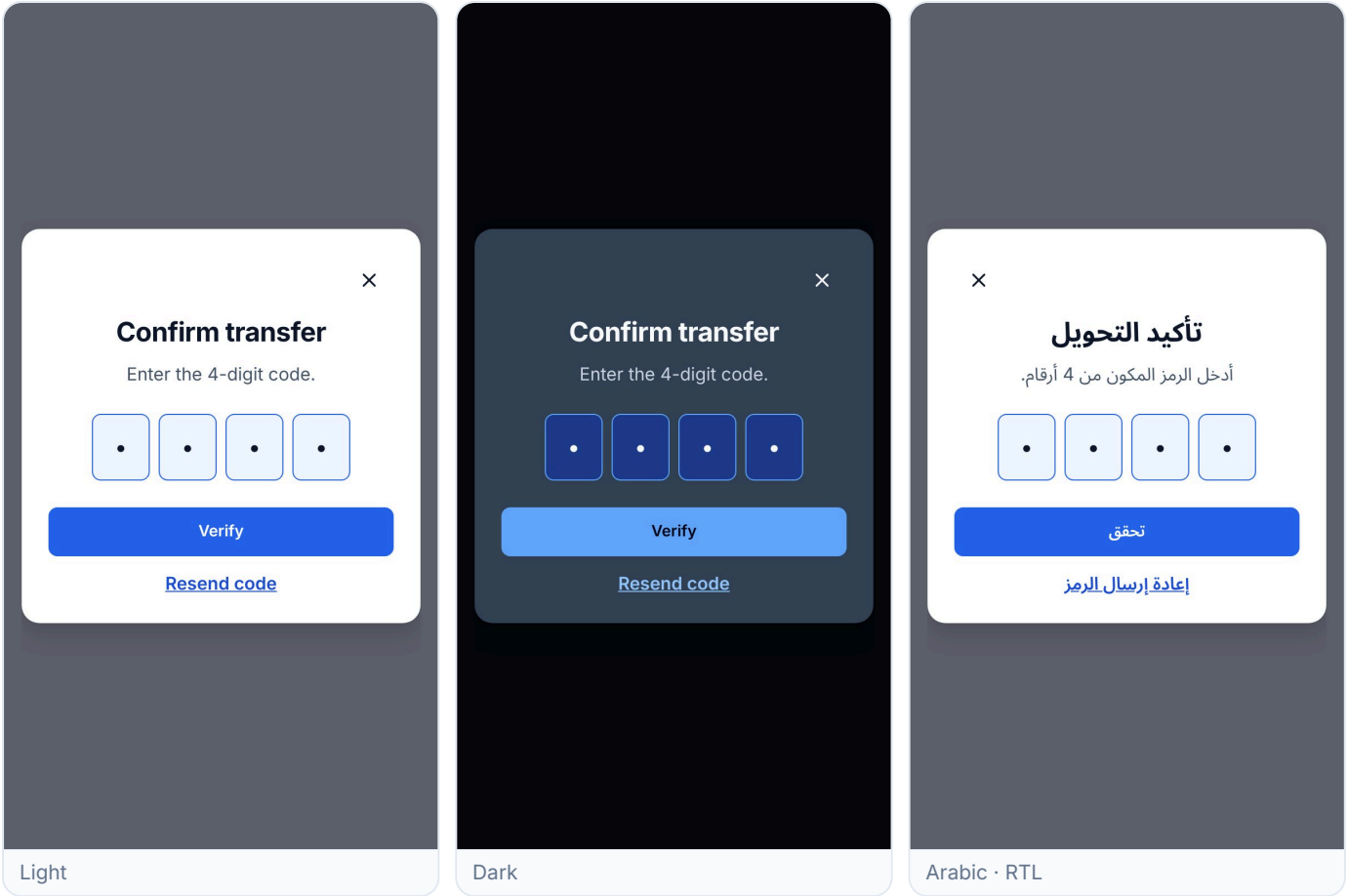
⌫

تحقق

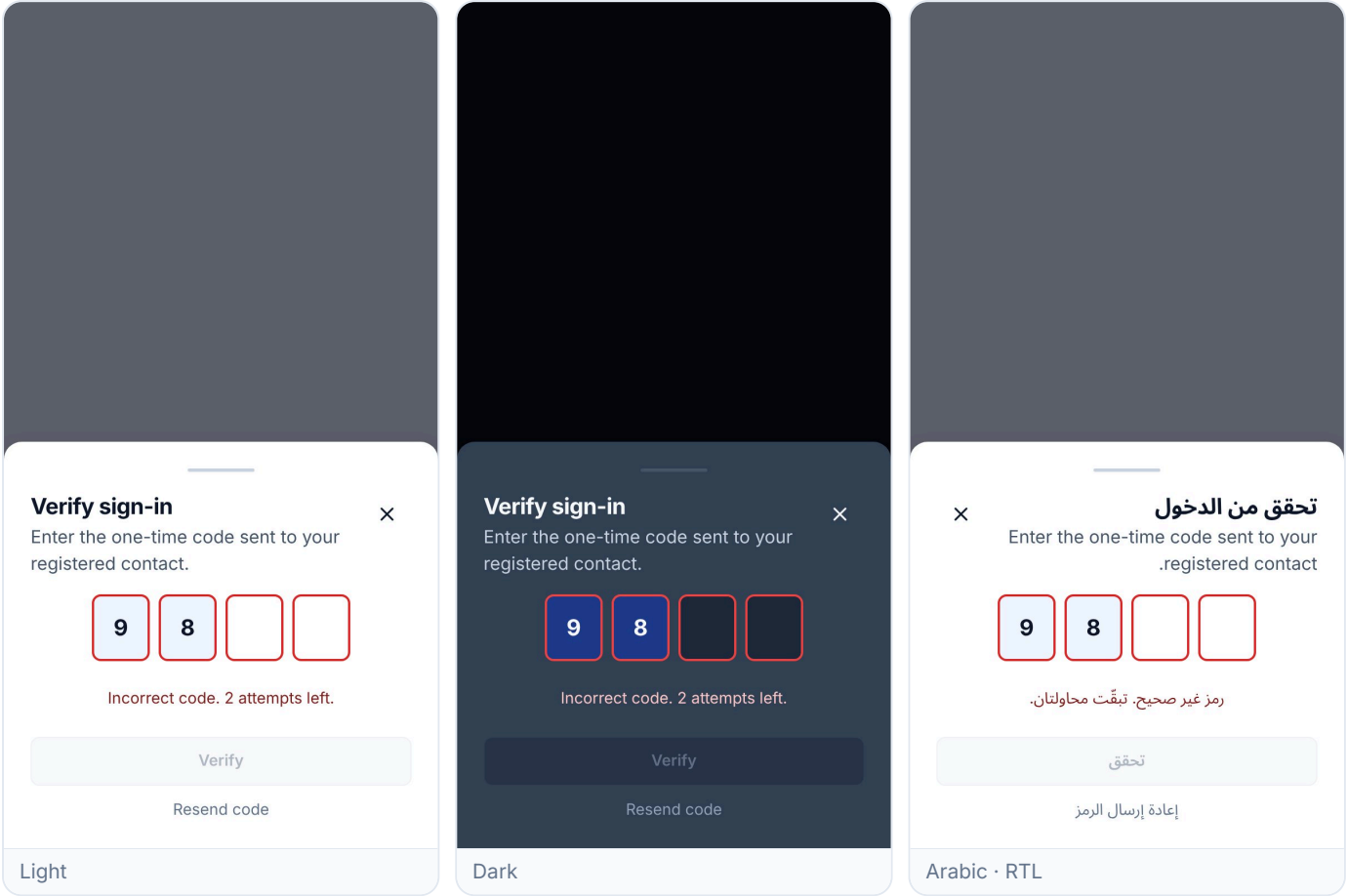
إعادة إرسال الرمز

Arabic · RTL

variant="overlay", secure, complete



variant="bottomSheet", error, resendDisabled



loading

Verifying...

Checking your code

4

8

2

1

Verify

Light

Verifying...

Checking your code

4

8

2

1

Verify

Dark

جارٍ التحقق...

جارٍ التحقق من الرمز

4

8

2

1

تحقق

Arabic · RTL

AI prompts

Paste the **context prompt** first (as a system prompt, project instructions, or the first message), then any task prompt. Replace `{placeholders}`. The same text is in [docs/AI_PROMPTS.md](#) for copy-paste.

0 Context prompt — paste first

You are working with **Bunyan** (`@bunyan/design-system`), a strongly typed React Native design system for iOS, Android and web (React 19, React Native ≥ 0.74 , TypeScript strict with `exactOptionalPropertyTypes`).

Non-negotiable rules

1. Build UI only from Bunyan exports. Import everything from `@bunyan/design-system`.
2. Never import from `react-native` in design-system code. Use `RNTheme` primitives (`RNTheme.View`, `RNTheme.Pressable`, `RNTheme.ScrollView`, `RNTheme.TextInput`, `RNTheme.RNText`, `RNTheme.Platform`, ...). Inside `src/design-system`, only `components/RNTheme/native.ts` may import `react-native`.
3. No raw colours, font sizes, spacing or radii. Read them from `useTheme().theme` (`theme.color.*`, `theme.spacing.*`, `theme.radius.*`, `theme.typography.*`), or use `themeStyle={{ theme }} => ({ ... }}` on `RNTheme` primitives.
4. Wrap the app once in `<ThemeProvider initialPreference="system" locale={locale}>` and, if toasts are used, `<ToastProvider>`.
5. Support light, dark and black themes and Arabic RTL. Use `logicalRow(direction)`, `logicalText(direction, align)` and `logicalAlignItems(...)` instead of hard-coding `row-reverse` or `left`/`right`. Number entry (OTP, NumPad) stays left-to-right.
6. Accessibility: every icon-only control has an `accessibilityLabel`; every input sits inside a `FormField` (or has a label); never use colour alone to convey state; keep touch targets ≥ 44 pt.
7. Use semantic props (`variant`, `tone`, `size`, `status`) instead of style overrides. The only public style escape hatch is `containerStyle` on `Input` (layout only).
8. All user-facing strings must be passable as props so they can be localised (labels like `clearLabel`, `confirmLabel`, `closeLabel`, `dismissLabel`, `retryLabel`).
9. Responsive: keep token sizes fixed and change layout per breakpoint with `useResponsive()` (`r.up('medium')`, `r.select({ compact, medium, expanded })`) or breakpoint maps on `List` (`columns={{ compact: 1, medium: 2 }}`). Use `r.ms()` only for fonts/radii that should grow a little; never scale everything. Use `makeStyles((theme, r) => ({...}))` for styles that depend on theme and size.
10. Brand tokens come from `createThemes({ shared, light, dark })` passed to `<ThemeProvider themes={...}>`; custom breakpoint names / design canvas come from the app's own `createResponsive({...)}` module – import responsive helpers from there if the project has one.

Building blocks

- Providers/hooks: `ThemeProvider`, `useTheme()` $\rightarrow$ `{ theme, mode, preference, setPreference, direction, isRTL, locale }`; `ToastProvider`, `useToast()` $\rightarrow$ `{ showToast, dismissToast, dismissAll }`.
- Typography & icons: `Text` (variant/tone/weight/align), `Heading` (level 1–6), `Icon` (16 names, sizes xs–xxl, tones).
- Actions: `Button` (primary|secondary|outline|ghost|danger|link; small|medium|large; loading; fullWidth; leading/trailing icons; `href`/`external` to navigate), `IconButton` (filled|outline|ghost; required accessibilityLabel). `Link` is an alias of `<Button variant="link">`.
- Forms: `FormField` (label/description/error/success/required; render-prop ids), `Input`, `PasswordInput`, `SearchInput`, `TextArea`, `Checkbox`, `Radio`, `Switch`, `Select` (options, searchable), `DatePicker`.
- Display: `Badge`, `Chip`, `Avatar`, `Divider`, `Card` (elevated|outlined|filled; pressable), `ListItem`.
- Collections: `List<T, D>` renders any component per data item – `Component`, `data`, `formatItem(item, index) => props`, `shareProps` (common props; item props win), `keyExtractor`, `flexDirection`, `flexWrap`, `columns`, `spacing` (points or token), `withDivider`, `cardVariant`, `isScrolling`, `loading` + `loadingConfig`, `emptyForm` (EmptyState props), `ListHeaderComponent`, `ListFooterComponent`. Use it instead of `data.map(...)` in screens.
- Feedback: `Alert`, toast via `useToast`, `Spinner`, `Skeleton`, `EmptyState`, `ErrorState`.
- Disclosure/navigation: `Accordion`, `Tabs` (line|pill), `Tooltip`.
- Overlays: `Modal` (primary/secondary actions), `BottomSheet`.
- Templates: `NumPad`, `OTPTemplate` (bottomSheet|overlay|fullScreen).
- Forms: `<Form initialValues validationSchema onSubmit fields formProps submitButton resetButton>` (Formik + Yup). Field `type`s: `Input/TextInput`, `PasswordInput`, `SearchInput`, `TextArea`, `Select/Picker`, `DatePicker`, `DateRangePicker`, `Checkbox`, `Switch/Toggle`, `RadioGroup`, `ChipsGroup`, `CheckboxGroup`, `RadioImageGroup`, `BoxGroup`, `SwatchGroup`, `Slider`, `AmountInput`, `AmountWithCurrencyInput`, `PhoneWithCountryInput`, `AmountField`, `FileInput`, `OTP`, `RepeatedControls`, `Progress`, `ActionText`, plus React elements and custom types via `registerFormFieldType`. Falsy entries are skipped (`cond && {...}`); `visibleWhen(values)` hides fields; texts accept `{ localeKey }` (`ThemeProvider translate`); Yup

messages may be locale keys; Yup ships inside Bunyan: always `import { Yup } from '@bunyan/design-system'` (never from 'yup'); it includes `Yup.mixed().dateRange(requireFrom, requireTo)`, and `dateRangeSchema()` is available too.

- Theming: `RNTheme.*` themed primitives, `createThemedComponent(Component, { displayName, baseStyle, defaultProps })`.
- Responsive: `useResponsive()` → `{ breakpoint, up, down, between, is, select, resolve, s, vs, ms, mvs, isPortrait, isLandscape }`; `useBreakpoint()`, `useResponsiveValue()`, `makeStyles()`, static `s/vs/ms/mvs`, `getResponsive(size)`, `createResponsive(config)`; own tokens via `createTheme` / `createThemes`.

How to answer

- Output complete, compiling TypeScript (`.tsx`) with typed props and no `any`.
- Keep state in the screen; components are controlled where possible.
- Show loading (`Skeleton`/`Spinner`), empty (`EmptyState`), and error (`ErrorState`) states for async data.
- If something is not possible with the existing components, say so and propose a new component that follows the "add a component" checklist instead of hand-rolling raw React Native.

Building app screens

B1 Build a new screen

Using the Bunyan rules above, build a `{ScreenName}` screen for a React Native app.

Purpose: {what the screen does}

Data: {shape of the data / API}

Actions: {primary action}, {secondary actions}

Requirements:

- Use only `@bunyan/design-system` components and `RNTheme` primitives; no raw colours or spacing.
- Include loading (`Skeleton`), empty (`EmptyState`) and error (`ErrorState` with `onRetry`) states.
- Must look correct in light, dark and black themes and in Arabic (`locale="ar-SA"`).
- Every interactive element has an accessible label; one primary Button per region.

Return the full `.tsx` file plus a short note on which components you used and why.

B2 Build a form with <Form>

Build `{FormName}` with Bunyan's declarative `<Form>`.

Fields: {name: type, required?, rules} ...

Submit: `{submitFn}`; show a success Toast after it resolves and an Alert on failure.

Rules:

- One `<Form>` with `initialValues`, a `validationSchema` built with Bunyan's `Yup` (`import { Yup } from '@bunyan/design-system'`, messages as locale keys) and a `fields` array – no hand-wired inputs.
- Pick field types from the catalogue (`TextInput`, `Picker`, `AmountInput`, `PhoneWithCountryInput`, `DateRangePicker`, `ChipsGroup`, `CheckboxGroup`, `FileInput`, `OTP`, `RepeatedControls...`). Use `cond && {...}` for conditional fields known up front and `visibleWhen` for fields that depend on other values.
- All labels/placeholders as `{ localeKey }`.
- Use `formProps={{ enableReinitialize: true }}` when `initialValues` come from saved state.
- If a field type is missing, write it with `registerFormFieldType` (typed via `FormFieldTypes` augmentation) instead of rendering raw inputs.

Return the screen, the schema and the locale keys you introduced.

B3 Build an OTP / PIN verification flow

Implement sign-in verification using `OTPTemplate``.

Variant: `{fullScreen | overlay | bottomSheet}`

Code length: `{4 | 6}`; delivery: `{SMS to +966...}`

Behaviour: `autoSubmit` when complete; show ``error`` text from the API; resend is disabled for `{30}` seconds with a countdown in ``resendLabel``; ``loading`` while verifying.

Use ``useNumPad`` on `{phones without hardware keyboards}`.

Keep verification logic in a hook (``useVerifyCode``) outside the template. Return the screen + hook.

B4 Render a collection with List

Render `{data description}`` with Bunyan's generic ``List``.

- Pick the item component: ``ListItem`` for rows, ``Card`` for tiles (``columns={2}``), ``Chip`` for tags (``flexDirection="row" flexWrap="wrap"``), ``Button`` for an action bar (``flexDirection="row"``), or a small custom component.
- Map API objects to props in ``formatItem`` (return ``null`` to skip an item) and put props common to all items (handlers, variant, size) in ``shareProps``.
- Use ``keyExtractor`` if items have no ``id``.
- Handle states with ``loading`` + ``loadingConfig`` and ``emptyForm``; add ``withDivider``, ``cardVariant``, ``ListHeaderComponent`` as the design needs.
- Do not write ``data.map(...)`` by hand.

Return the typed component and explain the chosen layout props.

B5 Build a settings screen

Build a Settings screen with Bunyan: sections rendered with ``Heading level={5}`` and ``ListItem`` rows; toggles with ``Switch``; a theme picker using ``Radio`` that calls ``useTheme().setPreference('system' | 'light' | 'dark' | 'black')``; a language row that switches between ``en`` and ``ar-SA``; a destructive "Delete account" row that opens a ``Modal`` with a ``danger`` primary action. Persist choices with `{storage}`. Make it fully RTL-correct.

B6 Build a dashboard / list screen

Build a `{Name}`` dashboard: a ``Tabs`` (line) header, summary ``Card`s` (elevated) with ``Badge`` status tones, and a list of ``ListItem`s` with ``Avatar`` leading and trailing ``Badge``. Pull-to-refresh via ``RNTheme.ScrollView`` ``refreshControl``. Show ``Skeleton lines={3}`` per card while loading, ``EmptyState`` with a create action when empty, ``ErrorState`` on failure, and ``useToast().showToast`` after mutations.

B7 Make an existing screen theme- and RTL-ready

Here is an existing React Native screen:

```
```tsx
{paste code}
```
```

Refactor it to Bunyan:

- Replace every ``react-native`` import with Bunyan components or ``RNTheme`` primitives.
- Replace hard-coded colours/sizes with theme tokens; replace ``flexDirection: 'row'`` + manual RTL code with ``logicalRow(direction)``.
- Replace custom buttons/inputs with ``Button``, ``IconButton``, ``Input``, ``FormField``.
- Keep behaviour identical. List every replacement you made in a table (before → after).

B8 Choose the right feedback pattern

For each of these situations, choose between ``Toast``, ``Alert``, ``Modal``, ``BottomSheet``, ``EmptyState``, ``ErrorState``, ``Skeleton`` and ``Spinner``, explain why in one sentence, and show the Bunyan code: {list of situations, e.g. "payment saved", "session about to expire", "no invoices yet", "server error loading list", "confirm delete"}

Extending & maintaining the design system

E1 Add a new component

Add a ``{ComponentName}`` component to Bunyan (``src/design-system/components/{ComponentName}/``).

Purpose: {purpose}

Props: {props with types, defaults}

Variants/states: {variants}

Follow the project checklist exactly:

1. ``{ComponentName}.tsx`` + ``index.ts``; export from ``components/index.ts``.
2. Import primitives only from ``../RNTheme`` (never ``react-native``); use ``useTheme()`` and tokens – no raw values.
3. ``memo`` + ``forwardRef`` when it wraps a native element; ref types from `RNTheme` (``ViewRef``, ``TextInputRef``).
4. RTL via ``logicalRow`` / ``logicalText``; directional icons mirror automatically.
5. Accessibility: role, label, state (``disabled``, ``selected``, ``checked``, ``expanded``, ``busy``), ≥ 44 pt target (hitSlop for compact sizes), localisable label props.
6. Respect ``exactOptionalPropertyTypes`` (spread optional props conditionally).
7. Add a story to ``AllComponents.stories.tsx``, a behaviour test in ``tests/``, and a section in ``docs/COMPONENTS.md`` (Purpose, Props, Variants, Accessibility, Example, Do/Don't, Edge cases).
8. ``npm run verify`` must pass (typecheck + jest, including the react-native import-boundary test).

E2 Add a custom Form field type

Add a ``{TypeName}`` field type for Bunyan's ``<Form>``.

Value shape: {value type}

Props: {extra config props}

1. Build the control as a normal component first (``components/{TypeName}/``), following the add-a-component checklist.
2. Augment the types: ``declare module '@bunyan/design-system' { interface FormFieldTypes { {TypeName}: {props} } }``.
3. Register it once: ``registerFormFieldType('{TypeName}', ({ field, value, setValue, setTouched, invalid, disabled, error, t, form }) => ...)``; pass ``{ wrap: false }`` only if the control renders its own label and error.
4. Map ``invalid`` to the control's error status, call ``setTouched`` on blur/selection, translate texts with ``t``.
5. Add a typecheck test (field accepted, wrong props rejected) and a runtime test with Formik + Yup.
6. Document it in `docs/FORMS.md` (value shape + props).

E3 Create a themed primitive with createThemedComponent

Create ``{Name}`` using ``createThemedComponent(RNTheme.{View|Pressable|...}, { displayName: '{Name}', baseStyle: ({ theme, direction }) => ({...}), defaultProps: ({ theme }) => ({...}) })``. Base style must use tokens only. Show usage with ``themeStyle`` and an explicit ``style`` override, and add a test asserting the theme style in dark mode (``renderWithTheme(ui, { initialPreference: 'dark' })``).

E4 Apply a brand's own tokens

Apply the `{Brand}` design tokens to Bunyan without forking it.

Tokens: {paste colours, font families, spacing, radii, breakpoints, Figma frame size}

1. Create `src/theme/brand.ts` with `createThemes({ shared, light, dark, black })` – shared for typography/spacing/radius/breakpoints, per mode for colours. Only override what differs.
2. Pass it to `- 3. If the brand uses different breakpoint names or a different design canvas, create `src/theme/responsive.ts` with `createResponsive({ breakpoints, guidelineWidth, guidelineHeight })` and export `useResponsive`, `makeStyles`, `s`, `ms` from it.
- 4. Check contrast (WCAG AA) of the new colours on their surfaces in every mode and list the ratios.
- 5. Show one screen before/after using only tokens (no raw values).

E5 Make a screen responsive

Make `{ScreenName}` work on phones, tablets and web/desktop with Bunyan's responsive API.

- Layout changes per breakpoint: use `useResponsive()` (`r.up('medium')`, `r.select({...})`) and `List` breakpoint maps for columns/spacing; cap content width on wide screens.
 - Only use `r.ms()` for headline sizes/illustrations that should grow slightly; keep body text and spacing tokens unscaled.
 - Put size-dependent styles in `makeStyles((theme, r) => ({...}))`.
 - Verify rotation and split-screen (the hook re-renders) and that touch targets stay ≥ 44 pt.
- Return the updated file and a table of what changes at each breakpoint.

E6 Add or change a design token / semantic colour

Add the semantic colour `{group}.{name}` (purpose: {purpose}).

- Add any new raw value to `tokens/primitives.ts` palette only.
- Map it in `themes/themes.ts` for light, dark and black, and extend `SemanticColors` in `themes/types.ts`.
- Check WCAG AA contrast ($\geq 4.5:1$ text, $\geq 3:1$ UI) against the surfaces it will sit on in every theme and report the ratios.
- Update `tests/tokens.test.ts` if the contract changes.

E7 Add an icon

Add the `{icon-name}` icon to `components/Icon/Icon.tsx`: extend the `IconName` union, add 24×24 stroke-only SVG children using `react-native-svg` primitives (round caps/joins, no fills), and add it to `DIRECTIONAL\_ICONS` if its meaning depends on reading direction. Update the Icon story and the docs list.

E8 Write tests for a component

Write `@testing-library/react-native` tests for `{Component}` using `renderWithTheme` from `tests/test-utils`. Cover: role and accessible name, each interactive callback, disabled/loading blocking interaction, controlled vs uncontrolled behaviour, RTL (`locale: 'ar-SA'`) styles, and dark-mode rendering. Prefer behaviour assertions over snapshots. Use `act` + fake timers for anything time-based.

E9 Review a change against Bunyan rules

Review this diff as a Bunyan maintainer:

```
```diff
{paste diff}
```
```

Check and report (file:line, severity, fix): direct `react-native` imports; raw colours/spacing; missing ally roles/labels/states; RTL mistakes (hard-coded `row-reverse`, `left`/`right`); un-localisable strings; `exactOptionalPropertyTypes` violations; missing story/test/docs; behaviour regressions. Finish with a merge recommendation.

E10 Audit the design system for bugs

Audit `src/design-system` for bugs. For each component consider: stale state when props change, timers/animations/promises not cleaned up on unmount, controlled vs uncontrolled mismatches, nested pressables, double RTL mirroring, accessibility state errors, and theme values that are invisible in dark/black mode. Output a table (component, bug, reproduction, fix) and then the patches with tests.

E11 Package and release

Prepare release `{version}`: bump `package.json`, run `npm run verify`, then `npm run pack` (runs `npm i && npm pack`; `prepack` cleans and rebuilds `dist/`). Inspect the tarball file list (`tar tzf bunyan-design-system-{version}.tgz`) to confirm `dist/`, `src/`, `docs/` and `README.md` are present and no tests or stories leak into `dist/`. Write release notes grouped as Added / Fixed / Changed.

Appendix

Type aliases

| | |
|-----------------------------------|--|
| <code>AlertTone</code> | <code>'information' 'success' 'warning' 'error'</code> |
| <code>BadgeTone</code> | <code>'neutral' 'primary' 'success' 'warning' 'error' 'information'</code> |
| <code>Breakpoints</code> | <code>Readonly<Record<string, number>></code> |
| <code>ButtonVariant</code> | <code>'primary' 'secondary' 'outline' 'ghost' 'danger' 'link'</code> |
| <code>CardVariant</code> | <code>'elevated' 'outlined' 'filled'</code> |
| <code>ComponentSize</code> | <code>'small' 'medium' 'large'</code> |
| <code>Direction</code> | <code>'ltr' 'rtl'</code> |
| <code>DisplayCalendar</code> | <code>'gregory' 'islamic-umalqura'</code> |
| <code>FeedbackStatus</code> | <code>'default' 'error' 'success'</code> |
| <code>FilePicker</code> | <code>(options: { multiple: boolean }) => Promise<PickedFile readonly PickedFile[] null undefined></code> |
| <code>FormDisplayFieldType</code> | <code>'Progress' 'ActionText'</code> |
| <code>FormFieldType</code> | <code>keyof FormFieldTypes</code> |
| <code>HeadingLevel</code> | <code>1 2 3 4 5 6</code> |
| <code>IconName</code> | <code>'alert-circle' 'arrow-down' 'arrow-down-left' 'arrow-up' 'arrow-up-right' 'backspace' 'calendar' 'check' 'chevron-down' 'chevron-left' 'chevron-right' 'close' 'eye' 'eye-off' 'info' 'search' 'success' 'warning' 'error' 'user' 'filter' 'upload' 'file' 'plus' 'minus' 'trash' 'image'</code> |
| <code>IconSize</code> | <code>'xs' 'sm' 'md' 'lg' 'xl' 'xxl'</code> |
| <code>IconTone</code> | <code>'primary' 'secondary' 'tertiary' 'inverse' 'success' 'warning' 'error' 'information'</code> |
| <code>ListFlexDirection</code> | <code>'row' 'column' 'row-reverse' 'column-reverse'</code> |
| <code>ListFlexWrap</code> | <code>'wrap' 'nowrap' 'wrap-reverse'</code> |
| <code>ListSlot</code> | <code>React.ReactNode React.ComponentType</code> |
| <code>ListSpacing</code> | <code>number keyof Theme['spacing']</code> |
| <code>LogicalAlign</code> | <code>'start' 'center' 'end'</code> |
| <code>OTPTemplateVariant</code> | <code>'bottomSheet' 'overlay' 'fullScreen'</code> |
| <code>ScrollViewRef</code> | <code>React.ComponentRef<typeof Native.ScrollView></code> |
| <code>TextInputRef</code> | <code>React.ComponentRef<typeof Native.TextInput></code> |
| <code>TextRef</code> | <code>React.ComponentRef<typeof Native.Text></code> |
| <code>TextTone</code> | <code>'primary' 'secondary' 'tertiary' 'inverse' 'link' 'error' 'success'</code> |
| <code>TextVariant</code> | <code>'body' 'bodySmall' 'caption' 'label' 'code'</code> |

| | |
|------------------------------|---|
| <code>TextWeight</code> | <code>'regular' 'medium' 'semibold' 'bold'</code> |
| <code>ThemeBreakpoint</code> | <code>keyof Theme['breakpoint']</code> |
| <code>ThemeMode</code> | <code>'light' 'dark' 'black'</code> |
| <code>ThemeOverrides</code> | <code>DeepPartial<Omit<Theme, 'mode'>></code> |
| <code>ThemePreference</code> | <code>ThemeMode 'system'</code> |
| <code>ToastTone</code> | <code>'neutral' 'success' 'warning' 'error' 'information'</code> |
| <code>Tokens</code> | <code>typeof tokens</code> |
| <code>TranslateFn</code> | <code>(key: string, params?: Record<string, unknown>) => string</code> |
| <code>ViewRef</code> | <code>React.ComponentRef<typeof Native.View></code> |

Complete runtime export list

`Accordion, ActionText, ActivityIndicator, Alert, AmountField, AmountInput, AmountWithCurrencyInput, Avatar, Badge, BottomSheet, BoxGroup, Button, Card, Checkbox, CheckboxGroup, Chip, ChipsGroup, DEFAULT_PHONE_COUNTRIES, DIRECTIONAL_ICONS, DatePicker, DateRangePicker, Divider, EmptyState, ErrorState, FileInput, Form, FormField, Heading, Icon, IconButton, Image, InlinePicker, Input, KeyboardAvoidingView, Link, List, ListItem, ListRoot, ListRow, ListRowCell, Modal, NumPad, OTPInput, OTPTemplate, OptionTiles, PasswordInput, PhoneInput, PhoneWithCountryInput, Pressable, ProgressBar, RNModal, RNSwitch, RNText, Radio, RadioGroup, RadioImageGroup, ScrollView, SearchInput, Select, Skeleton, Slider, Spinner, SwatchGroup, Switch, Tabs, Text, TextArea, TextInput, ThemeContext, ThemeProvider, ToastProvider, Tooltip, View, addBunyanYupMethods, blackTheme, borderWidth, breakpoint, colorLuminance, componentHeight, computeResponsive, controlProps, createResponsive, createTheme, createThemedComponent, createThemes, darkTheme, dateRangeSchema, defaultTranslate, flagEmoji, fontFamilyFor, formatAmount, formatFileSize, getResponsive, groupAmountText, heightForSize, horizontalPaddingForSize, iconSize, iconSizeForComponent, iconTokenForSize, isRTLLocale, isTextValue, isValidPhone, lightTheme, logicalAlignItems, logicalFlexDirection, logicalRow, logicalText, logicalTextAlign, mergeIds, mergeThemedStyles, motion, nativeDirection, needsMirroring, normalizeDigits, opacity, palette, parseAmount, radius, registerFormFieldType, resolveFieldType, resolveText, sanitizeAmountText, setDefaultFilePicker, shadow, spacing, statusBorderColor, themes, toE164, tokens, typography, useText, useTheme, useToast, validateDateRange, zIndex, { useResponsive, useBreakpoint, useResponsiveValue, makeStyles, s, vs, ms, mvs, scale, verticalScale, moderateScale, moderateVerticalScale, }`

Known limitations

- The `DatePicker` uses the native community picker; on web, supply a web implementation or replace the adapter.
- `Tooltip` wraps its child in a focusable `Pressable`; when the child is itself a button, keyboard users meet two tab stops.
- `BottomSheet` and `Modal` do not yet add bottom safe-area padding; wrap content in your safe-area view on devices with a home indicator.
- Screenshots were rendered with `react-native-web`; native platforms render system switches, spinners and pickers with their own look.